UMFST

Informatikai Kar

Premade Graph Explorer

League of Legends játékoskapcsolatok, teljesítménymutatók és gráfanalitikai kiterjesztések

Készítette: Tamási Krisztián-Erik

Marosvásárhely, 2026

Tartalomjegyzék

1 Bevezetés **8**

1.1 Problémafelvetés 8

1.2 Motiváció 8

1.3 A dolgozat célja 9

1.4 Kutatási kérdések 9

1.5 A dolgozat hozzájárulása 9

2 Problémafelvetés és motiváció **11**

2.1 Miért nem elég egy hagyományos statisztikai oldal? 11

2.2 A projekt szokatlan és érdekes volta 11

2.3 A központi szakdolgozati állítás 12

2.3.1 Hálózati pillér 12

2.3.2 Metrikai pillér 12

2.3.3 Rendszertechnikai pillér 12

2.4 Hatókör, nem-célok és őszinteségi szabályok 12

2.5 A dolgozat szerkezete 13

3 Szakirodalmi háttér **14**

3.1 Hálózatkutatás és közösségszerkezet 14

3.2 Közösségdetektálás és modularitás 14

3.3 Strukturális egyensúly és előjeles gráfok 15

3.4 Asszortativitás 15

3.5 Esport-analitika és játékos-teljesítménymérés 16

3.6 Legrövidebb utak és heurisztikus keresés 16

4 Riot API, adatforrások és adatgyűjtési stratégia **18**

4.1 A használt Riot API-végpontok logikája 18

4.1.1 Régiós routing és végpont-topológia 19

4.1.2 Collector állapotgép és premade-probe . . . 19
4.2 Gyűjtési fókusz és queue-választás . . . 20
4.3 Premade-fókusz és gyűjtési stratégiai intuíció . . . 21
4.4 Rate limit, megbízhatóság és operatív kompromisszumok . . . 21
4.5 A flexset és a soloq gyűjtési stratégiájának összehasonlítása . . . 22
4.5.1 Flexset: premade-fókuszú, apex-tier Flex Queue . . . 22
4.5.2 Soloq: teljesítmény-alapvonal, kontroll-adatkészlet . . . 23
4.5.3 Megállási feltételek és reprodukálhatóság . . . 23
4.6 Adatminőség, torzítás és óvatos interpretáció . . . 24

5 A rendszer korai változata és az eredeti pipeline 25
5.1 A korai projekt jellege . . . 25
5.2 Az első pontozási logika . . . 25
5.3 A korai gráfépítés és export . . . 26
5.4 A korábbi régiós és ország-alapú ág kialakulása . . . 26
5.5 Miért kellett a rendszert továbbépíteni? . . . 26
5.6 Miért kellett a régi szövegréteget is megtartani? . . . 27

6 A rendszerarchitektúra evolúciója 28
6.1 Áttekintés . . . 28
6.2 Miért nem egyetlen technológia? . . . 30
6.2.1 Miért maradt a C# böngésző különálló, és nem integrálódott a fő frontend-be? . . . 30
6.2.2 Elvetett architekturális alternatívák . . . 30
6.2.3 Miért maradt a Node középen? . . . 31
6.3 Evolúciós fázisok és architekturális kompromisszumok . . . 31
6.4 Több-adatkészletes architektúra . . . 31
6.4.1 A dataset mint életciklus-egység . . . 32
6.5 A jelenlegi adatállapot . . . 33
6.6 Perzisztencia és relációs modell . . . 34

7 Több-adatkészletes architektúra 35
7.1 Miért vált szükségessé a dataset-regiszter? . . . 35
7.2 A jelenlegi regiszterben szereplő adatkészletek . . . 36
7.3 A dataset-választás propagálása a backend és a Rust felé . . . 37
7.4 Őszinte állapotkép a migrációról . . . 37

8 SQLite perzisztencia és klasztermodellezés **38**
8.1 Miért maradt központi elem az SQLite? . . . 38
8.2 A `players` tábla mint gazdagított játékosprofil . . . 38
8.2.1 Sémanormalizáció és visszafelé kompatibilitás . . . 39
8.2.2 SQLite pragmák és írási kompromisszumok . . . 39
8.3 A klasztermodell és a két fő tábla szerepe . . . 39
8.4 A perzisztált klaszterek aktuális nagyságrendje . . . 40
8.5 A két klasztercsalád részletes összehasonlítása . . . 41
8.5.1 python_population . . . 41
8.5.2 rust_pathfinding . . . 41
8.5.3 A két csomád koegzisztenciájának módszertani értéke . . . 41
8.6 Miért fontos a perzisztencia a kutatási fejezetekhez? . . . 42

9 Feedscore és opscore metrikák **43**
9.1 Gyűjtési stratégia . . . 43
9.2 Normalizálás PUUID alapján . . . 43
9.3 A jelenleg implementált pontozási rendszer . . . 44
9.3.1 Artifact-szintű bontás . . . 44
9.3.2 Szerepkörérzékeny súlyozás . . . 45
9.3.3 Feedscore . . . 47
9.3.4 Percentilis alapú normalizálás . . . 48
9.4 A metrikák jelenlegi empirikus képe . . . 48
9.5 Miért volt szükség a korai képletek leváltására? . . . 49
9.6 Mit nem tud a jelenlegi pontozás? . . . 49
9.7 Kiegészítő ország-alapú elemzés . . . 49

10 Gráfépítés, klaszterezés és perzisztencia **50**
10.1 A gráf felépítésének alapelve . . . 50
10.2 Szűrés és zajcsökkentés . . . 50
10.3 Python klaszterpipeline . . . 51
10.4 Klaszterek mint első osztályú adatok . . . 51
10.5 Két klasztercsalád együttélése . . . 53
10.6 Kapcsolattípusok aggregálása a későbbi Rust-modellhez . . . 53

11 Klaszterekhez kötött ország- és régióelemzés mint korábbi modul **54**
11.1 A modul eredeti szerepe . . . 54

11.2 Miért volt ez akkor logikus irány? 54
11.3 A jelenlegi, ellenőrzött állapot értelmezése 54
11.4 Miért maradjon mégis a dolgozatban? 55

12 Rust futtatókörnyezet és útvonalkeresés 56
12.1 Miért került a futásidejű réteg Rustba? 56
12.2 A Rust gráfállapot 56
12.2.1 A gráfállapot felépítési sorrendje 57
12.3 Támogatott útvonalkereső algoritmusok 57
12.4 A hivatalos engine-spec szerepe 58
12.5 Egzakt A* és alsó becslések 58
12.5.1 A landmark-alapú ALT heurisztika 59
12.5.2 A landmark-kiválasztás stratégiája 59
12.5.3 Klaszterugrás-alapú becslés és tie-break 59
12.6 Backend API és orchestráció 60
12.7 Strukturális egyensúly mint kísérleti analitika 61
12.8 Asszortativitás mint megvalósított analitika 61
12.9 A Graph V2 artifact-pipeline és a Rust motor összefüggése 62
12.10 Kísérleti futtatók és robusztusság 62

13 Pathfinder Lab és algoritmus-összehasonlítás 63
13.1 A Pathfinder Lab szerepe a teljes rendszerben 63
13.2 A támogatott algoritmusok összehasonlítási logikája 63
13.3 Kézi vezérlés, replay és magyarázhatóság 64
13.4 Az algoritmusok részletes leírása 65
13.4.1 Szélességi keresés (BFS) 65
13.4.2 Dijkstra-algoritmus 65
13.4.3 Kétirányú keresés 66
13.4.4 A* keresés és heurisztika 66
13.5 A gráfmód és a súlyozás együttes hatása 66
13.6 Miért fontos ez túl a UI-n? 67

14 A 3D bird's-eye sphere és globális hálózati vizualizáció 68
14.1 A frontend szerepe 68
14.2 Miért nem egyetlen képernyő? 68
14.3 A 3D globális nézet 69

14.4 A bird's-eye manifest technikai tartalma . . . 69
14.5 A `flexset` mint asszociatív core-periphery játékosgráf . . . 70
14.6 Pathfinder Lab . . . 71
14.7 Assortativity és központisági oldalak . . . 72
14.8 Betweenness Centrality oldal . . . 72
14.9 Player Detail oldal . . . 73
14.10 A Graph V2 artifaktum-pipeline részletei . . . 73
14.10.1 Az artifact-csomag összetétele . . . 73
14.10.2 A summary.md mint gépi és emberi kontextus . . . 74
14.10.3 Verziókezelés és cache-érvénytelenítés . . . 74
14.11 A backend API-szerződés . . . 75
14.11.1 Az aktuális főbb végpontok . . . 75
14.11.2 Dataset-propagáció . . . 75
14.11.3 A JSON-alapú kérés-válasz protokoll . . . 75

15 A strukturális egyensúly mint módszertani határeset 76
15.1 Miért került le a fő eredmények közül? . . . 76
15.2 Az enemy-él szemantikai problémája . . . 76
15.3 Mit őriz meg a dolgozat? . . . 77
15.4 Módszertani tanulság . . . 77

16 Asszortativitási elemzés 78
16.1 Az asszortativitás szerepe a dolgozatban . . . 78
16.2 A számítási definíció . . . 78
16.2.1 Él-végponti minta kiválasztása . . . 79
16.2.2 Gráfmód-specifikus él-súlyozás . . . 79
16.2.3 Klaszteren belüli és klaszterek közötti bontások . . . 79
16.2.4 Missing value policy . . . 80
16.3 Asszortativitási eredmények — flexset . . . 80
16.4 Asszortativitási eredmények — soloq . . . 80
16.5 Mit mondanak ezek a számok? . . . 81
16.5.1 Az `opscore` robusztusabb mint a `feedscore` . . . 81
16.5.2 Gráfmód-érzékenység mint módszertani eredmény . . . 81
16.6 A randomizációs infrastruktúra helye . . . 82

17 Párhuzamos Brandes-féle betweenness centralitás 83

17.1 Miért szükséges a betweenness centralitás ebben a kontextusban? 83
17.2 Az algoritmus: Brandes 2001 83
17.3 A súlyozott mód és a költségmodell 84
17.4 Párhuzamos végrehajtás Rayonnal 84
17.5 A projection felépítése 85
17.6 Normalizálás és eredményinterpretálás 86
17.7 Centralitás a bridge-orbit hipotézis tesztelésére 86
17.8 Kísérletfuttató infrastruktúra és nullmodellek 86
17.8.1 Signed-balance sweep mint módszertani diagnosztika 86
17.8.2 Asszortativitási szignifikanciateszt 87
17.8.3 A kísérletfuttató infrastruktúra módszertani értéke 87

18 Kísérleti futtatások, validáció és eredmények 88
18.1 Verifikációs lépések 88
18.2 Az aktív állapot összefoglalása 88
18.3 A korpusz összetétele mint értelmezési háttér 89
18.4 A két klasztercsalád gyakorlati különbsége 89
18.5 Analitikai eredmények 90
18.6 A frontend és a rendszerkoherencia értékelése 90
18.7 A korábbi ország-elemzési ág helye a jelenlegi rendszerben 90

19 Rendszerszintű értékelés és megvitatás 92
19.1 Mi bizonyult a legerősebb rendszertervezési döntésnek? 92
19.2 A rendszer erősségei 93
19.3 Fenyegetések a validitásra 94
19.4 Miért értékes a projekt szakdolgozatként? 94
19.5 A megvalósult és a jövőbeli elemek szétválasztása 95

20 A két adatkészlet empirikus összehasonlítása: `flexset` versus `soloq` 96
20.1 Miért szükséges a két adatkészlet összehasonlítása? 96
20.2 A két dataset konstruktív különbségei 96
20.2.1 Queue-szemantika és kapcsolatgeneráló mechanizmus 96
20.2.2 Adatskála és meccssűrűség 97
20.3 A két dataset vizuális klaszterstruktúrájának különbsége 97
20.4 A metrikafedettség összehasonlítása 98
20.5 A teljes összehasonlítás tanulságai 98

21 Összegzés **100**
21.1 A projekt összefoglalása 100
21.2 A dolgozat védhető fő következtetései 100

22 Az adatbázis-engineering kiegészítő alprojekt: databaseproject és C# böngésző **102**
22.1 A kiegészítő alprojekt szerepe és motivációja 102
22.2 A formális relációs séma 102
22.2.1 Nézetek és összetett lekérdezések 103
22.2.2 Tárolt eljárások és triggerek 103
22.3 Az ASP.NET Core / C# adatbázis-böngésző 104
22.3.1 Adatbázis-generálás és importálás 104
22.3.2 Dataset-szelektálás 104
22.3.3 Táblabögészés, szűrés, lapozás és JSON-nézetek 104
22.3.4 Technikai architektúra 105
22.4 A kiegészítő alprojekt helye a dolgozatban 105
22.4.1 Miért valódi alprojekt, nem csak dokumentáció? 105
22.4.2 A C# böngésző mint prezentációs eszköz 105
22.4.3 A databaseproject korlátai 105

23 Jövőbeli fejlesztések **107**
23.1 Jövőbeli fejlesztési irányok 107
23.1.1 Brandes-központiság benchmarkolása és artifact-integrációja 107
23.1.2 Adatvezérelt opscore-kalibráció 107
23.1.3 Erősebb nullmodellek az asszortativitáshoz 108
23.1.4 Visszavont vagy elhalasztott irányok 108
23.2 Zárszó 108

1. fejezet

Bevezetés

1.1 Problémafelvetés

A kompetitív online játékok egyik visszatérő problémája, hogy a résztvevők nagyon erős véleményeket fogalmaznak meg egymás teljesítményéről, miközben ehhez rendszerint csak részleges, szubjektív és rövid távú benyomások állnak rendelkezésre. Egy elveszített mérkőzés után gyorsan megjelenik a bűnbakkeresés, de sokkal ritkábban történik meg az, hogy a játékosok valóban adatalapon próbálják megérteni, mi történt a meccsben, hogyan teljesítettek ők maguk, és hogyan helyezhető el ez a teljesítmény egy tágabb közösségi hálózatban.

A *League of Legends* különösen érdekes terep ehhez, mert egyszerre tartalmaz rendkívül gazdag mérkőzésadatokat, ismétlődő csapattársi és ellenfél-kapcsolatokat, valamint olyan társas mintázatokat, amelyek gráfként ábrázolhatók és elemezhetők. Ha egy játékos hosszabb időn keresztül ugyanazokkal az emberekkel kerül egy meccsbe – akár szövetségesként, akár ellenfélként –, akkor ebből egy olyan kapcsolati háló építhető fel, amely túlmutat az egyetlen mérkőzés pillanatnyi statisztikáin.

Ebből a felismerésből indult a dolgozat: nem egyszerűen egy újabb meccsnéző vagy statisztikai felületet akartam készíteni, hanem egy olyan rendszert, amely a valós mérkőzéstörténetből játékoshálót épít, ezt a hálót gráfelméleti és rendszerszintű szemlélettel elemzi, majd az eredményeket egy vizuálisan erős, ugyanakkor védhető frontend felületen mutatja meg.

1.2 Motiváció

A projekt eredeti motivációja személyes és technikai egyszerre. Játékosként újra és újra felmerült bennem a kérdés, hogy vajon a visszatérő negatív tapasztalatok mennyire egyediek, mennyire mintázatszerűek, és egyáltalán látszik-e valami abból a hálózatból, amelynek részeként ezek a meccsek zajlanak. Informatikai oldalról pedig kifejezetten vonzott az a gondolat, hogy a Riot API által biztosított részletes meccsadatokból nemcsak táblázatokat, hanem gráfokat, klasztereket, útvonalakat és teljesítményalapú hálózati mutatókat is lehessen kinyerni.

A projekt időközben jóval nagyobb rendszerré nőtt, mint egy egyszerű adatvizualizáció. A jelenlegi állapotban már külön adatgyűjtő réteg, normalizált pontszámítási pipeline, Python-alapú gráfépítés és klaszterezés, SQLite perzisztencia, Rust futtatókörnyezet, többoldalas React frontend, valamint kutatási jellegű gráfanalitikai modulok dolgoznak együtt.

1.3 A dolgozat célja

A dolgozat célja egy olyan, valós mérkőzésadatokra épülő, többkomponensű rendszer bemutatása, amely:

- League of Legends meccsekből játékosok közötti kapcsolatokat gyűjt és tisztít,
- a játékosokhoz szerepkör-érzékeny teljesítménymutatókat rendel,
- az ismétlődő együtt- és egymás ellen játszásból gráfot épít,
- a gráfot klaszterekre, útvonalakra, core-periphery mintázatokra és kutatási célú metrikus kapcsolatokra bontja,
- a számításigényes részeket determinisztikus, exportálható és újrafuttatható módon kezeli,
- az eredményeket interaktív, de nem UI-első szemlélettel mutatja meg.

A hangsúly tudatosan azon van, hogy a rendszer ne csak érdekes látvány legyen, hanem szakdolgozati szempontból is védhető legyen: világos adatútvonalakkal, expliciten dokumentált döntésekkel, reprodukálható kimenetekkel és óvatos interpretációval.

1.4 Kutatási kérdések

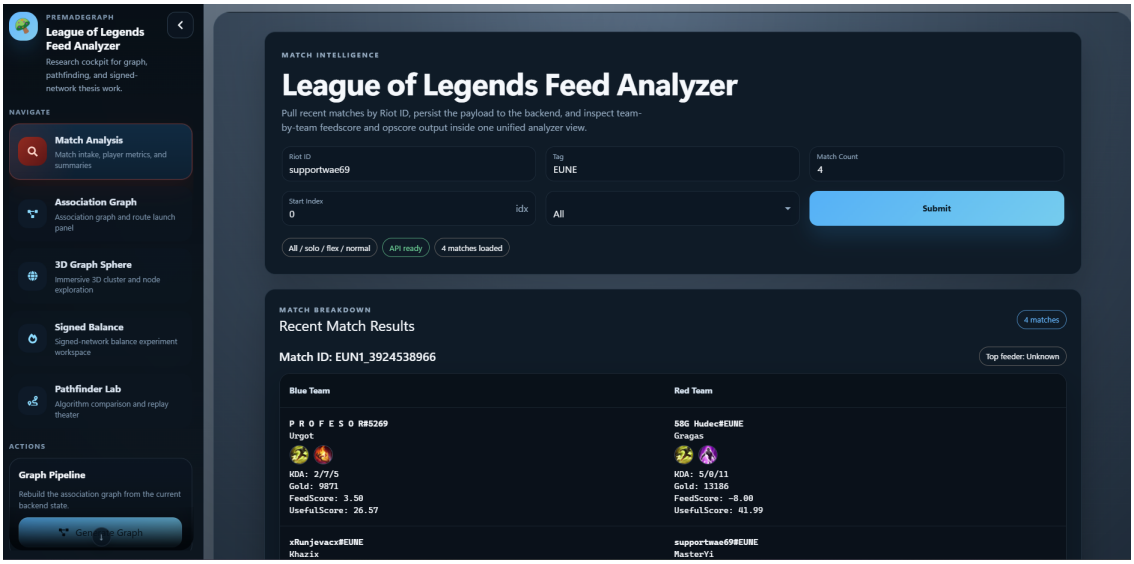
A jelen dolgozatban a következő kutatási kérdésekre keresem a választ:

- Hogyan építhető fel valós League of Legends mérkőzéstörténetből egy olyan játékosháló, amely nem pusztán az együtt játszást, hanem az ellenfél-kapcsolatokat is megőrzi?
- Milyen módon lehet a játékosok teljesítményét úgy pontozni, hogy az figyelembe vegye a tényleges szerepkört, de közben megmaradjon értelmezhetőnek?
- Mennyiben indokolt a rendszer különböző rétegeit Python, SQLite, Node.js, Rust és React komponensekre bontani?
- Mutat-e a jelenlegi gráf pozitív asszortativitást a `opscore` és `feedscore` tekintetében?
- Miben különbözik a `flexset` és a `soloq` kapcsolati szerkezete, és mennyiben írható le a `flexset` asszociatív core-periphery játékosgráfként?

1.5 A dolgozat hozzájárulása

A dolgozat gyakorlati és szakmai hozzájárulásai a jelenlegi implementáció alapján a következők:

- több-adatkészletes, regiszter-alapú mérkőzésgyűjtési és feldolgozási architektúra;
- szerepkörérzékeny, adatbázis-alapú `opscore`/`feedscore` normalizáció;
- Python oldali gráfépítés és klaszterezés SQLite perzisztenciával;
- Rust oldali útvonalkeresés BFS, Dijkstra, kétirányú keresés, egzakt A* és Brandes-központisági támogatással;
- `flexset` és `soloq` dataset-összehasonlítás asszociatív gráfértelmezéssel;
- játékosteljesítményre számolt asszortativitási elemzés több gráfmódban;
- előre kiszámított 3D globális gráfnézet és többoldalas, kutatásbarát frontend.



1.1. ábra: A jelenlegi frontend egyik belépési pontja: meccselemzési és adatgyűjtési felület

2. fejezet

Problémafelvetés és motiváció

2.1 Miért nem elég egy hagyományos statisztikai oldal?

A legtöbb, játékosoknak szánt statisztikai felület egyetlen mérkőzésre, egyetlen rangsorra vagy néhány alapmutatóra szűkíti le az értelmezést. Ez önmagában hasznos lehet, de nem válaszolja meg azt a kérdést, amely engem a projekt elejétől fogva érdekelt: vajon a mérkőzések mögött kirajzolódik-e egy tartós társas szerkezet, és ha igen, ez a szerkezet hogyan kapcsolódik a teljesítményhez?

Ebből a szempontból a *Premade Graph* lényegében két problémát kezel egyszerre. Az első probléma adatmérnöki: hogyan lehet nyers Riot API válaszokból olyan hosszabb élettartamú adatkorpuszt felépíteni, amelyben a játékosok azonosítása, a meccsek tárolása, a pontszámítás és a gráfprojekció újrafuttatható módon történik. A második probléma kutatási: hogyan lehet ebből a korpuszból olyan állításokat megfogalmazni, amelyek nem csak látványosak, hanem meg is védhetők.

Ez az oka annak, hogy a dolgozat fókusza tudatosan nem egyetlen UI-oldal vagy egyetlen algoritmus. A projekt értéke éppen abban áll, hogy az adatgyűjtés, a tisztítás, a pontszámítás, a gráfépítés, a klaszterezés, a futásidejű analitika és a vizualizáció ugyanannak a rendszernek az egymásra épülő rétegei.

2.2 A projekt szokatlan és érdekes volta

A szakdolgozat témája azért különösen érdekes, mert a kiinduló adathalmaz nem egyszerű játékoslista, hanem ismétlődő interakciós háló. A rendszer ugyanazt a két játékost nemcsak egyszeri ellenfelekként látja, hanem potenciális visszatérő szövetségesként, visszatérő ellenfélként, klasztertagként, hídcsúcsként vagy egy nagyobb globális hálózat részeként is.

Ez több okból ad erős szakdolgozati alapot:

- a League of Legends mérkőzésadatai elég gazdagok ahhoz, hogy valódi teljesítményprofilok legyenek számíthatók;
- a visszatérő együttjátszás és egymás elleni előfordulás gráfelméleti szempontból értelmezhető kapcsolatokat hoz létre;
- a szövetségesi és ellenfél-jelleg együttese miatt a projekt nem csak súlyozott, hanem több-

féle kapcsolatprojekcióként is vizsgálható;
- a rendszer implementációs oldalon is érdekes, mert több nyelv és futtatókörnyezet közötti felelősségmegosztást kell megindokolni.

Ez a kombináció ritkább, mint elsőre látszik. Sok játékelemző projekt vagy a vizuális oldalnál áll meg, vagy csak egy-egy táblázatos mutatóig jut el. Itt viszont ugyanannak a rendszernek része a nyers adatgyűjtés, a szerepkörérzékeny score-réteg, a klaszterezés, az útvonalkeresés, az asszortativitási analitika, a Brandes-központiság és a 3D globális megjelenítés.

2.3 A központi szakdolgozati állítás

A dolgozat fő tézise az, hogy *a valós mérkőzéstörténetből felépített League of Legends játékosháló egyszerre kezelhető mint asszociatív játékosgráf, mint teljesítményprofilokkal dúsított adatstruktúra, és mint rendszertechnikai szempontból komoly, többnyelvű analitikai platform*. Ennek a tézisnek három pillére van.

2.3.1 Hálózati pillér

A meccsekből olyan gráf építhető, amelyben a játékosok közötti ismétlődő együttjátszás és egymás elleni előfordulás már nem véletlenszerű zajként, hanem mérhető szerkezeti mintázatként jelenik meg. A `flexset` esetében ennek legerősebb olvasata az asszociatív core-periphery szerkezet: a központi régióban hídgazdag, visszatérő ally-csoportok, a periférián pedig sok kis, gyengén kapcsolt lokális csoport jelenik meg. Ez teszi lehetővé a klaszterezést, az útvonalkeresést, az asszortativitást és a későbbi centralitási validálást.

2.3.2 Metrikai pillér

A játékosokhoz rendelhető olyan `feedscore` és `opscore`, amely nem tökéletes vagy végleges igazság, de elég értelmezhető és következetes ahhoz, hogy későbbi gráfanalitikai bemenet legyen. A szerepkörérzékeny súlyozás és a percentilis-alapú normalizáció itt nem kozmetikai döntés, hanem a teljes későbbi elemzés előfeltétele.

2.3.3 Rendszertechnikai pillér

A projekt a jelenlegi állapotában már túlmutat azon, amit egyetlen script vagy egyetlen technológia kényelmesen elbírna. A Python, az SQLite, a Node/Express, a Rust és a React együttese nem véletlen felhalmozódás, hanem olyan architekturális válasz, amely a különböző részfeladatok eltérő természetéből következik. Ezt a dolgozatban külön, döntési és kompromisszum-alapú szemlélettel fogom indokolni.

2.4 Hatókör, nem-célok és őszinteségi szabályok

A projekt mérete könnyen csábíthatna arra, hogy minden meglévő ötletet kész eredményként mutassak be. Ezt a dolgozatban tudatosan kerülöm. Három szabályt tekintek fontosnak.

- Amit a kódbázisban jelenleg ténylegesen megvalósultnak és ellenőrizhetőnek találtam, azt kész állapotként írom le.
- Amit a repository megőriz korábbi kísérleti ágként, de a jelenlegi adatsnapshotban nincs érdemi kimenete, azt korábbi vagy kiegészítő modulként kezelem.
- Amit a fejlesztési dokumentáció csak jövőbeli célként fogalmaz meg, azt nem nevezem kész feature-nek.

Ennek a szabálynak különösen nagy jelentősége van a következő területeken:

- a klaszterekhez kötött ország- és régióelemzés kódja megvan, de a jelenlegi ellenőrzött adatbázis-pillanatképben nem tartalmaz kitöltött országcímkéket;
- a több-adatkészletes architektúra implementált, de a történetileg gazdag, régi `playersrefined.db` állomány migrációja a mostani repository-állapotban még nem teljesen zárult le;
- a már implementált Brandes-központiság teljes valós adatkészletes benchmarkolása és az erősebb permutációs nullmodellek jelenleg további kutatási irány.

2.5 A dolgozat szerkezete

A dolgozat logikája a projekt fejlődését követi. Előbb bemutatom a problémafelvetést és a szakirodalmi hátteret, majd a Riot API-ra épülő adatforrásokat és a korai pipeline-t. Ezután térek rá a pontozási rendszerre, a gráfépítésre és a régi ország-alapú elemzési ágra. A középső fejezetek az architektúra evolúcióját, a több-adatkészletes modellt, az SQLite perzisztenciát és a Rust futtatókörnyezetet tárgyalják. Végül külön fejezetekben mutatom be a Pathfinder Labot, a 3D bird's-eye sphere nézetet, az asszortativitási és központisági elemzéseket, valamint a strukturális egyensúly módszertani visszaminősítését, majd a validációt, a rendszerszintű értékelést és a jövőbeli fejlesztéseket.

3. fejezet

Szakirodalmi háttér

3.1 Hálózatkutatás és közösségszerkezet

A gráfok társas kapcsolatok modellezésében régóta központi szerepet játszanak. A videojátékos közösségek esetében különösen izgalmas kérdés, hogy a játékosok közötti ismétlődő interakciók mennyiben alkotnak felismerhető struktúrákat, szubközösségeket, hidakat és lokális mintázatokat. Girvan és Newman klasszikus munkája rámutatott arra, hogy a hálózat élei nem pusztán kapcsolódási elemek, hanem közösségszerkezetet leíró, eltérő fontosságú viszonyok is lehetnek [9]. Fortunato átfogó áttekintése pedig jól összefoglalja, miért központi probléma a közösségdetektálás a modern hálózatelemzésben [10].

A jelen projektben a közösségfogalom nem öncélú. Azért fontos, mert a többször ismétlődő kapcsolatokra épülő játékoscsoportok:

- értelmezhetőbbé teszik a nagyméretű hálózatot,
- segítenek elkülöníteni a valós kapcsolati magokat a zajtól,
- támaszt nyújtanak a későbbi útvonal- és analitikai számításoknak.

3.2 Közösségdetektálás és modularitás

A Python oldali populációs klaszterezés jelenleg a NetworkX `greedy_modularity_communities` eljárását használja, amely a Clauset–Newman–Moore féle mohó modularitásmaximalizálási megközelítésre épül [12, 5]. Ez a választás tudatos kompromisszum: nem a leglátványosabb, hanem a jelenlegi adatkészlet és pipeline mellett a legegyszerűbben magyarázható és stabilan reprodukálható megoldás. Fontos megjegyezni, hogy a futásidejű gráfállapotot – amelyen az útvonalkeresés és az analitikák futnak – a Rust motor önállóan, a meccs-JSON fájlokból és az SQLite-adatbázisból építi fel; a Python pipeline szerepe tehát elsősorban a populációs közösségek perzisztálása, nem a teljes gráfreprezentáció előállítása.

A dolgozatban ezért nem azt állítom, hogy ez az egyetlen vagy minden helyzetben legjobb közösségdetektáló eljárás, hanem azt, hogy a jelenlegi rendszerben ez adja a legvédhetőbb kiindulópontot a közösségek strukturált tárolására és vizsgálatára.

Ugyanakkor módszertanilag fontos kimondani a modularitás-optimalizáció ismert korlátait is. Fortunato és Barthelemy klasszikus eredménye szerint a modularitásra építő közösségdetektálás

felbontási korlátba ütközhet, vagyis a kisebb, de valós szerkezetek egy része a globális optimum érdekében összeolvadhat [11]. Ez a jelen projektben azért releváns, mert a `python_population` klasztereket nem szabad ontológiai „igazi közösségként” kezelni. Sokkal pontosabb azt mondani, hogy a választott gráf, súlymodell és heurisztika mellett ezek adják a legjobb, reprodukálható populációs közösségképet.

Ez a szakirodalmi háttér azt is megvilágítja, miért védhető a két klasztercsalád együttélése. Ha a közösségdetektálás eleve input- és célfüggő feladat, akkor nem meglepő, hogy a populációs Python-klaszterek és a futásidejű Rust-komponensek nem esnek egybe. A dolgozat szempontjából ez nem hiba, hanem annak jele, hogy a rendszer külön optimalizálja a népességi értelmezhetőséget és a keresési használhatóságot.

3.3 Strukturális egyensúly és előjeles gráfok

Az előjeles kapcsolatok vizsgálatának klasszikus alapját Heider egyensúlyelmélete [6], illetve Cartwright és Harary gráfelméleti általánosítása adja [7]. Az alapgondolat intuitív: a pozitív és negatív kapcsolatokból felépülő triádok egy része stabilabb, más része feszültebb szerkezetet jelez.

Ez a dolgozat szempontjából háttérként fontos, mert a projekt adatmodellje nemcsak azt tudja megmondani, hogy két játékos hányszor került kapcsolatba, hanem azt is, hogy ez a kapcsolat inkább szövetségesi vagy inkább ellenfél-jellegű volt. Így a gráf formálisan projektálható egy előjeles hálóvá, ahol:

- a domináns `ally` kapcsolat pozitív élként,
- a domináns `enemy` kapcsolat negatív élként

jelenik meg.

A modern signed-network irodalom azért is hasznos háttér, mert rámutat: az előjeles hálók viselkedése nem csak elméleti játék. Leskovec és munkatársai nagyméretű online hálózatokon mutatták meg, hogy a pozitív és negatív élek kombinációja valódi szerkezeti mintázatokat hoz létre [17], míg későbbi összefoglalók és empirikus vizsgálatok azt hangsúlyozzák, hogy a strukturális egyensúlyt mindig a konkrét projekciós szabályok és adatkeletkezési mód fényében kell értelmezni [18, 19]. A jelen dolgozatban éppen ez a korlát vált döntővé: az `enemyWeight` nem elég megbízható negatív társas kötés, ezért a strukturális egyensúly nem fő empirikus eredményként, hanem módszertani határesetként jelenik meg.

3.4 Asszortativitás

Newman munkája az asszortatív keveredést olyan hálózati jelenségként írja le, amikor a kapcsolódó csomópontok hasonló vagy éppen eltérő tulajdonságúak [8]. A jelen projektben ezt a gondolatot a játékosok teljesítménymutatóira alkalmazom: nem azt kérdezem, hogy a magas fokszámú csúcsok kapcsolódnak-e egymáshoz, hanem azt, hogy a gráfban összekapcsolt játékosok hasonló `opscore` vagy `feedscore` értékeket hordoznak-e.

Ez azért különösen hasznos, mert:

- könnyen értelmezhető,

- számszerűsíthető,
- a gráf jelentésétől függően más és más eredményt adhat.

Éppen ezért a dolgozatban két gráfmódot is megkülönböztetek:

- `social-path`: csak a pozitív, szövetségesi támaszú kapcsolatokra épít,
- `battle-path`: az összes ismétlődő együtt- vagy egymás ellen játszást figyelembe veszi.

3.5 Esport-analitika és játékos-teljesítménymérés

A League of Legends és az általánosabb MOBA-játékok esport-analitikája önálló kutatási területté vált. Mora-Cantallops és Sicilia átfogó irodalmi áttekintése megmutatja, hogy a MOBA-játékok egyedi kombinációt kínálnak verseny, együttműködés, játékosélmény és társas interakció között, és ez a kombináció különlegesen gazdag adatforrást teremt empirikus vizsgálatokhoz [22].

A játékos-teljesítmény modellezése Novak és munkatársai munkájában is megjelenik: profi League of Legends meccsekre kidolgozott teljesítménymodellek rámutatnak, hogy a domainspecifikus változók és a strukturált adatfeldolgozás fontosabb, mint az általános statisztikai modellek [24]. Ez erősíti a jelen projekt szerepkörérzékeny pontozási megközelítésének indokoltságát.

Bahrololloomi és munkatársai a hasonló adatgyűjtési stratégiát, amelyet a jelen projekt is alkalmaz — kiindulópontból való mintavétel és ismétlődő meccskapcsolatok felderítése — gépi tanulásos keretben vizsgálják [23].

A csapathálózat és teljesítmény összefüggése Sánchez-González és González-Bailón munkájában is megjelenik, ahol profi LoL csapatok hálózati struktúráját kapcsolják össze a csapathatékonysággal [26]. Ez megalapozza azt a nézetet, hogy a jelen projektben használt klaszter- és kapcsolatstruktúra nem puszta vizualizáció, hanem potenciálisan prediktív erejű analitikai egység.

Mindezek alapján a jelen projekt a meglévő esport-analitikai irodalom egy természetes empirikus kiterjesztése: nem profi, hanem amatőr/félprofesszionális rangsorolt játékos-hálózatokat vizsgál, és nem egyetlen meccsre, hanem ismétlődő kapcsolatokra épít.

3.6 Legrövidebb utak és heurisztikus keresés

A gráfok másik természetes vizsgálati iránya az útvonalak keresése. A klasszikus legrövidebb út problémát Dijkstra alapozta meg [13], míg az A* keresés formális alapját Hart, Nilsson és Raphael adták meg [14]. A jelen projektben azért vált ez fontos kérdéssé, mert a játékosháló nemcsak statikus objektumként érdekes, hanem úgy is, mint egy olyan kapcsolati tér, ahol két játékos között különböző „ismeretségi láncok” húzódhatnak.

A Rust rétegben bevezetett egzakt A* implementációhoz a rendszer ALT-szerű alsó becsléseket is használ, amelyeket a Goldberg–Harrelson-féle irány tovább inspirált [15]. A lényeg itt sem az, hogy a dolgozat egy új legrövidebb út algoritmust találna fel, hanem az, hogy a játékosháló nagyságrendjéhez és a demo-igényekhez illeszkedően stabil, védhető és gyors futtatókörnyezetet alakítson ki.

Szakdolgozati szempontból itt különösen fontos különbséget tenni „gyorsnak tűnő” és valóban helyes heurisztikus keresés között. Sok gyakorlati rendszer a képernyőn látható távolságot is heurisztikaként használja, ami látványos lehet, de nem feltétlenül admisszibilis. A jelen projektben a helyességi garancia nem a layoutból, hanem a gráfra illesztett alsó becslésekből származik. A térbeli távolság legfeljebb tie-break szerepet kap. Ezzel a rendszer jobban igazodik a klasszikus A* elméleti követelményeihez, és elkerüli, hogy a vizualizációs koordináták összekeveredjenek a kapcsolati költséggel.

4. fejezet

Riot API, adatforrások és adatgyűjtési stratégia

A rendszer elsődleges adatforrása a Riot Developer Portal és a kapcsolódó League of Legends API-készlet [1, 2]. Ezek lehetővé teszik:

- játékosfiókok azonosítását,
- meccslisták lekérdezését,
- részletes meccs-JSON-ok letöltését,
- résztvevői statisztikák és szerepkörök feldolgozását.

Az API ugyanakkor nem korlátlan: a fejlesztői kulcsok szigorú sebességkorlátokkal működnek, és ezek az architektúrát is befolyásolják. Ez magyarázza, hogy a projektben miért különül el világosan az adatgyűjtő, a normalizáló, a gráfépítő és a futtatókörnyezeti réteg. A cél ugyanis nem az, hogy minden egyszerre, ugyanabban a kérésben történjen, hanem az, hogy a rendszer különálló, újrafuttatható lépésekből álljon össze.

4.1 A használt Riot API-végpontok logikája

A jelenlegi projekt a Riot API több rétegét használja, de a legfontosabb tervezési döntés az volt, hogy a rendszer központi azonosítójává a `puuid` váljon. Ez a döntés végigvonul az egész kódbázison. A felhasználóbarát név és tag ugyan hasznos a frontendben, de az adatbázis és a gráfépítés szempontjából az azonosíthatóság elsődleges.

A pipeline logikailag az alábbi lépésekre bontható:

1. egy kiinduló játékos vagy seed-lista alapján a rendszer fiókinformációt kér le;
2. a fiókból meccsazonosító-listát szerez;
3. a meccsazonosítókhoz letölti a részletes match JSON-okat;
4. a résztvevői sorokból `puuid`-alapú játékosprofilt és kapcsolati adatot képez;
5. az eredményt datasethez kötve perzisztálja.

Ez azért szerencsés, mert a névváltozások, régiós sajátosságok és meccslistás lekérdezések nem keverednek össze a későbbi gráf- és pontszámítási lépésekkel. A `puuid` egyszerre stabil összekötő kulcs a nyers meccsadat, a játékosprofil és a gráfcsúcs között.

4.1.1 Régiós routing és végpont-topológia

A konkrét implementáció szintjén ez a logika azt is jelenti, hogy a meccslisták és a részletes meccsadatok a region routing logikát követik. A jelenlegi collector a `match-v5` végpontokat az `europe.api.riotgames.com` címen keresztül hívja. Ez első látásra technikai apróságnak tűnhet, de a teljes adatgyűjtési megbízhatóság múlik rajta: hibás routing mellett a rendszer úgy tűnhet működőképesnek, hogy közben a lekérdezések egy része konzisztensen elbukik.

4.1. táblázat: A gyűjtőréteg fő végpontjai és szerepük

Végpontcsalád vagy réteg	Szerep a pipeline-ban
Fiók- és seed-feloldás	Kiinduló játékosok azonosítása és stabil kulcshoz kötése
`/lol/match/v5/matches/by-puuid/{puuid}/ids`	Meccsazonosítók lekérése egy adott játékoshoz
`/lol/match/v5/matches/{matchId}`	A meccs-JSON letöltése minden résztvevő adattal
Lokális normalizálás	A letöltött JSON-ból PUUID-alapú játékosprofil és későbbi gráfinput képzése

Az architekturális tanulság az, hogy a külső, kvótás réteget minél hamarabb érdemes lokális állapottá alakítani. A rendszer ezért a külső API-hívásokat csak a nyers meccs-JSON-ig viszi, utána a feldolgozás már lokális fájlokon és adatbázison történik.

4.1.2 Collector állapotgép és premade-probe

A `backend/match_collector.py` vizsgálata alapján a collector nem egyszerű lineáris letöltőszkript, hanem paraméterezhető állapotgép. A konfiguráció tartalmazza többek között a `datasetId`, `rawDbPath`, `matchesDir`, `collectorMode`, `queueType`, `matchesPerPlayer`, `maxIterations`, `probeMatchCount` és `minimumPremadeRepeats` mezőket. Ennek következtében a gyűjtés explicit módon szabályozható, újrafuttatható és különböző célokra hangolható.

A tipikus futás így írható le:

1. a backend kijelöli az aktív datasetet és előkészíti az útvonalakat;
2. a collector véletlen vagy konkrét `puuid` alapján jelölt játékost választ;
3. rövid meccslista-lekéréssel premade-probe történik;
4. ha a jelölt elég ismétlődő csapattársi mintázatot mutat, teljesebb gyűjtés indul;
5. a letöltött meccsek eseményes logolás mellett fájlba kerülnek;
6. az eredmény később normalizáló és gráfépítő pipeline-ba lép tovább.

Ez a részlet azért fontos, mert a premade-fókusz nem csak a későbbi gráfelemzés szintjén jelenik meg, hanem már a mintavételi mechanizmusban is. A rendszer tehát nem utólag próbálja kiszűrni a visszatérő kapcsolatok jelentőségét, hanem a gyűjtési stratégiát is erre hangolja.

[PLACEHOLDER]

Collector állapotgép — rate limit kezelés, checkpointing, seed-terjeszkedés és queue-szűrés

4.1. ábra: A collector paraméterezhető állapotgépének folyamatábrája

4.2 Gyűjtési fókusz és queue-választás

A rendszer nem a Riot teljes játékuniverzumát próbálja egyetlen adatfolyamba összesöpörni, hanem tudatosan olyan queue-típusokból építkezik, amelyekben az ismétlődő kapcsolatok és a premade-jelleg reálisabban megjelenhet. Ez az oka annak, hogy a repository dokumentációja és gyűjtőszkriptjei külön foglalkoznak az EUNE és a Master/SoloQ jellegű mintákkal, illetve a gyors tesztadatsorokkal.

A projekt korai adatgyűjtési fázisában a `default` dataset vegyes queue-tartalmat gyűjtött. A 2026. április 24-én ellenőrzött állapotban a queue-eloszlás a következő volt (ez a `default` datasetre vonatkozik, nem az aktív `soloq`/`flexset` datasetekre):

4.2. táblázat: Queue-eloszlás az aktív `default` dataseten

Queue ID	Darabszám	Értelmezés
420	1371	Ranked Solo/Duo
440	1351	Ranked Flex
450	358	ARAM
400	313	Normal Draft
900	45	URF / rotációs mód
480	20	Normal Blind
700	9	Clash
740	2	Intro/AI jellegű mód
870	2	Egyéb speciális queue

Ez két fontos dolgot jelez. Egyrészt a korpusz zöme olyan játékmódból jön, ahol a kompetitív ismétlődés reális. Másrészt a dataset vegyes: a rangsorolt játékon kívül ARAM és normál

draft elemek is bekerültek. Emiatt a dolgozatban különösen fontos, hogy a gráfot ne „barátsági gráfként”, hanem ismétlődő játékkapcsolati hálóként kezeljem.

4.3. táblázat: Térképeloszlás az aktív `default` dataseten

Map ID	Darabszám	Értelmezés
11	3113	Summoner’s Rift
12	358	Howling Abyss

A map-eloszlás azt is mutatja, hogy a korpusz döntően a klasszikus Summoner’s Rift környezetre épül. Ez azért lényeges, mert a szerepkörök, az útvonalkeresési interpretáció és a teljesítménymutatók is ebben a közegben a legjobban értelmezhetők.

4.3 Premade-fókusz és gyűjtési stratégiai intuíció

A projekt mögött végig ott állt az a gyakorlati intuíció, hogy az igazán érdekes kapcsolatok nem az egyszeri, véletlenszerű találkozásokból születnek, hanem a visszatérő együtt- vagy egymás ellen játszásból. Emiatt a gyűjtőszkriptek és a dokumentáció egy része kifejezetten premade-szerű vagy ismétlődő kapcsolatok felderítésére van hangolva.

Ez a szemlélet a teljes későbbi rendszerre hatással volt:

- a gráfépítésben élküszöb jelent meg;
- az asszortativitás és a központisági elemzés az ismétlődő kapcsolatokat tekinti komoly jelöltnek;
- a pathfinder-költségek is a kapcsolaterősséget veszik figyelembe.

Másképp fogalmazva: a gyűjtési stratégia nem előszoba, hanem a későbbi gráfjelentés egyik meghatározó forrása.

Ez a döntés ugyanakkor tudatos torzítást is bevezet. Ha a gyűjtés a visszatérő, premade-szerű kapcsolatokat részesíti előnyben, akkor a korpusz kevésbé reprezentálja az egyszeri és lazább találkozásokat. A jelen dolgozat céljai mellett ez elfogadható kompromisszum, mert a kutatási kérdés középpontjában éppen a tartósabb kapcsolati magok állnak. A háló tehát nem a teljes populáció semleges lenyomata, hanem az ismétlődő interakciók felé célzottan dúsított részvilág.

4.4 Rate limit, megbízhatóság és operatív kompromisszumok

A Riot API használata az architektúrában nem pusztán külső függőség, hanem korlátozó tényező is. A fejlesztői kulcsok lekérdezésszámhoz kötöttek, ezért a gyűjtőrétegben explicit sebességkorlátkezelés, várakozási logika és részfeladatokra bontás jelent meg. A dokumentációban rögzített rate-limit elemzés ezt nem adminisztratív részletként, hanem rendszertervezési kényszerként kezeli.

Ennek közvetlen következményei:

- a meccsletöltés nem kérésenkénti frontend-akcióként, hanem háttérfolyamatként fut;

- a nyers adatállapotot érdemes lokálisan cache-elni;
- a grafikus és analitikai oldal nem függhet attól, hogy egy élő API-lekérés éppen sikeres-e.

Ez a döntés később az egész rendszer egyik alapelve lett: *a demo- és kutatási réteg a már összegyűjtött korpuszra épül, nem online API-kérésekre*.

Az implementáció szintjén ez több konkrét védelmet jelent. A collector saját másodpercenkénti várakozást alkalmaz, külön kétperces ablakban számlálja a kéréseket, `HTTP 429` esetén harminc másodpercet vár, majd újrapróbálkozik, és a már elmentett meccseket fájllétezés alapján kihagyja. Ez a működés idempotenssé teszi a letöltést: egy megszakadt vagy újraindított futás nem írja felül feleslegesen a korábbi állományokat, és nem épít újabb bizonytalanságot a nyers korpuszba.

4.4. táblázat: A collector fő megbízhatósági mechanizmusai

Mechanizmus	**Funkció**
Saját `requestsPerSecond` késleltetés	A rövid távú API-terhelés kontrollja
Kétperces számláló	Illeszkedés a Riot kulcshasználati limitjeihez
`429` újrapróbálás	Robusztusabb működés átmeneti limitütközéskor
Létező meccsfájl kihagyása	Duplikált mentés és újra-letöltés elkerülése
Eseményes stdout log	Backend- és frontend-oldali állapotkövetés támogatása

Ezek a döntések nem puszta üzemeltetési kényelmi funkciók. A reprodukálható számítógépes kutatás egyik alapelve, hogy az eredményekhez vezető lépések nyomon követhetők és lehetőleg automatizálhatók legyenek [20]. A collector parametrizálhatósága, eseményes logolása és idempotens mentési logikája pontosan ezt a célt szolgálja.

4.5 A flexset és a soloq gyűjtési stratégiájának összehasonlítása

A két kutatási dataset nem véletlenszerűen jött létre, hanem eltérő gyűjtési stratégiák eredménye. Ez a különbség az analitikai eredmények értelmezéséhez is nélkülözhetetlen.

4.5.1 Flexset: premade-fókuszú, apex-tier Flex Queue

A `flexset` dataset gyűjtési konfigurációja a következő főbb stratégiai döntéseken alapul:

- **Queue-szűrés**: kizárólag 440-es queue ID-jű (Ranked Flex) meccsek;
- **Seed-populáció**: EUNE Master+ ranglista-pozíciójú játékosok;
- **Premade-probe**: a collector egy rövid meccslista-minta alapján ellenőrzi, hogy a jelölt játékos rendelkezik-e elegendő ismétlődő csapattársi mintázattal. Ha igen, a teljes meccsgyűjtés megkezdődik;

- **Minimális premade ismétlések**: a `minimumPremadeRepeats` paraméter azokat a seedelési jelölteket vonja be, amelyeknél az ismétlődő kapcsolatok valószínűsége meghaladja a küszöböt;
- **Seed-terjeszkedés**: az egyes játékosok feldolgozásakor a rendszer a csapattársaik azonosítóit is felveszi a gyűjtési sorba.

Ez a stratégia egyfajta *premade-központú snowball sampling*: a már gyűjtött játékosok szomszédain keresztül bővül a mintakör. Az eredmény egy erősen összefüggő, visszatérő kapcsolatokban gazdag szociális háló.

4.5.2 Soloq: teljesítmény-alapvonal, kontroll-adatkészlet

A `soloq` dataset gyűjtési konfigurációja szándékosan más fókusszal rendelkezik:

- **Queue-szűrés**: kizárólag 420-as queue ID-jű (Ranked Solo/Duo) meccsek;
- **Seed-populáció**: EUNE Master+ Solo/Duo Queue ranglistán szereplő játékosok — ez *nem azonos* a flexset seed-populációjával, amely a Flex Queue ranglistájára épül; a két ranglista eltérő mechanikán alapul, és eltérő játékosbázist hoz létre;
- **Premade-probe**: itt a probe-logika *nem* szűr premade-jelenlét alapján, hanem elsősorban teljesítményadat-elérhetőség alapján;
- **Kisebb korpusz**: a konfigurált korlátok 2000 meccs körül zárnak le (vs. flexset 4980).

A soloq gyűjtés tehát *nem premade-ra optimalizált*: a Solo/Duo Queue nem ösztönzi az 5-fős premade csapatokat, és a kollekciós stratégia sem keresi aktívan a visszatérő szövetségesi mintákat. Ez teszi a soloq-ot alkalmas kontrollalappá.

4.5. táblázat: A két dataset gyűjtési stratégiájának összehasonlítása

Dimenzió	**flexset**	**soloq**
Queue ID	440 (Flex)	420 (Solo/Duo)
Premade-probe	Igen, ismétlések alapján	Nem
Seed-terjeszkedés	Csapattársak alapján	Listalekéréssel
Gyűjtési cél	Szociális struktúra	Teljesítmény-alapvonal
Corpus mérete	4980 meccs	2000 meccs
Szerep a projektben	Fő kutatási adatkészlet	Kontroll-adatkészlet

Ez a kettős stratégia az egyik legfontosabb módszertani döntése a projektnek. Azzal, hogy a projekt explicit módon gyűjtötte és felcímkézte a két eltérő karakterű datasetet, lehetővé tette azt a kontrolláltabb összehasonlítást, amelynek eredményei a jelen dolgozat empirikus fejezeteit adják.

4.5.3 Megállási feltételek és reprodukálhatóság

Mindkét dataset gyűjtése a következő megállási feltételek valamelyikén áll le: elérte a konfigurált maximális meccsszámot; elérte a konfigurált maximális iterációszámot; a gyűjtési sor kiürült; vagy manuálisan le lett állítva.

A nyers meccs-JSON fájlok fájlrendszeren perzisztálódnak. Az idempotens létező-fájl-ellenőrzés garantálja, hogy már letöltött meccsek nem kerülnek újra letöltésre, ami lehetővé teszi a fokozatos, megszakítható gyűjtést.

4.6 Adatminőség, torzítás és óvatos interpretáció

A Riot API formálisan pontos, de a dolgozat szempontjából az adat így sem „semleges”. A mintavételt erősen befolyásolja:

- milyen seed játékosokról indult a gyűjtés;
- mely queue-k kerültek előtérbe;
- milyen mélyre ment a meccslista-lekérés;
- a premade-kapcsolatok milyen minimumismétlés után tekinthetők érdekesnek.

Ezért a dolgozat minden későbbi eredményénél fontos megjegyzés, hogy a hálózat *nem a teljes League of Legends univerzum reprezentációja*, hanem egy adott gyűjtési stratégia által létrehozott, vizsgálható részvilág. Ez nem gyengeség, hanem korrekt keretfeltétel.

További torzításforrás, hogy a dataset fokozatosan épülő archívumként működik. A meccsek nem egyetlen, szigorúan lezárt időablakból származnak, ezért a játékosok `match_count` értékei részben azt is tükrözik, mennyire került rájuk fókusz a gyűjtési folyamatban. Emiatt a score-réteg és az időbeliség kapcsolatát mindig óvatosan kell kezelni: a magasabb mintaszám nem kizárólag aktivitást, hanem gyűjtési figyelmet is jelenthet.

5. fejezet

A rendszer korai változata és az eredeti pipeline

5.1 A korai projekt jellege

A repository története alapján a rendszer első változata jóval közelebb állt egy adatvezérelt játékosnézőhöz és kísérleti gráf-exportáló eszközhöz, mint a jelenlegi, több rétegre bontott kutatási platformhoz. Ebben a szakaszban a cél elsősorban az volt, hogy:

- a Riot API-ból érkező nyers mérkőzésadatok egy helyre kerüljenek;
- a játékosokhoz legalább alapvető pontszámok rendelődjenek;
- a kapcsolatokból egyszerű gráf és klaszterkimenet szülessen;
- a projekt vizuálisan is megmutatható legyen.

Ez a korai szakasz nem eldobandó előtörténet, hanem a jelenlegi rendszer kiindulópontja. A dolgozatban azért tartom meg hangsúlyosan, mert ebből érthető meg, miért alakult ki később a többnyelvű architektúra és a szigorúbb perzisztenciamodell.

5.2 Az első pontozási logika

Az eredeti `add_new_players.js` egyszerűbb, kézzel összeállított képletekkel dolgozott. A korai `feedscore` lényegében a halálokat büntette a kill és assist mennyiségéhez képest, míg az első `opscore` főként a következő komponensekre támaszkodott:

- ölések,
- asszisztok,
- arany,
- vision score.

Ez a kezdeti modell nyers volt, de fontos előnye volt: gyorsan megmutatta, hogy a tiszta KDA-logikán túl is lehet értelmes, legalább részben kompozit teljesítménymutatót számolni. Később ebből nőtt ki az artifact-szintű és szerepkörérzékeny rendszer.

5.3 A korai gráfépítés és export

A korai `build_graph.py` és a hozzá kapcsolódó exportlépések alapvetően Python-központúak voltak. A hangsúly ekkor még a gyors láthatóságon és nem a perzisztens újrafelhasználhatóságon volt. A pipeline:

- meccsekből kapcsolati gráfot épített,
- összefüggő komponenseket és részgráfokat vizsgált,
- HTML és PyVis exportokat készített,
- gyors vizuális ellenőrzést tett lehetővé.

Ez a megközelítés jó prototípus volt, de rövid időn belül látszott a korlátja: a klaszterek és gráfnézetek csak exportként éltek tovább, és nem váltak a rendszer belső, lekérdezhető objektumaivá.

5.4 A korábbi régiós és ország-alapú ág kialakulása

A projekt korai időszakában erősebb szerepet kapott az a gondolat is, hogy a klaszterekből ország- vagy régiószintű következtetések szülessenek. Ennek nyomai ma is megtalálhatók a `fetch_clusters.py`, `assign_countries.py` és `conclude.py` szkriptekben. A megközelítés lényege az volt, hogy a klaszterek tagneveiből valamilyen földrajzi címke becsülhető, majd ehhez aggregált teljesítményadatok rendelhetők.

Ez a gondolat önmagában izgalmas, de a dolgozat mostani állapotában már világos, hogy módszertanilag bizonytalanabb, mint a közvetlenül a gráfstruktúrára és a meccsadatokra támaszkodó elemzések. Emiatt ezt a vonalat nem törlöm ki a történetből, hanem kifejezetten korábbi, exploratív modulként kezelem.

5.5 Miért kellett a rendszert továbbépíteni?

A korai változat legfontosabb korlátai a következők voltak:

- egyetlen adatbázis és egyetlen meccskönyvtár köré szerveződött;
- a klaszterek és exportok nem voltak igazán első osztályú perzisztens objektumok;
- a pontozás még nem volt kellően szerepkörérzékeny;
- a futásidejű útvonalkeresés és a nehezebb gráfanalitika nem vált el élesen a frontend-logikától.

Ezek a hiányok nem kudarcot jeleztek, hanem természetes érési pontokat. A jelenlegi rendszer legjobban akkor érthető, ha nem egyetlen pillanatfelvételként, hanem ilyen fokozatos evolúció eredményeként nézzük.

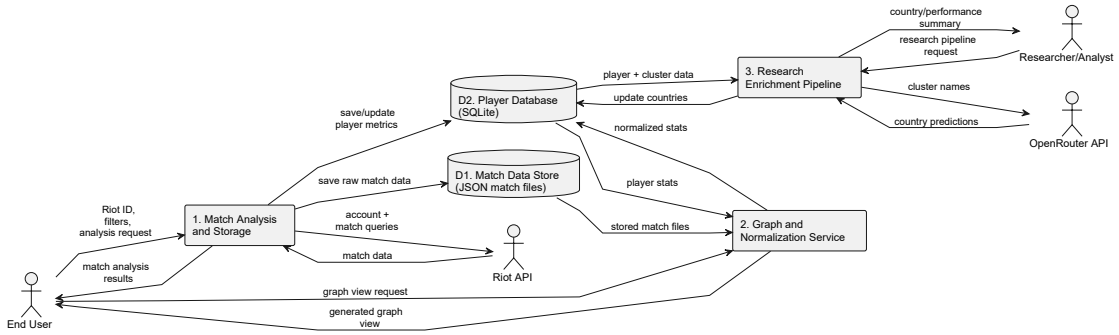
Visszatekintve a projekt korai szakaszának legtermészetesebb csapdája az *akkumulatív script-architectúra* lett volna, amelyben minden új feature egy újabb script formájában kerül be, és az egész rendszer globális állapottól függ. Ez a minta sok hasonló méretű adat-kutatási projektben megfigyelhető. Azzal, hogy a projekt tudatosan bevezette a dataset-absztrakciót, a Rust-szintű analitikai elválasztást és a perzisztens klasztermodellt, elkerülte ezt a csapdát.

Különösen fontos fordulópontot jelentett az a felismerés, hogy *a kutatási analitikának nem szabad a frontend-állapottól függenie*. Ha az asszortativitás kiszámítása frontend-kérésre, a böngészőben lévő adatokból történne, akkor egyrészt nem lenne reprodukálható, másrészt nagyon érzékeny lenne a felhasználói interakcióra. A Rust-oldali, parancssori futtathatóság ezt a problémát gyökeresen oldja meg.

5.6 Miért kellett a régi szövegréteget is megtartani?

Szakdolgozatírás közben könnyű lenne azt a hibát elkövetni, hogy a projekt korábbi állapotát egyszerűen "lecserélem" a jelenlegi, erősebb verzióra. Ezt tudatosan nem teszem. A korai pipeline, a `conclude.py`-szerű ág és a régi klaszterértelmezések azért maradnak a dolgozatban, mert ezek adják meg a fejlődés irányát. A mostani rendszer nem légüres térben született, hanem ezekből nőtt ki.

A country-alapú ágat (a `assign_countries.py` és a `conclude.py` modulokat) jelenleg elavultnak tekintem: az ország-hozzárendelés megbízhatósága nem elegendő ahhoz, hogy a projekt fő állításait erre épüljenek, és a kód aktív karbantartás nélkül is a repositoryban marad mint történeti dokumentum. Ugyanakkor az a gondolat, hogy a klasztereket értelmezési egységként kezeljük — és ne csak vizuális exportként —, ebből az ágból nőtt ki. Ebben az értelemben az ott szerzett tapasztalat a jelenlegi klasztermodel egyik alapköve.



5.1. ábra: A korai Python-alapú adatgyűjtő és gráfépítő pipeline adatáramlása

6. fejezet

A rendszerarchitektúra evolúciója

6.1 Áttekintés

A projekt jelenlegi architektúrája tudatosan hibrid. Első ránézésre ez bonyolultabbnak tűnhet, mint egyetlen nyelvre vagy egyetlen futtatókörnyezetre építeni mindent, de a jelen esetben éppen ez a szétválasztás teszi védhetővé a rendszert.

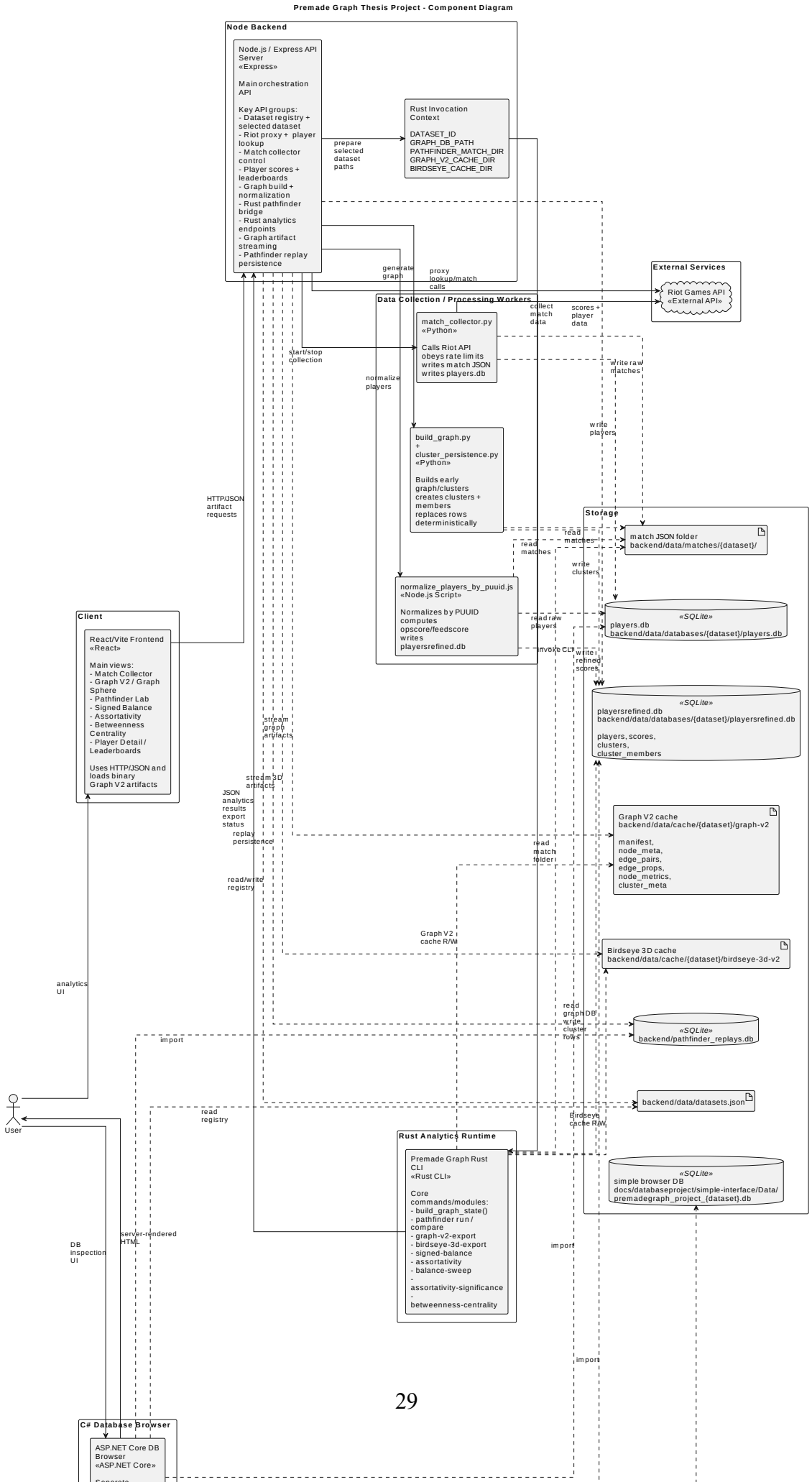
A feladat ugyanis több, egymástól eltérő természetű részre oszlik:

- adatgyűjtés és tömeges JSON-feldolgozás;
- statisztikai és szerepkör-érzékeny pontszámítás;
- gráfépítés és klaszterezés;
- futási időben használható, gyors útvonalkeresés és analitika;
- böngészőben is jól prezentálható vizualizáció.

Ezeket a projekt az alábbi rétegek között osztja szét:

- **Python**: gyűjtés, gráfépítés, közösségdetektálás, exportfolyamatok;
- **SQLite**: központi perzisztens tárolás;
- **Node.js / Express**: API-héj és orchesztráció;
- **Rust**: útvonalkeresés, asszortativitás, központiság, globális gráf-artifaktumok;
- **React + Vite**: interaktív frontend és vizuális prezentáció.

Az alábbi komponensdiagram a teljes rendszer magas szintű szerkezetét mutatja be: a Python adatgyűjtő és gráfépítő worker-eket, a Node.js/Express API réteget, a Rust analitikai motort, a React/Vite frontendet és a C# ASP.NET Core adatbázis-böngészőt, valamint az ezeket összekötő kommunikációs csatornákat és tárolási egységeket. A diagram jól mutatja, hogy a frontend–Rust Pathfinder összeköttetés a Node.js API-n keresztül fut: a frontend közvetlenül nem hívja a Rust motort, hanem HTTP-végpontokon keresztül kommunikál a Node rétegrel, amely a Rust folyamatot stdin/stdout csatornán keresztül vezérli.


Premade Graph Thesis Project - Component Diagram
Node Backend
Node.js / Express API Server
«Express»
Main orchestration API
Key API groups:
- Dataset registry + selected dataset
- Riot proxy + player lookup
- Match collector control
- Player scores + leaderboards
- Graph build + normalization
- Rust pathfinder bridge
- Rust analytics endpoints
- Graph artifact streaming
- Pathfinder replay persistence
Rust Invocation Context
DATASET_ID
GRAPH_DB_PATH
PATHFINDER_MATCH_DIR
GRAPH_V2_CACHE_DIR
BIRDSEYE_CACHE_DIR
prepare selected dataset paths
generate graph
proxy lookup/match calls
External Services
Riot Games API
«External API»
Data Collection / Processing Workers
collect match data
scores + player data
match_collector.py
«Python»
Calls Riot API
obeys rate limits
writes match JSON
writes players.db
start/stop collection
write raw matches
normalize players
write players
build_graph.py
+
cluster_persistence.py
«Python»
Builds early graph/clusters
creates clusters + members
replaces rows deterministically
HTTP/JSON artifact requests
Storage
read matches
match JSON folder
backend/data/matches/{dataset}/
read matches
write clusters
normalize_players_by_puuid.js
«Node.js Script»
Normalizes by PUUID
computes opscore/feedscore
writes playersrefined.db
read raw players
«SQLite»
players.db
backend/data/databases/{dataset}/players.db
invoke CLI
write refined scores
Client
React/Vite Frontend
«React»
Main views:
- Match Collector
- Graph V2 / Graph Sphere
- Pathfinder Lab
- Signed Balance
- Assortativity
- Betweenness Centrality
- Player Detail / Leaderboards
Uses HTTP/JSON and loads binary Graph V2 artifacts
«SQLite»
playersrefined.db
backend/data/databases/{dataset}/playersrefined.db
players, scores,
clusters,
cluster_members
stream graph artifacts
stream 3D artifacts
JSON analytics results export status
replay persistence
Graph V2 cache
backend/data/cache/{dataset}/graph-v2
manifest,
node_meta,
edge_pairs,
edge_props,
node_metrics,
cluster_meta
read match folder
read/write registry
Graph V2 cache R/W
Birdseye 3D cache
backend/data/cache/{dataset}/birdseye-3d-v2
analytics UI
read graph DB
write cluster rows
«SQLite»
backend/pathfinder_replays.db
import
User
read registry
backend/data/datasets.json
Birdseye cache R/W
Rust Analytics Runtime
Premade Graph Rust CLI
«Rust CLI»
Core commands/modules:
- build_graph_state()
- pathfinder run / compare
- graph-v2-export
- birdseye-3d-export
- signed-balance
- assortativity
- balance-sweep
- assortativity-significance
- betweenness-centrality
«SQLite»
simple browser DB
docs/databaseproject/simple-interface/Data/premadegraph_project_{dataset}.db
server-rendered HTML
DB inspection UI
import
import
C# Database Browser
ASP.NET Core DB Browser
«ASP.NET Core»

6.2 Miért nem egyetlen technológia?

A rendszer felépítésében a legfontosabb architekturális elv az volt, hogy *a kontextus dönti el a technológiát*, nem fordítva. A Python kiváló az adatgyűjtéshez, a JSON-feldolgozáshoz és a hálózatépítés gyors iterációjához. A Rust viszont sokkal alkalmasabb a nagyméretű, ismételten meghívott gráfalgoritmusokhoz, ahol fontos a determinisztikus viselkedés, a teljesítmény és a strukturált export. Az SQLite kis projektméret mellett is kényelmesen biztosítja a perzisztenciát, a React pedig jó eszköz a demo- és elemzőfelületek építéséhez.

Ez a szétválasztás nem „tech stack gyűjtögetés”, hanem tudatos munkamegosztás:

- az adatfeldolgozó lépések külön újrafuttathatók,
- a számításigényes részek nincsenek a böngészőre tolva,
- az eredményeket tárolni, exportálni és később újrafelhasználni is lehet.

6.2.1 Miért maradt a C# böngésző különálló, és nem integrálódott a fő frontend-be?

A C# / ASP.NET Core adatbázis-böngésző tudatosan különálló eszközként él. A fő frontend (React / Vite) kutatási és vizualizációs célú: gráfnézetek, pathfinder, asszortativitás és betweenness-központiság. A böngésző ezzel szemben *adatbázis-inspekciós* eszköz: nyers táblasorok, JSON-értékek, szűrés, lapozás. Ezek különböző felhasználói igényeket elégítenek ki, és különböző technológiai stack-et igényelnek. Egy React-ba beágyazott adatbázis-nézet könnyen bonyolítaná a frontend bundle-t, miközben az ASP.NET Core-os szerver-oldali megközelítés JavaScriptes kliensoldali logika nélkül is stabil, könnyen telepíthető eszközt ad.

6.2.2 Elvetett architekturális alternatívák

A végső döntés mellett az elutasított alternatívákat is érdemes dokumentálni. A projekt fejlődése során legalább négy természetes irány merülhetett volna fel:

6.1. táblázat: Fontos architekturális alternatívák és elutasításuk oka

Alternatíva	Rövid előny	Miért nem ez lett a végső irány?
Minden Pythonban	Gyorsabb kezdeti iteráció	A futásidejű gráfalgoritmusoknál és a nagyobb exportoknál gyengébb teljesítmény- és memóriakontroll
Minden Rustban	Erős típusosság és egységes motor	A gyűjtés és a gyors kísérletezés jelentősen nehezebbé és lassabbá vált volna
Frontend-központú számítás	Egyszerűbb demoarchitektúra	A böngészőre tolta volna a számításigényes gráfmunkát, gyengítve a reprodukálhatóságot
Szerveres SQL-adatbázis	Erősebb többfelhasználós modell	A projekt lokális, bemutatható és kutatásorientált természete mellett az SQLite kedvezőbb kompromisszum maradt

Különösen fontos itt a Python és Rust közti munkamegosztás. A Python megőrizte a gyors adatmanipuláció és prototípuskészítés előnyeit, míg a Rust a stabil futásidejű gráfmag szerepét kapta.

A kettő közé a Node/Express réteg került, amely szerződéses API-határként működik.

6.2.3 Miért maradt a Node középen?

Felmerülhetne, hogy a frontend miért nem közvetlenül a Rust motorral kommunikál, vagy miért nem a Python szolgálja ki a teljes alkalmazást. A `backend/server.js` alapján a Node rétegnek önálló felelőssége van:

- dataset-regisztert kezel és aktív datasetet választ;
- futás közbeni kulcsokat és környezeti útvonalakat központosít;
- háttércollector folyamatokat indít és monitoroz;
- HTTP-végpontokat biztosít a Rust parancsok fölé;
- a bird's-eye artifaktumokat és egyéb cache-eket konzisztensen szolgálja ki.

Ez a köztes réteg csökkenti a frontend és a számítási mag közti csatolást. A böngészőnek nem kell folyamatindítással, fájlrendszer-útvonalakkal vagy dataset-propagálással foglalkoznia; ezek a backendben maradnak.

6.3 Evolúciós fázisok és architekturális kompromisszumok

Az architektúra fejlődését három, viszonylag jól elkülöníthető fázisban lehet leírni.

1. **Korai script-fázis**: a hangsúly a nyers adat begyűjtésén, a gyors score-számításon és a kezdeti gráfexporton volt.
2. **Perzisztens pipeline-fázis**: megjelent az igény arra, hogy a klaszterek és játékosprofilok ne csak pillanatnyi exportként, hanem visszatölthető adatstruktúraként éljenek tovább.
3. **Kutatási platform-fázis**: a Rust analitika, a multi-dataset modell és a külön research oldalak megjelenésével a projekt már nem csak vizualizációs eszköz, hanem kísérleti környezet lett.

Minden fázisban más minőségi attribútum került előtérbe: először a fejlesztési sebesség, utána a reprodukálhatóság, végül a teljesítmény és az elemzési mélység. Ez segít megérteni, miért nem "inkonzisztencia" a több réteg, hanem a rendszer természetes evolúciója.

6.4 Több-adatkészletes architektúra

A korábbi, egyetlen adatbázisra és egyetlen meccsmappára épülő modell helyett a jelenlegi rendszer már több, egymástól független adatkészletet támogat. A `backend/data/datasets.json` regiszter alapján minden dataset külön rendelkezik:

- nyers adatbázissal,
- finomított adatbázissal,
- meccs-JSON könyvtárral,
- cache könyvtárral.

A dolgozat írásakor az aktív dataset a `default` azonosítójú alap-adatkészlet volt, de a regiszterben a `smoke-test-dataset` és a `flexset` is szerepel. Ez fontos előrelépés a korábbi állapothoz képest, mert lehetővé teszi, hogy különböző mintákon, különböző gyűjtési stratégiákkal is fusson ugyanaz az analitikai pipeline.

6.2. táblázat: A több-adatkészletes architektúra logikai elemei

Elem	Szerep
Dataset registry	Az elérhető adatkészletek metaadatait és az aktív dataset kijelölését tárolja
Raw database	A gyűjtéshez és kezdeti beolvasáshoz szükséges játékosadatok
Refined database	A normalizált pontszámokat, klasztereket és származtatott mezőket tartalmazza
Matches directory	A tényleges meccs-JSON állományok helye
Cache directory	Futás közbeni vagy előre generált artifaktumok tárolása

6.4.1 A dataset mint életciklus-egység

A dataset a jelenlegi rendszerben nem statikus mappát, hanem együtt mozgó állapotcsomagot jelent. Egy új dataset létrehozása után több, egymástól elválasztható fázison mehet keresztül:

1. nyers seed és játékosállapot kialakítása;
2. match collector futtatása;
3. PUUID-alapú normalizálás és score-frissítés;
4. Python gráfépítés és klaszterperzisztencia;
5. Rust runtime-felépítés és bird's-eye export;
6. kutatási analitikák lefuttatása.

Ez a bontás a dolgozat szempontjából azért erős, mert világos köztes állapotokat ad. Egy asszortativitási vagy központisági futtatásnak nem kell újra elindítania a Riot API-gyűjtést, és egy klaszterezési változtatásnak sem kell a teljes pipeline-t újraépítenie. A dataset tehát a reprodukálható munkafolyamat egységeként is értelmezhető.

[PLACEHOLDER]

Dataset életciklus: gyűjtés → nyers DB → finomított DB → gráf-cache → analitikai eredmény

6.2. ábra: Egy dataset teljes életciklusának diagramja a Riot API-tól az analitikai kimenetig

6.5 A jelenlegi adatállapot

Az aktív adatállapotot 2026. április 28-án közvetlenül a repository aktuális állapotából ellenőriztem. Az aktív dataset a `soloq`. Az ellenőrzött fő adatállományok és adatbázisok adatai:

Flexset dataset (EUNE Master+ Flex Queue):

- meccs-JSON állományok: **4981** fájl a `backend/data/matches/flexset/` könyvtárban;
- Graph V2 összegző jelentés: `matchCount: 4980` (egy fájl eltérés ellenőrizendő);
- nyers adatbázis (`players.db`): **15 147** játékos;
- finomított adatbázis (`playersrefined.db`): **15 421** játékos, **599** klaszter, **5741** klaszertag.

SoloQ dataset (EUNE Master+ Solo/Duo Queue, aktív):

- meccs-JSON állományok: **2001** fájl a `backend/data/matches/soloq/` könyvtárban;
- Graph V2 összegző jelentés: `matchCount: 2000`;
- nyers adatbázis (`players.db`): **2822** játékos;
- finomított adatbázis (`playersrefined.db`): **6768** játékos, **140** klaszter, **1538** klaszertag.

Fontos megjegyezni, hogy a soloq dataset klaszterei jellemzően *duo párokat* jelölnek: mivel a Solo/Duo Queue maximálisan kétfős premade csapatokat enged, az ismétlődő kapcsolatokból kiemelkedő klaszterek döntő többsége két-három játékosra szűkül. Ez strukturálisan különbözik a flexset klasztereitől, ahol akár ötfős premade csapat is kialakulhat rendszeresen ismétlődő együttsként.

Ez a pont fontos: a jelen dolgozat nem hipotetikus rendszert ír le, hanem egy olyan állapotot, amelyben a pipeline már nem csak tervekből áll, hanem két, egymással összehasonlítható, eltérő queue-szemantikájú, valós adatokkal feltöltött és ténylegesen lefuttatott kutatási korpusszal rendelkezik.

A meccsszámban megfigyelhető egy-fájlos eltérés (4981 fájl vs. 4980 matchCount a Graph V2 összegzőben) metodikai megjegyzést igényel: ez feltehetően egy olyan meccsfájl, amelyet a gyűj-

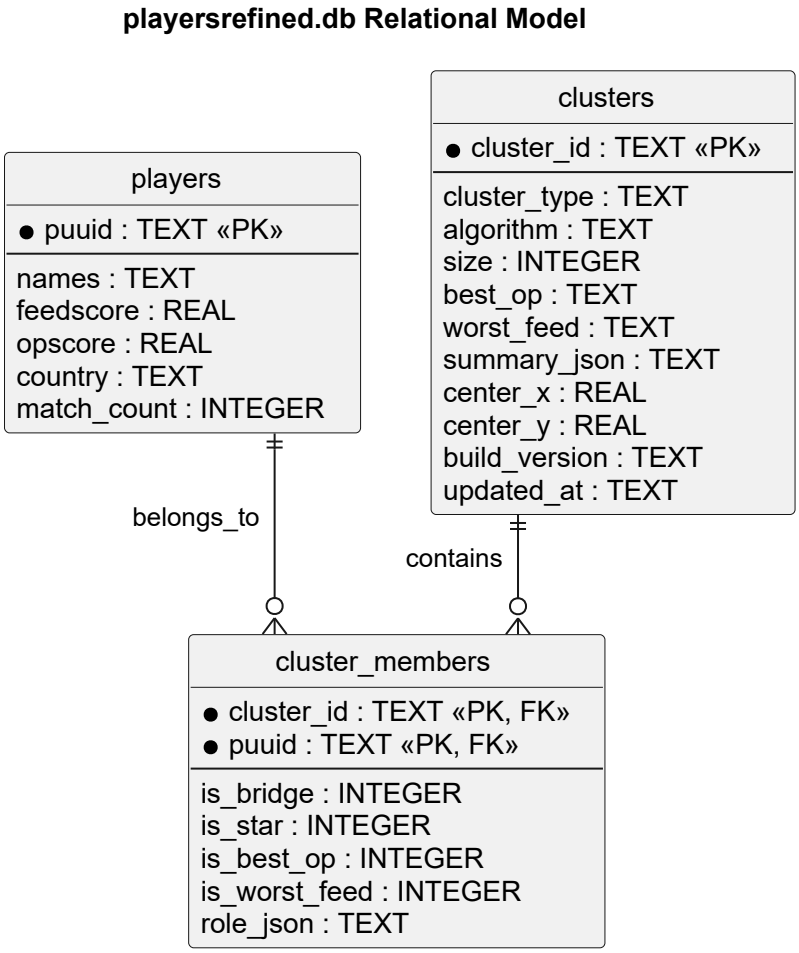
tő letöltött, de a Graph V2 pipeline valamilyen validációs szűrőn kizárt. Ez nem kritikus eltérés, de jól mutatja, hogy a pipeline nem pusztán fájlszámlálás alapján dolgozik.

6.6 Perzisztencia és relációs modell

A projekt egyik legfontosabb érése az volt, hogy a klaszterek kikerültek a „csak JSON export” kategóriából, és első osztályú adatbázis-objektumokká váltak. A jelenlegi séma központi táblái az SQLite perzisztenciára épülnek [3]:

- `players`,
- `clusters`,
- `cluster_members`.

A `players` tábla nemcsak a név- és országmezőket, hanem a normalizált `opscore`, `feedscore`, szerepkör, meccsszám és artifact-szintű részeredményeket is tárolja. A `clusters` és `cluster_members` páros pedig lehetővé teszi, hogy ugyanazon játékoshoz többféle klasztertípus (`python_population`, `rust_pathfinding`) is kötődjön.



6.3. ábra: Relációs modell a klaszter- és játékosperzisztenciához (`players`, `clusters`, `cluster_members`)

7. fejezet

Több-adatkészletes architektúra

7.1 Miért vált szükségessé a dataset-regiszter?

Az egyetlen adatbázisos modell korai prototípushoz még elegendő volt, de amint a projekt egyszerre kezdett demo-, smoke-test- és kutatási célokat szolgálni, világossá vált, hogy a különböző minták szétválasztása nélkül a rendszer nehezen reprodukálható. Ugyanaz a frontend, ugyanaz a Rust futtatókörnyezet és ugyanaz a Python pipeline más eredményt adhat, ha eltérő bemeneti korpuszon fut. Ezt a különbséget nem rejtett konfigurációs fájlokban, hanem explicit dataset-regiszterben érdemes kezelni.

A dataset-regiszter bevezetésének szükségességét konkrét igényváltozás indokolta:

- **Smoke-teszt igény**: a fejlesztői pipeline-t kis adathalmazzal kell tudni validálni, anélkül hogy az éles adatbázist módosítani kellene;
- **Kutatási szétválasztás**: a Flex Queue és a Solo/Duo Queue strukturálisan eltérő gráfot hoz létre; az eredmények összekeveredése módszertanilag hibás lenne;
- **Reprodukálhatósági garancia**: ha a fő kutatási dataset változik, a régi dataset-állapot is ellenőrizhető maradjon;
- **Pipeline-átjárhatóság**: a Rust analitika, a Python klaszterpipeline és a Node/Express API mind ugyanarra a dataset-re álljon rá, ne legyen "backend igazság" és "Rust igazság" egymás mellett.

Az **elvetett alternatíva** a konfigurációs fájlon alapuló manuális váltás lett volna: a felhasználó kézzel módosít egy útvonalat, és a rendszer azt használja. Ez nem skálázódik, mert a több állapot egyszerre való megtartása nem lehetséges, és az összekeveredési hibák nehezen detektálhatók.

A `backend/data/datasets.json` pontosan ezt a feladatot látja el. A dataset fogalma a jelenlegi rendszerben nem csak annyit jelent, hogy "másik mappa", hanem egy teljes, együtt mozgó adategységet:

- nyers adatbázissal,
- finomított adatbázissal,
- meccs-JSON könyvtárral,
- cache-könyvtárral,
- megjeleníthető metaadatokkal.

7.2 A jelenlegi regiszterben szereplő adatkészletek

A dolgozat írásakor ellenőrzött `backend/data/datasets.json` regiszterben négy dataset szerepelt:

- `default` — az eredeti telepítésből migrált, vegyes queue-tartalmú alapkorpusz;
- `smoke-test-dataset` — gyors infrastruktúratesztelésre szánt átmeneti adatkészlet;
- `flexset` — EUNE Master+ Flex Queue meccsek, kifejezetten premade és ismétlődő csapattársi kapcsolatok tanulmányozására;
- `soloq` — EUNE Master+ Solo/Duo Queue meccsek, kontroll- és teljesítmény-alapvonal-gyűjtés céljából.

Az **aktív dataset a `soloq`**. Ez a döntés az adatbázis-regiszterben is rögzített: a `currentDatasetId` mező értéke `"soloq"`.

A négy dataset jelenléte nem véletlen felhalmozódás, hanem tudatos kutatásstratégia eredménye. A `default` megőrzi az eredeti, heterogén korpuszt és a korábbi fejlesztési előtörténetet. A `smoke-test-dataset` a pipeline gyors ellenőrzésére szolgál anélkül, hogy a főadatokat érintené. A `flexset` és a `soloq` egymással párban értelmezhetők: az előbbi a premade és csoportos játékhoz kötődő társas jeleket erősíti, az utóbbi pedig ugyanannak a szervernek és rangsornak egyénibb, kevésbé premade-fókuszú meccstörténetét képviseli.

7.1. táblázat: A négy regisztrált dataset összefoglalója

Dataset ID	Szerepkör	Jellemzők
`default`	Alap/migrált	Vegyes queue, heterogén tartalom
`smoke-test`	Pipeline-teszt	Átmeneti, infrastruktúra-validálásra
`flexset`	Fő kutatási dataset	EUNE Master+ Flex, premade-fókusz
`soloq`	Kontroll/baseline	EUNE Master+ Solo/Duo, teljesítmény-alap

A `flexset` esetében a regiszter metaadata tartalmazza a gyűjtési stratégia dokumentumát és a collector konfigurációs fájl hivatkozását (`apex-flex-collector.json`). A `soloq` hasonlóan strukturált: külön collector konfigurációs fájllal és gyűjtési stratégia-dokumentummal rendelkezik. Ez a párhuzamos konfiguráció kifejezetten mutatja, hogy a két dataset nem eseti gyűjtés, hanem tudatosan eltérő stratégiával felépített, összehasonlítható korpusz.

7.2. táblázat: A dataset-regiszter funkcionális szerepei

Funkció	Jelentőség
Aktív dataset kijelölése	Meghatározza, hogy a backend és a frontend mely adatkorpuszt használja alapértelmezetten
Eltérő tárolási útvonalak	Megakadályozza, hogy a teszt- és főadatok összekeveredjenek
Külön cache-ek	A bird's-eye és egyéb előre generált artifaktumok elkülönülnek
Kísérleti összevethetőség	Azonos pipeline különböző mintákon is futtatható

7.3 A dataset-választás propagálása a backend és a Rust felé

A dataset-ek architektúrájának egyik legfontosabb technikai előnye az, hogy a választás nem áll meg a Node-szinten. A backend a kiválasztott datasetből környezeti változókat és útvonalakat képez a Rust futtatókörnyezet felé is, például:

- `DATASET_ID`,
- `GRAPH_DB_PATH`,
- `PATHFINDER_MATCH_DIR`,
- `PATHFINDER_BIRDSEYE_CACHE_DIR`.

Ez építészetileg nagyon fontos döntés. Nincs külön "frontend igazság", "backend igazság" és "Rust igazság" ugyanarról az adatállapotról: a dataset-választás a teljes végrehajtási láncon végigmegy.

A `server.js` szintjén ez úgy valósul meg, hogy a szerver a kiválasztott datasetből egységes környezeti változókészletet képez, például `DATASET_ID`, `GRAPH_DB_PATH`, `PATHFINDER_MATCH_DIR` és `PATHFINDER_BIRDSEYE_CACHE_DIR` formában. A Rust runtime ezeket fogyasztja, így a teljes végrehajtási lánc ugyanarra az adatvilágra áll rá. Ez különösen fontos a védhetőség szempontjából: a frontend által mutatott eredmény nem különálló demonstrációs adatból, hanem ugyanabból a datasetből jön, amelyen a kutatási analitika is fut.

7.4 Őszinte állapotkép a migrációról

A több-adatkészletes architektúra implementált, de a repository ellenőrzött pillanatképe alapján a történetileg gazdagabb `playersrefined.db` állomány még nem teljesen simult bele a dataset-könyvtárstruktúrába. A vizsgálat során az látszott, hogy a dataset könyvtár alatti `playersrefined.db` üres volt, miközben a repository gyökerében lévő, régebbi `playersrefined.db` tartalmazta a valós, nagyobb korpuszt.

Ezt a dolgozatban nem hibaként, hanem átmeneti evolúciós állapotként kezelem. A lényeg az, hogy az új architektúra iránya egyértelmű, de a migrációs történet nem minden részlete tekinthető végleg lezártnak. Ez tipikus kutatás-közeli projektjelenség: az architektúra előbb válik helyessé, mint teljesen sterilré.

8. fejezet

SQLite perzisztencia és klasztermodellezés

8.1 Miért maradt központi elem az SQLite?

A projekt mérete és célja szempontjából az SQLite továbbra is kifejezetten jó kompromisszum. Nem igényel külön adatbázisszerver-felügyeletet, könnyen verziózható a séma, gyors a lokális fejlesztésben, és a demo közben sincs szükség külön infrastruktúrára. Ennél is fontosabb, hogy a perzisztens modell egyszerűen ellenőrizhető: a játékos-, klaszter- és tagsági sorok közvetlenül lekérdezhetők.

Egy szakdolgozatban ez külön előny. Nem csak a frontend JSON-kimeneteit lehet megmutatni, hanem az alapul szolgáló relációs tárolást is.

[PLACEHOLDER]

SQLite ER-diagram: players, clusters, cluster_members, pathfinder_replays táblák és kapcsolataik

8.1. ábra: A fő SQLite adatbázis egyedkapcsolati diagramja

8.2 A `players` tábla mint gazdagított játékosprofil

A jelenlegi `players` tábla már jóval több egy egyszerű névjegyzéknél. A fő mezőcsoportok:

- azonosítók és névmezők;
- aggregált teljesítménymutatók (`feedscore`, `opscore`);
- mintanagyságot leíró mezők (`match_count`, `matches_processed`);

- szerepkör és szerepkör-bizonyosság;
- artifact-szintű komponensek.

A 2026. április 24-én vizsgált állapotban a tábla **25 618** játékost tartalmazott. A szerepkör-eloszlás közel kiegyensúlyozott képet mutatott a fő lane-szerepkörök között, miközben `UNKNOWN` kategóriába is került **3120** játékos. Ez a gyakorlatban azt jelenti, hogy a rendszer nem próbál mindenáron erőltetett lane-hozzárendelést végezni ott, ahol a nyers adat nem kellően megbízható.

8.1. táblázat: Szerepkör-eloszlás a vizsgált `players` állapotban

Szerepkör	Játékosok száma
TOP	4571
MIDDLE	4517
JUNGLE	4484
BOTTOM	4471
UTILITY	4455
UNKNOWN	3120

8.2.1 Sémanormalizáció és visszafelé kompatibilitás

A `backend/normalize_players_by_puuid.js` egyik fontos, de könnyen rejtve maradó tulajdonsága, hogy nem tiszta, egyszer létrehozott sémával számol. A modul deklarálja a `players` tábla kanonikus oszlopkészletét, és szükség esetén a történetileg felhalmozott eltérésekhez is alkalmazkodik. Ez a projekt evolúciós természetéből következik: a score-réteg, a szerepkörmezők és az artifact-szintű komponensek nem egyszerre, hanem fokozatosan kerültek be a modellbe.

Ennek szakdolgozati jelentősége is van. A rendszer nem laboratóriumi körülmények között egyszer megtervezett és lezárt adatbázissal dolgozik, hanem olyan struktúrával, amely a fejlesztés közben bővült. A normalizáló modul ezért egyben migrációs réteg is: megőrzi a régebbi állapotok használhatóságát, miközben előkészíti az újabb analitikai mezőket.

8.2.2 SQLite pragmák és írási kompromisszumok

A normalizáló pipeline az SQLite-ot nem alapbeállításokkal használja. A kód explicit módon `WAL` naplózást, `NORMAL` szinkronizációt, memóriabeli ideiglenes tárolást és foreign key támogatást állít be. Ez a választás jól mutatja, hogy az SQLite jelenléte nem „egyszerűségi lustaság”, hanem tudatos kompromisszum. A rendszernek lokálisan futó, ismételten újraíró, de nem többszáz egyidejű felhasználót kiszolgáló analitikai adattárra van szüksége; erre a célra ez a konfiguráció kifejezetten jó arányt ad a sebesség, a stabilitás és a hordozhatóság között.

8.3 A klasztermodell és a két fő tábla szerepe

A klaszterek perzisztenciáját a `clusters` és `cluster_members` páros adja. A `clusters` tábla főbb elemei:

- `cluster_id`,

- `cluster_type`,
- `algorithm`,
- `size`,
- `summary_json`,
- térbeli koordináták,
- build-verzió és frissítési idő.

A `cluster_members` tábla ezzel szemben tagsági és szerepkörjellegű információt tárol:

- a klaszter és játékos összerendelését;
- `is_bridge`, `is_star`, `is_best_op`, `is_worst_feed` jelzőket;
- szerepköri összegzést `role_json` formában.

Ez a modell azért erős, mert a klaszter nem puszta név vagy címke, hanem lekérdezhető objektum, amelyhez külön metaadatok és tagsági szerepek kötődnek.

A `backend/cluster_persistence.py` további fontos döntése, hogy a klaszterek cseréjét `cluster_type` szerint végzi. Vagyis egy új `python_population` build felülírja a régi Python-klasztereket, de nem nyúl a `rust_pathfinding` családhoz. A művelet előbb törli az adott típushoz tartozó régi tagsági sorokat, majd a klasztermetaadatokat, és ezután írja be az új eredményt. Ez a szemantika nagyon hasznos, mert ugyanazon adatbázisban több klasztercsalád élhet együtt anélkül, hogy egy újrafuttatás mindent elsöpörne.

A `summary_json`, `role_json`, `build_version` és `updated_at` mezők a kutatási visszakövethetőséget is javítják. A klaszter így nem pusztán tagsági halmaz, hanem olyan objektum, amelynek eredete, típusa és részleges összefoglaló állapota is megmarad.

8.4 A perzisztált klaszterek aktuális nagyságrendje

A két fő kutatási dataset finomított adatbázisainak ellenőrzött állapota alapján a klaszterstruktúra lényegesen eltér a két korpuszban:

Flexset (`playersrefined.db`):

- **599** klaszter a `clusters` táblában;
- **5741** klasztertagsági rekord a `cluster_members` táblában;
- a Graph V2 artfaktumban **1531** látható klaszter, átlagos méret **3.750**, legnagyobb **20**.

SoloQ (`playersrefined.db`):

- **140** klaszter a `clusters` táblában;
- **1538** klasztertagsági rekord a `cluster_members` táblában;
- a Graph V2 artfaktumban **870** látható klaszter, átlagos méret **1.768**, legnagyobb **20**.

A két szám közötti különbség metodikailag fontos. A `clusters` táblában tárolt klaszterek a Python pipeline által meghatározott, populációs alapú közösségek. A Graph V2 látható klasztereinek száma magasabb, mert a Graph V2 külön, kötetlen ally-csoportként kezeli azokat a csomópontokat is, amelyek csak vizualizációs szempontból alkotnak csoportot. A flexset lényegesen sűrűbb klaszterstruktúrát mutat: az átlagos klaszterméret (3.750) a soloq-hoz képest (1.768) közel duplája, ami empirikusan is alátámasztja, hogy a Flex Queue erősebb ismétlődő csoportkapcsolatokat termel.

Ez önmagában is fontos rendszerszintű eredmény, mert mutatja, hogy a többdataset-architektúra valódi különbséget fog meg, nem csupán méretbeli eltérést.

8.5 A két klasztercsalád részletes összehasonlítása

A rendszerben két klasztercsalád él párhuzamosan, amelyek különböző módszerrel és célra épülnek:

8.5.1 python_population

A `python_population` típusú klaszterek a Python gráfépítési pipeline kimenetei. Ezeket a `backend/build_graph.py` és a Louvain-moduláris közösségdetektálás hozza létre a nyers meccsadatok alapján. Főbb jellemzők:

- a klaszterméret nincs kemény korláthoz kötve (a moduláris detektálás saját maga dönt a méretekről);
- az algoritmus a globális moduláris optimalizáció szempontjából határozza meg a csoportokat;
- a flexset dataseten például közepes méretű, $\sim$10–15 fős klaszterek is keletkezhetnek, ha a moduláris struktúra azt indokolja;
- a Python klaszterek inkább *közösségdetektálási* perspektívát tükröznek.

8.5.2 rust_pathfinding

A `rust_pathfinding` típusú klaszterek a Rust motor futásidejű klaszterezési logikájából jönnek. Az implementáció az ally-kapcsolatokon alapuló erős-összefüggési szegmensekre bontja a gráfot, kemény méretkorláttal (maximum 20 tag). Főbb jellemzők:

- a 20 tagos korlát garantálja a klaszterek áttekinthetőségét a frontend vizualizációban;
- az oversized összefüggő komponensek determinisztikusan kerülnek szétvágásra;
- a Rust klaszterek főként *pathfinding és vizualizációs* célra optimalizáltak;
- az `orbitScore` és `orbitRadius` értékek kizárólag a Rust klaszterekre értelmezhetők.

8.5.3 A két csomád koegzisztenciájának módszertani értéke

A két klasztercsalád nem egymást helyettesíti, hanem egymást kiegészíti. A Python klaszterek adnak egy moduláris alapú közösségstruktúrát; a Rust klaszterek ennek futásidejű, vizualizáció-kompatibilis változatát. Az a tény, hogy mindkét klaszter-nézet egyszerre létezik az adatbázisban, lehetővé teszi, hogy például az asszortativitási bontásoknál explicit módon megkérdezhetjük: a Python-közösségek belső éleire vagy a Rust-vizualizációs csoportok belső éleire mért asszortativitás-e erősebb?

8.6 Miért fontos a perzisztencia a kutatási fejezetekhez?

Az asszortativitási, központisági és kísérleti signed-balance számítások látszólag a Rust oldalon történnek, de valójában a korábban kialakított perzisztens modell nélkül sokkal nehezebben lennének értelmezhetők. A klaszterekhez kötött bontások, a játékosprofilokhoz rendelt score-ok és a későbbi exportok mind arra támaszkodnak, hogy a játékos és a közösségi kontextus ugyanabban a perzisztens rendszerben elérhető.

Ez jó példa arra, hogy az adatbázistervezés nem csak kiszolgáló infrastruktúra, hanem a kutatási érték egyik feltétele.

9. fejezet

Feedscore és opscore metrikák

9.1 Gyűjtési stratégia

A projekt adatgyűjtő rétege a Riot API-ra épül. A jelenlegi architektúrában a meccsek begyűjtése nem egyetlen frontend gombnyomásra történő monolitikus folyamat, hanem paraméterezhető háttérfeladat, amely a datasethez kötve fut. A backend oldalon a gyűjtéshez tartozik:

- aktív dataset kiválasztása,
- lekérdezési limit figyelembevétele,
- iterációszám és meccsszám paraméterezése,
- queue-típus szerinti szűrés,
- véletlen vagy konkrét játékos-alapú indítás.

Ez azért fontos, mert a Riot API korlátai mellett csak olyan rendszer tartható karban hosszabb távon, amely a gyűjtést külön modulnak tekinti, és nem keveri össze a futási idejű vizualizációval.

9.2 Normalizálás PUUID alapján

A nyers adatok összefűzésének kulcsa a `puuid` alapú normalizálás. A játékosnevek változhatnak, a különböző meccsekben eltérő formában is megjelenhetnek, ezért a rendszer az egyedi azonosítót tekinti az elsődleges kulcsnak. A normalizálás során a pipeline:

- végigolvassa a meccs-JSON fájlokat,
- résztvevőnként kinyeri a releváns mezőket,
- szerepkört rendel a `teamPosition` alapján,
- meccsszintű artifactokat számol,
- játékosszinten átlagol és normalizál,
- a végeredményt a `players` táblába írja vissza.

Ez adja a rendszer egyik központi alapját, mert minden későbbi gráfanalitika erre a normalizált játékosprofilra támaszkodik.

9.3 A jelenleg implementált pontozási rendszer

[PLACEHOLDER]

Score pipeline: nyers meccsstatisztikák → szerepkör-szorzók → percentilis-normalizálás → opscore/feedscore

9.1. ábra: A pontozási pipeline logikai lépései a nyers adatoktól a normalizált végső metrikáig

A projektben használt `opscore` és `feedscore` nem dobozból kivett, külső szolgáltatásból átvett mutatók, hanem a rendszerben megvalósított, explicit képletekre épülő pontszámok. A pontozás célja nem az, hogy „megmondja az igazságot”, hanem az, hogy a nyers meccsadatokat egy összehasonlítható, szerepkörérzékeny skálára emelje.

9.3.1 Artifact-szintű bontás

Minden mérkőzésből nyolc alapkomponenst számol a rendszer:

$$\text{kda} = \text{kills} + 0.965 \cdot \text{assists} - 0.25 \cdot \text{deaths} \tag{9.1}$$

$$\text{economy} = \frac{\text{goldEarned}}{\text{gameDurationMinutes} \cdot 10} + 0.5 \cdot \text{efficiency} \tag{9.2}$$

$$\text{map_awareness} = 0.15 \cdot \text{visionScore} + 0.05 \cdot \text{wardsPlaced} + \ldots \tag{9.3}$$

$$\text{utility} = 0.08 \cdot \text{enemyChampionImmobilizations} + 0.001 \cdot \text{totalHeal} + \ldots \tag{9.4}$$

$$\text{damage} = 0.2 \cdot \frac{\text{physical} + \text{magic} + \text{trueDamage}}{1000} \tag{9.5}$$

$$\text{tanking} = 0.05 \cdot \frac{\text{totalDamageTaken}}{\text{gameDurationMinutes}} \tag{9.6}$$

$$\text{objectives} = 0.002 \cdot \text{damageDealtToBuildings} + 0.25 \cdot \text{turretTakedowns} + \ldots \tag{9.7}$$

$$\text{early_game} = 0.5 \cdot \text{firstBloodKill} + 0.2 \cdot \text{firstBloodAssist} + \ldots \tag{9.8}$$

Az artifactokra bontásnak két előnye van. Egyrészt átláthatóbb, hogy miből áll össze a végső pontszám. Másrészt később könnyebb módosítani vagy kalibrálni a rendszert, mint ha egyetlen, nehezen értelmezhető képletet használnánk.

Ez a bontás közvetlenül a `backend/lib/scoring_utils.js` implementációjából jön, vagyis nem utólagos, szövegben kitalált kategóriákról van szó. A rendszer minden mérkőzésre ténylegesen kiszámolja ezeket a komponenseket, majd játékosszinten átlagolja őket. Emiatt a végső score

visszafejthetőbb és jobban auditálható, mint egy monolitikus, több tucat nyers mezőt egyszerre összemosó képlet lenne.

9.3.2 Szerepkörérzékeny súlyozás

A pontozás egyik legfontosabb döntése az volt, hogy a rendszer nem próbál utólagosan, heuristikusan szerepkört kitalálni, hanem a Riot által szolgáltatott `teamPosition` mezőt használja. Ez tudatos döntés: a játékos saját bevallása (vagy a matchmaking rendszer szerinti beosztása) megbízhatóbb kiindulópont, mint akár a legjobb heurisztikus lane-detektálás. A támogatott szerepkörök: `TOP`, `JUNGLE`, `MIDDLE`, `BOTTOM`, `UTILITY`, valamint `UNKNOWN`.

A különböző artifactok eltérő súlyt kapnak szerepkörönként. Az ok egyszerű: a League of Legends mechanikája különböző teljesítmény-dimenziókat értékel más-más szerepkörben. Ha például egy támogatót az ADC-vel azonos sebzési elvárással értékelünk, garantáltan hamis alacsony pontszámot kapunk minden olyan játékra, amelyben a support helyesen játszott – csak más eszközöket használt.

Az egy mérkőzésre jutó nyers `opscore` tehát a szerepkör-specifikus súlyokkal korrigált artifactok összege:

$$\text{match_opscore_raw} = \sum_{a \in A} w_{r,a} \cdot \text{artifact}_a \tag{9.9}$$

ahol A az artifactok halmaza, r az adott meccsre vonatkozó szerepkör, $w_{r,a}$ pedig az adott szerepkör–artifact párhoz tartozó szorzó.

A `backend/scoring_config.js` alapján a szerepkörlogika teljes szorzóhalmazzal van rögzítve. A 9.1 táblázat a teljes konfigurációt tartalmazza:

9.1. táblázat: Teljes szerepkör-szorzótábla a `scoring_config.js` alapján

Szerepkör	kda	economy	map	utility	damage	tanking	objectives	early_g.
TOP	1.10	1.05	0.90	0.60	1.15	1.00	1.20	1.05
JUNGLE	1.20	0.70	1.40	0.80	1.00	0.80	1.40	1.30
MIDDLE	1.00	1.10	1.20	1.00	1.05	0.80	1.00	1.10
BOTTOM	1.20	1.30	1.10	0.70	1.40	0.60	0.90	1.00
UTILITY	0.70	0.40	1.80	1.80	0.50	1.10	0.70	0.80
UNKNOWN	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

A TOP súlyozás indoklása

A TOP lane specializált harcoshelyen áll, általában tartós harcokat vív. A `damage` (1.15) és `objectives` (1.20) szorzók a split-pusher és a csatadöntő funkciót tükrözik. A `map_awareness` (0.90) szándékosan alacsonyabb, mert a TOP lane természetes izolációja csökkenti a valósidejű térképkontroll-elvárást. A `utility` (0.60) szintén alacsony, mert a TOP-on leggyakrabban harcos vagy tank típusú bajnokok játszanak, akiknek az utility-eszköztára szűkebb. A `kda` (1.10) és `early_game` (1.05) közepes szintjei a mérsékelt korai jelenléttel összhangban vannak.

A JUNGLE súlyozás indoklása

A jungler az egyetlen szerepkör, amely az egész térképen aktív. Ez magyarázza a `map_awareness` (1.40) és `objectives` (1.40) legmagasabb értékeit az összes szerepkör között. A `kda` (1.20) és `early_game` (1.30) szorzók azt tükrözik, hogy a jungler early gank sikerei és az első objektívumok megszerzése közvetlen hatással van a csapat egészének tempójára. Az `economy` (0.70) szorzó szándékosan alacsony: a jungler aranytermelése természetesen alacsonyabb, mint a lane-játékosoké, és ezt nem szabad büntetni. A `damage` (1.00) és `tanking` (0.80) semleges értékek, mert a jungler bajnokpool nagyon heterogén — alacsonyabb elvárás fair ezért ebben a két dimenzióban.

A MIDDLE súlyozás indoklása

A mid laner kiegyensúlyozott profil, mivel ez az egyetlen ösvény, amely természetesen részt tud venni mind a korai, mind a késői fázisban. A `map_awareness` (1.20) tükrözi, hogy a mid lane geográfiai középpontból nyomást tud kifejteni mindkét szárnyra. Az `economy` (1.10) és `damage` (1.05) az arany-hatékony bajnokprofilt jelzi. A `utility` (1.00) semleges, mert a mid laner bajnokkészlete mind utility-mágusokat, mind burst-mágusokat tartalmaz. A kiegyensúlyozott profil nem azt jelenti, hogy a mid laner "átlagos", hanem azt, hogy valóban sokirányú elvárást kellene kielégítenie.

A BOTTOM (ADC) súlyozás indoklása

Az ADC a csapat fő sebzésháza. A `damage` (1.40) a legmagasabb a szerepkörök között, és a `economy` (1.30) is kiemelkedő, mert az ADC aranytermelése alapjaiban határozza meg, milyen itemekkel tud belépni a csatákba. A `kda` (1.20) szorzó azt tükrözi, hogy az ADC halálai különösen drágák — egy elveszített ADC csatadöntő következményű lehet. A `utility` (0.70) és `tanking` (0.60) szorzók alacsonyak, mert az ADC tipikusan fragile és utility-szegény szerepkör. Az `objectives` (0.90) semlegeshez közeli: az ADC strukturális sebzése hasznos, de jellemzően nem az objektívum-megszerzés elsődleges felelőse.

A UTILITY (support) súlyozás indoklása

A support súlyozás a leginkább szimmetrikusan inverz az ADC-hez képest. A `map_awareness` (1.80) és `utility` (1.80) a legmagasabb értékek az egész rendszerben, mert a support tevékenységének lényege a térképkontroll biztosítása és a csapattársak ellátása. A `damage` (0.50) és `economy` (0.40) viszont a legalacsonyabb értékek, mert a support bajnokok tervezésük szerint alacsony sebzésre és korlátozott arányra játszanak. Ez a legfontosabb és legérzékenyebb pont az egész szerepkörrendszerben: ha a supportot az ADC-vel azonos damage/economy elvárással értékeljük, minden support automatikusan alacsony opscore-t kap, amivel az értelmesség elvész. A jelen konfiguráció ezt expliciten ellensúlyozza.

A `tanking` (1.10) mérsékelt emelése a tank-support és engage-support archetípusokat tükrözi. Az `early_game` (0.80) enyhén csökkentett, mert sok support bajnok a korai fázisban védelmi vagy összekötő szerepet tölt be, nem aktív first-blood-kereső. A `kda` (0.70) szándékos csökkentése azt fejezi ki, hogy a support nem elsősorban kill-generátor, és a haláltűrési paraméterrel együtt kezelendő.

Haláltűrési paraméterek

A `feedscore` számításában a haláltűrési szorzó (`deathTolerance`) szerepkörönként különböző:

9.2. táblázat: Haláltűrési paraméterek szerepkörönként (`DEATH_TOLERANCE`)

Szerepkör	Haláltűrés
TOP	1.00
JUNGLE	1.05
MIDDLE	1.00
BOTTOM	1.15
UTILITY	0.80
UNKNOWN	1.00

Ez a paraméterkészlet finomabb viselkedési elvárásokat fejez ki, mint pusztán a role multiplierek. A `BOTTOM` haláltűrése a legmagasabb (1.15): az ADC-k tipikusan agresszív korai meccseket is vállalnak, és bizonyos számú halál az ADC szerepkörben elkerülhetetlen az optimális játékstílusnál. A `UTILITY` haláltűrése viszont a legkisebb (0.80): a support halálai különösen drágák, mert megszakítják az engage-eket, leengedik a ward-elhelyezési lefedettséget és elvesznek a csapat peel-kapacitásából. Az 1.05-ös jungler-értéke azt tükrözi, hogy a korai invázió és a kockázatos gank-útvonalak inherens részei a jungler feladatának.

A kézi tervezés védelme és korlátai

Ezek a szorzók kézi tervezésűek, játékszemantikán és domain-ismereten alapulnak. A dolgozatban nem állítom, hogy statisztikailag optimálisak, csak azt, hogy:

- átláthatóak és auditálhatók: minden szorzó értéke explicit,
- játékszemantikailag indokolhatók: minden értékhez adható magyarázat,
- elegendően differenciáltak ahhoz, hogy a következő analitikai réteg számára érdemi inputot adjanak.

A kézi tervezés korlátja az, hogy nem tudhatjuk biztosan, a szorzók az optimális értékek-e. Például a JUNGLE map_awareness szorzójának 1.40-ről 1.50-re növelése érdemes lenne-e? A jelenlegi rendszer erre nem ad statisztikai választ. Ez azonban nem csökkenti az implementált rendszer értékét: egy interpretálható, magyarázható és módosítható súlyrendszer, amely Domain Specific Knowledge-ra épül, jobb kiindulópont egy gráfanalitikai elemzés számára, mint egy feketedoboz gépi tanulásos modell, amelynek paraméterei nem értelmezhetők.

A teljesítésbelileg korrektebb irány — amelyet a jövőbeli munkában megfogalmazunk — az adatvezérelt kalibráció lenne: nagy meccsadatkorpuszon tanult súlyok, amelyek visszaigazolják vagy finomítják a jelenlegi kézi szorzókat, miközben megőrzik az interpretálhatóságot.

9.3.3 Feedscore

A `feedscore` tudatosan sokkal egyszerűbb mutató:

$$\text{feedscore} = \text{deaths} - 0.35 \cdot (\text{kills} + \text{assists}) \tag{9.10}$$

Ezt a rendszer szerepkörönként eltérő haláltűréssel módosítja. Ez azért hasznos, mert például egy support halálmintázata nem ugyanúgy értelmezhető, mint egy marksmané. A végeredmény így továbbra is egyszerű és magyarázható marad, de nem tekinti minden szerepkört azonos viselkedésűnek.

A `feedscore` egyik gyakorlati előnye az, hogy kevésbé összetett, mint az `opscore`. Emiatt jó kontrollmetrika az asszortativitási fejezetben: ha a gráfmód váltására ez a mutató erősen reagál, akkor nagy valószínűséggel maga a kapcsolatdefiníció változtatja meg a szerkezetet, nem pedig egy túlbonyolított score-recept instabilitása.

9.3.4 Percentilis alapú normalizálás

A nyers pontszámokat a backend a teljes adatkészlet eloszlásához viszonyítva normalizálja. A jelenlegi implementáció a következő horgonyokat használja:

- alsó horgony: 5. percentilis,
- közép: 50. percentilis,
- felső horgony: 95. percentilis.

Ezután egy szakaszos leképezés történik a $[0, 10]$ tartományra, ahol a medián körüli tartomány humánusabban széthúzott, a legfelső 5% pedig már elérheti a 10-es plafont. Ez azért szerencsés, mert a pontozás így nem egyetlen szélsőséges outlierhez igazodik.

A jelenlegi implementáció középső célértéke **6.6**. Ha a nyers játékosszintű értéket x, az alsó, középső és felső horgonyt pedig rendre f, c és u jelöli, akkor a leképezés intuitív alakja a következő:

$$\text{score}(x) = \begin{cases} 6.6 \cdot \frac{x-f}{c-f}, & \text{ha } x \leq c \\ 6.6 + (10 - 6.6) \cdot \frac{x-c}{u-c}, & \text{ha } x > c \end{cases} \tag{9.11}$$

Az implementáció természetesen levágja a szélsőértékeket és kezeli a degenerált helyzeteket is, de a fenti forma jól mutatja a szándékot: a medián körüli tömeg nem az 5-ös értékhez, hanem egy valamivel élhetőbb középhez igazodik. Ez UX-szempontból és védekezhetőség szempontjából is előnyös, mert a teljes korpusz nem torlódik össze a skála közepe alatt.

Ez egyben azt is jelenti, hogy a `feedscore` és `opscore` nem abszolút mértékegység, hanem datasetfüggő skála. A dolgozat ezért a score-okat elsősorban belső összehasonlító mutatóként kezeli.

9.4 A metrikák jelenlegi empirikus képe

A közvetlenül ellenőrzött adatbázis-pillanatkép alapján a teljes rendszerben a `opscore` átlaga körülbelül **5.721**, míg a `feedscore` átlaga **5.882** volt. A számok önmagukban nem jelentik azt, hogy "az átlagos játékos 5.7-es", mert a skála részben normalizáció eredménye, de azt jól mutatják, hogy a rendszer ténylegesen kitöltött, használható eloszlást állított elő.

Ugyanilyen fontos a mintanagyság kérdése. Az ellenőrzött `players` állapotban:

- az átlagos `match_count` körülbelül **1.355** volt;
- a medián `match_count` **1**;
- legalább 5 meccsel **537** játékos rendelkezett;
- legalább 10 meccsel **337** játékos rendelkezett.

Ez a tény fontos módszertani figyelmeztetés. A score-réteg teljes korpuszra ki van számolva, de a játékosok nagy része rövid megfigyelési múltból származik. Emiatt a dolgozatban minden olyan állításnál, amely stabilabb teljesítményprofilra utalna, különösen óvatosnak kell lenni.

Ezt erősíti az is, hogy a jelenlegi konfigurációban a `minMatchesForMeaningfulScore` értéke mindössze 1. Ez technikailag lehetővé teszi, hogy a rendszer szinte minden megfigyelt játékoshoz számoljon mutatót, de módszertanilag megköveteli, hogy a magasabb rendű interpretációkban külön figyeljünk a mintanagyságra. A score-számítás tehát implementált és hasznos, de a stabilitási állításokat még nem szabad automatikusan ráépíteni.

9.5 Miért volt szükség a korai képletek leváltására?

A korai pontozás ugyan működött, de három szempontból gyorsan szűknek bizonyult:

- túl kevés jellemzőből dolgozott;
- a szerepkörök közötti különbséget nem kezelte elég finoman;
- nehezen volt összekapcsolható a későbbi kutatási analitikával.

Az artifact-szintű modell ezért nem csak pontosságnövelés volt, hanem szerkezeti előrelépés. A végső `opscore` mögött így már több részkomponens áll, amelyek később külön is vizsgálhatók vagy kalibrálhatók.

9.6 Mit nem tud a jelenlegi pontozás?

A pontozórendszer erősen gyakorlatias, de nem állítok róla többet annál, mint amit ténylegesen tud. Jelenleg **nincs** benne gépi tanulásos súlytanulás: a kézi tervezésű szorzók megbízhatóak és átláthatóak, de nem statisztikailag kalibráltak. Ez egy potenciális jövőbeli fejlesztési irány, de a dolgozatban csak azt tekintem késznek, ami a kódbázisban már ténylegesen megvalósult.

9.7 Kiegészítő ország-alapú elemzés

A repository megtartja a korábbi, klaszteralapú ország-hozzárendelési és régiós összegzési vonalat is. Az `assign_countries.py` egy AI-alapú, játékosnév-klaszterekből következtető kísérleti címkéző pipeline-t valósít meg, míg a `conclude.py` ország szerinti aggregációt készít a `players` táblából.

Ezt a dolgozatban azonban tudatosan másodlagos, *kiegészítő* modulnak tekintek. Ennek oka, hogy bár a kód implementált és működőképes, az ország-hozzárendelés bizonytalansága jóval nagyobb, mint a gráfstruktúrára és teljesítménymutatókra épülő elemzéseké. Ezért a dolgozat fő állításait nem erre, hanem a gráfanalitikai rétegre építem.

10. fejezet

Gráfépítés, klaszterezés és perzisztencia

10.1 A gráf felépítésének alapelve

A rendszer alapgráfja meccsekből épül. Minden mérkőzés résztvevői között páronként kapcsolat keletkezik, és ezek a kapcsolatok a teljes adathalmazban aggregálódnak. A Python oldali gráfépítés első közelítésben egy súlyozott, irányítatlan együtt-játszási hálót állít elő, ahol az él súlya az adott két játékos közös előfordulásainak számát jelzi.

Ez a kezdeti forma azonban túl zajos lenne önmagában. Egyetlen közös meccs még nem jelent érdemi társas kapcsolatot, ezért a pipeline már korai szinten is preferálja az ismétlődő kapcsolatokat.

A `backend/new_build_graph.py` alapján a gráfépítés explicit módon a meccskönyvtár állományaiból indul ki, és a jelenlegi formában az `EUN1_` prefixű, `.json` kiterjesztésű állományokat dolgozza fel. A szkript mérkőzésenként végigmegy a résztvevők listáján, és minden résztvevőpárra növeli az aggregált él súlyát. A korai Python nézet ebből a szempontból teljes co-presence háló: nem különíti el szigorúan a szövetségesi és ellenfélkapcsolatot, hanem a közös mérkőzésbeli előfordulást veszi alapnak.

Ha a mérkőzések halmazát M, az egyes mérkőzések résztvevőhalmazát pedig P_m jelöli, akkor az alapél-súly a korai modellben a következőképpen fogható meg:

$$w(u, v) = \sum_{m \in M} \mathbf{1}[\{u, v\} \subseteq P_m] \tag{10.1}$$

Ez a leegyszerűsített nézet jól használható populációs klaszterezésre, de önmagában még nem elég a signed analitikához. Éppen ez indokolta a későbbi Rust oldali párkapcsolat-modell bevezetését, ahol az `allyWeight` és `enemyWeight` külön mezőként él tovább.

10.2 Szűrés és zajcsökkentés

A gyakorlatban a projekt egyik legfontosabb heurisztikus döntése a gyenge élek szűrése. A Python oldali gráfépítésben a tipikus munkamód a `min_weight >= 2` küszöb, ami azt jelenti, hogy az egyszeri, véletlenszerű együtt-előfordulások jelentős része kiesik a központi hálóból.

Ez a küszöb nem abszolút igazság, hanem interpretálhatósági kompromisszum:

- magasabb küszöb esetén a háló tisztább, de sok szerkezet eltűnik;
- alacsonyabb küszöb esetén több kapcsolat marad, de nő a zaj.

A dolgozat későbbi eredményei – különösen az asszortativitás, a centralitás és a core-periphery értelmezés – erősen függenek ettől a választástól, ezért ezt nem technikai részletnek, hanem az elemzés részének tekintem.

A zajcsökkentés azonban nem áll meg az élküszöbnél. A gráfépítő pipeline kötegelt SQL-lekérdezéssel kapcsolja hozzá a `players` táblából a normalizált profilokat, így a gráf nem lebeg a meccs-JSON-ok fölött önmagában, hanem rögtön összekötődik a score-réteggel. Ez azért hasznos, mert a klaszterösszegzések és a későbbi analitikák már ugyanarra a kanonikus játékosvilágra támaszkodnak.

10.3 Python klaszterpipeline

A `new_build_graph.py` a jelenlegi fő Python pipeline. Feladatai:

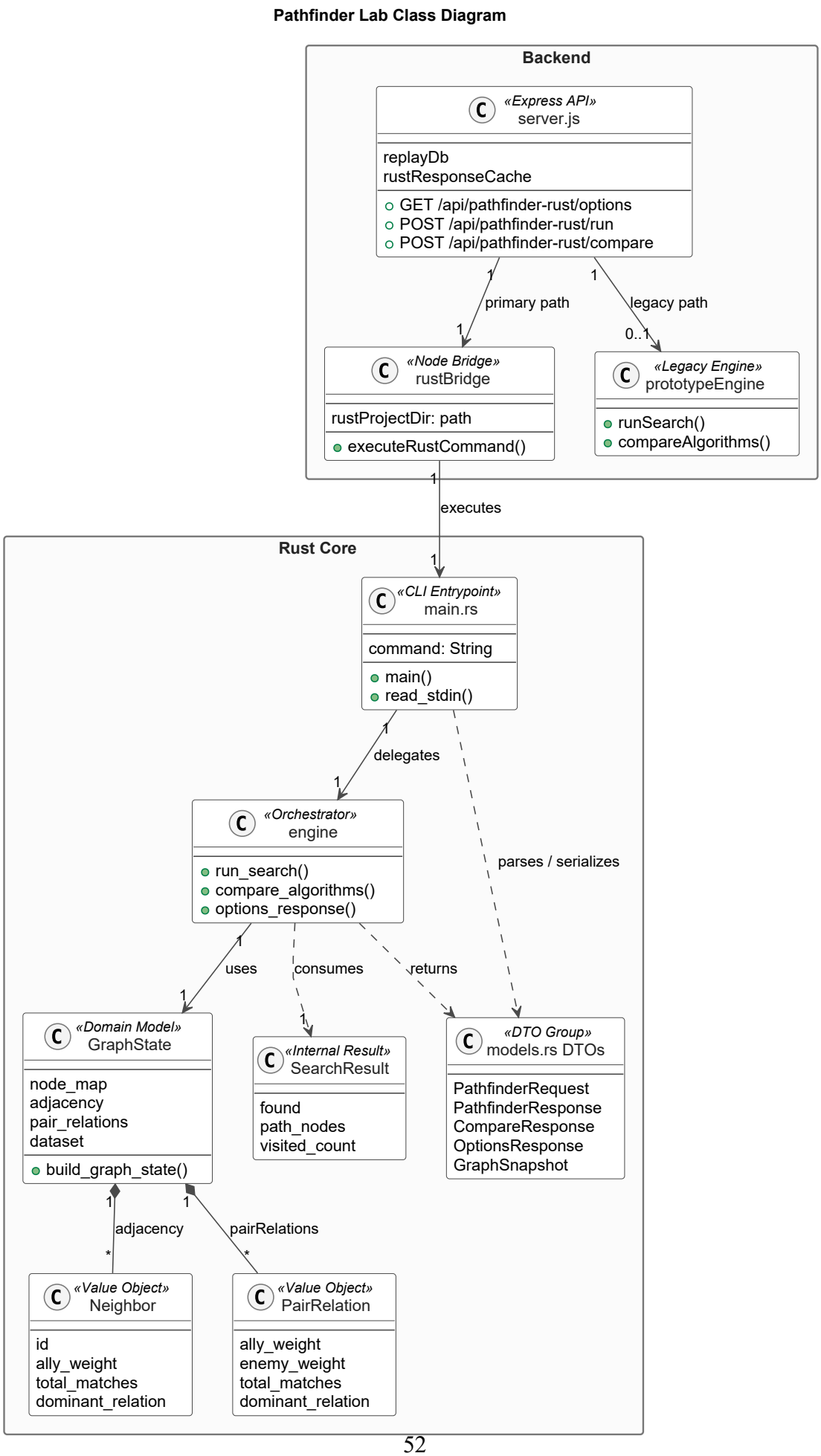
- meccs-JSON állományok betöltése;
- súlyozott gráf felépítése;
- játékosstatisztikák hozzákapcsolása az adatbázisból;
- gyenge élek szűrése;
- közösségek meghatározása a mohó modularitásmaximalizálási eljárással;
- export JSON és HTML formátumban, többek között PyVis alapú vizualizációval [4];
- klaszterek perzisztálása `python_population` típusként.

A pipeline egyik jó tulajdonsága, hogy a vizualizációs export és az adatbázis-perzisztencia nem kizárják, hanem kiegészítik egymást. Az exportfájlok továbbra is hasznosak debugoláshoz és gyors ellenőrzéshez, de a rendszer szempontjából már nem azok az elsődleges igazságforrások.

10.4 Klaszterek mint első osztályú adatok

A jelenlegi rendszerben a klaszterek önálló, lekérdezhető adatbázis-objektumok. A `clusters` tábla a klaszter metaadatait, a `cluster_members` pedig a tagsági sorokat tárolja. Ez több okból fontos:

- a frontendnek nem kell minden alkalommal újragenerálnia a klaszterlogikát;
- a Rust futtatókörnyezet és a Python pipeline ugyanazt a perzisztens modellt tudja használni;
- a későbbi analitikai kiterjesztésekhez már rendelkezésre áll a közösséginformáció.



10.1. ábra: A Pathfinder osztálydiagramja — a gráf- és klaszterobjektumok tárolási modellje

10.5 Két klasztercsalád együttélése

A projekt jelenlegi állapotának egyik fontos sajátossága, hogy két külön klasztercsalád létezik egymás mellett:

- `python_population`: a teljesebb, populációs nézetet szolgáló közösségek;
- `rust_pathfinding`: a futásidejű útvonalkereséshez és globális nézethez használt klaszterek.

Ez első ránézésre redundánsnak tűnhet, de valójában funkcionális szétválasztás. A Python klaszterek a teljes háló, a népességi mintázatok és a kiegészítő ország-alapú réteg szempontjából hasznosak. A Rust klaszterek viszont a gyors futásidejű grafikus és útvonalkereső nézetekhez vannak igazítva. Mivel eltérő célokat szolgálnak, nem célszerű erőből egyetlen univerzális klaszterfogalomba gyömöszölni őket.

10.6 Kapcsolattípusok aggregálása a későbbi Rust-modellhez

Bár a korai Python pipeline elsősorban együtt-játszási nézetből indul ki, a projekt egészének szempontjából fontos volt, hogy a kapcsolatok mögötti ally/enemy információ se vesszen el. Ez vezetett el ahhoz, hogy a Rust gráfállapot már nemcsak súlyozott élként, hanem külön `allyWeight` és `enemyWeight` mezőpárral gondolkodik.

Ez a döntés egyfajta híd a korai és a jelenlegi rendszer között:

- a Python réteg megőrzi a teljesebb, populációs szemléletet;
- a Rust réteg ugyanabból a meccstörténetből részletesebb kapcsolatszemantikát épít;
- a `battle-path` nézet, az asszortativitási összehasonlítás és a módszertani signed-balance kísérlet csak emiatt válik lehetővé.

11. fejezet

Klaszterekhez kötött ország- és régióelemzés mint korábbi modul

11.1 A modul eredeti szerepe

A projekt egy korábbi szakaszában a klaszterekből nemcsak gráfstruktúrát, hanem földrajzi jellegű következtetéseket is szerettem volna kinyerni. Az elképzelés az volt, hogy ha egy klaszteren belül a játékosnevek és tag-ek alapján valamilyen domináns ország vagy régió valószínűsíthető, akkor a közösségekhez aggregált teljesítmény- és viselkedésprofil is rendelhető.

Ennek a gondolatnak a kódbázisban ma is megvannak a nyomai:

- `fetch_clusters.py` a klaszterekből névlistás exportot készít;
- `assign_countries.py` kísérleti országcímkézést futtat;
- `conclude.py` aggregált ország-alapú összegzést készít.

11.2 Miért volt ez akkor logikus irány?

A korai rendszerben a klaszterek még elsősorban "emberi csoportként" tűntek izgalmasnak. Ha egy csoportban sok visszatérő játékos szerepelt, könnyen felmerült, hogy ezek valamilyen nyelvi, nemzeti vagy régiós mintázatot is követhetnek. Ez különösen vonzó ötlet volt azért, mert a puszta gráfstruktúrán túl egyfajta társas magyarázó réteget ígért.

Ezen a ponton azonban fontos különválasztani az intuíciót és a védhető állítást. Az országhozzárendelés soha nem közvetlen Riot-mezőből származott, hanem név- és klaszternintázatok alapján következtetett címke volt. Ez jóval bizonytalanabb, mint a közvetlenül meccs-JSON-ból és gráfkapcsolatokból levezetett mutatók.

11.3 A jelenlegi, ellenőrzött állapot értelmezése

A dolgozat készítésekor ellenőrzött `playersrefined.db` pillanatképben a `country` mező nem tartalmazott érdemi, széles körben kitöltött adatot. Ez azt jelenti, hogy a korábbi ország-alapú ág

ugyan implementációs értelemben létezik, de a mostani rendszerbemutatás fő bizonyító anyagát nem erre lehet építeni.

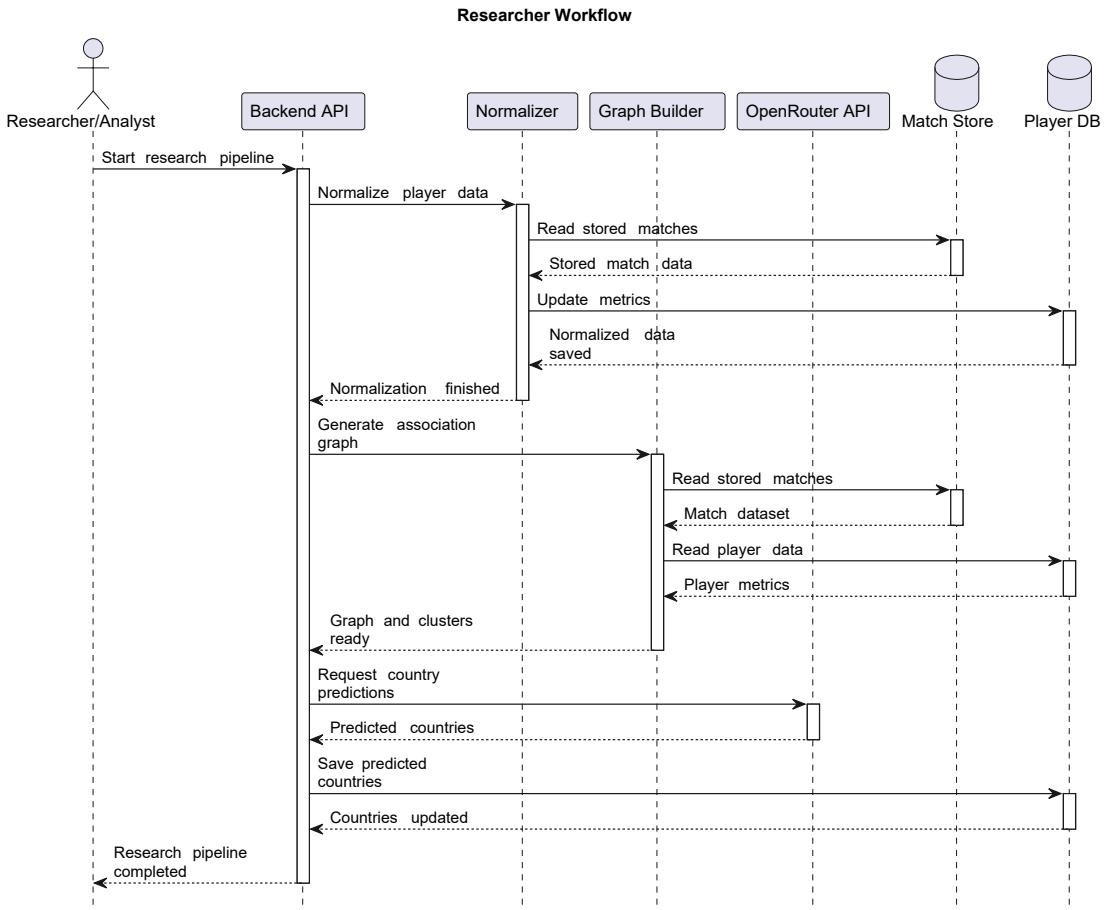
Ezt a dolgozatban tudatosan így is kezelem:

- történeti és módszertani szempontból fontos modul;
- a projekt fejlődésének valódi része;
- de a jelenlegi verzióban nem ez a legerősebb, legstabilabb kutatási réteg.

11.4 Miért maradjon mégis a dolgozatban?

Azért tartom fontosnak, hogy ez a modul ne tűnjön el egyetlen félmondatban, mert jól mutatja a projekt gondolkodásmódját. A rendszer nem egyetlen előre rögzített hipotézis mentén fejlődött, hanem több analitikai irányt is kipróbált. A country-ág végül nem a legerősebb bizonyítási vonallá vált, de hozzájárult ahhoz, hogy a klasztereket ne puszta geometriai objektumként, hanem értelmezési egységként kezeljem.

Ebből a szempontból a modul megőrzése a dolgozatban nem teher, hanem hitelességi előny: látszik belőle, hogy a projekt történetileg valóban fejlődött, és nem utólag lett egyetlen steril narratívára visszavágva.



11.1. ábra: A korai ország-klaszter pipeline szekvenciadiagramja — a frontend, a backend és a Python worker közötti folyamat az ország-hozzárendelési kísérletben

12. fejezet

Rust futtatókörnyezet és útvonalkeresés

12.1 Miért került a futásidejű réteg Rustba?

A projekt korai szakaszában elegendő volt egy könnyebb, főleg frontend-demóra épülő útvonalkereső prototípus. Ahogy azonban a háló, a felület és az analitikai ambíciók nőttek, egyre nyilvánvalóbbá vált, hogy a futásidejű gráfmodellnek külön, stabilabb végrehajtási környezetre van szüksége.

A Rust bevezetésének legfontosabb okai:

- nagyobb teljesítmény és kiszámíthatóbb memóriakezelés,
- determinisztikusabb, exportálható futásidejű gráfállapot,
- pontosabb kontroll a gráfstruktúrák és algoritmusok felett,
- tisztább határ a böngésző és a számítás között.

12.2 A Rust gráfállapot

A Rust réteg a gráfot közvetlenül a meccs-JSON állományokból és a játékos-adatbázisból építi fel. Két nézetet hoz létre:

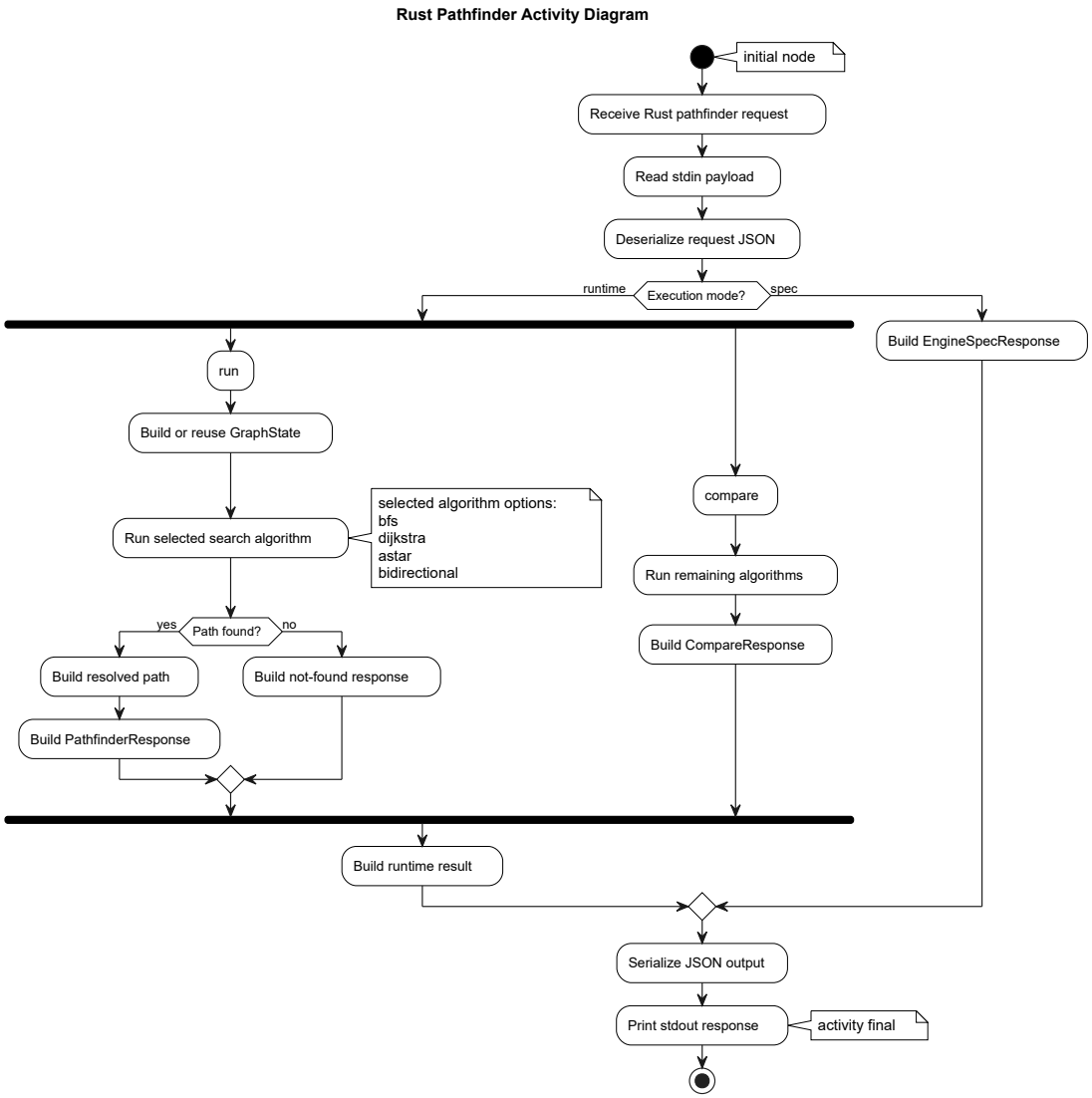
- **population graph**: globális, súlyozott együtt-játszási nézet;
- **pathfinder graph**: olyan futásidejű modell, amely már külön tárolja az `allyWeight`, `enemyWeight`, illetve `totalMatches` értékeket.

Ez az a pont, ahol a rendszer átmegy egyszerű együtt-játszási hálóból olyan gráffá, amelyen már nemcsak közösségeket, hanem előjeles kapcsolatokat és többféle útvonalmódot is lehet vizsgálni.

A `backend/pathfinder-rust/src/engine/graph.rs` alapján a `GraphState` valójában több, egymásra épülő réteget tart egyben. A szomszédsági listák mellett tárolja a `pair_relations` szerkezetet, a játékosprofilok gyors elérését, a klasztertagságot, a klaszter-összegzéseket, a landmark-távolságokat, a minimumköltségeket és az aktív datasethez tartozó snapshot-metaadatot is. Ez azt jelenti, hogy a runtime nem pusztán kereséshez optimalizált gráf, hanem a kutatási analitikák és az exportok közös memóriabeli alapja.

12.2.1 A gráfállapot felépítési sorrendje

A `build_graph_state()` rutin sorrendje önmagában is fontos architekturális állítás. A motor előbb feloldja az aktív dataset útvonalait, betölti a játékosprofilokat, felépíti az élstruktúrát, létrehozza vagy frissíti a Rust oldali klasztereket, majd ezután készíti elő a gyorsító struktúrákat, például a landmark- és minimumköltség-táblákat. Vagyis a teljesítményoptimalizálás nem egy üres, steril gráf fölött történik, hanem egy már szemantikailag gazdag állapoton.



12.1. ábra: A Rust Pathfinder keresési tevékenységdiagramja — a négy algoritmus döntési pontjai és visszaterjesztési lépései

12.3 Támogatott útvonalkereső algoritmusok

A jelenlegi Rust futtatókörnyezet négy algoritmust támogat:

- BFS,
- Dijkstra,

- kétirányú keresés,
- A*.

Az algoritmusok két különböző élértelmezésben futhatnak:

- `social-path`: szövetségesi kapcsolatok mentén,
- `battle-path`: összes ismétlődő kapcsolaton.

A súlyozott mód esetén az erősebb, többször ismétlődő kapcsolatok olcsóbb élköltséget kapnak. Ez nem puszta UX-extra, hanem egy olyan modellválasztás, amely azt fejezi ki, hogy az erősebb ismétlődő viszonyok „közelebb vannak” egymáshoz a gráfban.

A `search.rs` konkrét implementációja különösen egyszerű költségmodellt használ, ami erősíti a magyarázhatóságot. Súlyozatlan módban minden él költsége fix `1000`. Súlyozott módban a rendszer az él súlyának reciprokával arányos, egészre kerekített költséget használ:

$$c(u, v) = \left\lceil \frac{1000}{w(u, v)} \right\rceil \tag{12.1}$$

ahol $w(u, v)$ az adott útvonalmódnak megfelelő kapcsolaterősség. Ennek jelentése intuitív: a sokszor ismétlődő kapcsolat „olcsóbb”, vagyis a gráfban közelebb van. Ez a választás egyszerre egyszerű, determinisztikus és könnyen védhető.

12.4 A hivatalos engine-spec szerepe

A Rust réteghez tartozó `engine-spec` kimenet különösen hasznos a dolgozat szempontjából, mert világosan dokumentálja, milyen szerződést vállal a motor. A jelenlegi specifikáció alapján:

- támogatott algoritmusok: `bfs`, `dijkstra`, `bidirectional`, `astar`;
- támogatott útvonalmódok: `social-path`, `battle-path`;
- a `social-path` csak pozitív szövetségesi támaszú éleken fut;
- a `battle-path` bármely ismétlődő ally vagy enemy kapcsolaton átmehet.

Ez a fajta explicitség nem csak fejlesztői kényelmi funkció. A szakdolgozatban ez segít megmutatni, hogy a rendszer viselkedése nem rejtett, hanem leírható és ellenőrizhető.

Ehhez szorosan kapcsolódik a `main.rs serve` módja is, amely stdin/stdout alapú kérés-válasz borítékokkal működik. Ez lehetővé teszi, hogy ugyanaz a Rust motor önálló parancssori eszközként és backend mögötti alfolyamatként is használható legyen. A rendszer így nem keveri össze a gráfmagot a webszerver-felelősséggel: a Rust réteg végrehajtó marad, a HTTP-szintű integráció pedig a Node oldalán történik.

12.5 Egzakt A* és alsó becslések

A rendszer A* implementációja tudatosan úgy lett kialakítva, hogy ne vizuális trükk, hanem *helyes legrövidebb út algoritmus* legyen. A dokumentáció és a kódbázis alapján a heurisztika két alsó becslést kombinál:

- landmark-alapú ALT alsó korlát,
- klaszterugrás-alapú alsó becslés.

A végső heurisztika ezek maximuma. A térbeli layout-távolság legfeljebb tie-break jelleggel jelenik meg, nem hordozza a helyességi garanciát. Ez a döntés különösen fontos szakdolgozati szempontból, mert így az A* gyorsítás nem megy a korrekt eredmény rovására.

12.5.1 A landmark-alapú ALT heurisztika

Az ALT (A* + Landmarks + Triangle inequality) heurisztika [15] a következő elvből indul ki. Ha L egy landmark-csomópont és $d(u, L)$ az u csomópontból L-be vezető legrövidebb út ismert távolsága, akkor a háromszög-egyenlőtlenség alapján:

$$d(u, t) \geq |d(L, t) - d(L, u)| \quad \text{minden } L \text{ landmarkra} \tag{12.2}$$

Ha több landmark is rendelkezésre áll, a legjobb (legmagasabb) alsó becslés választandó. A rendszer az összes konfigurált landmark-ra kiszámolja ezt a korlátot, és a maximumukat adja az A* heurisztikaként.

A landmark-távolságok (`landmark_distances`) a `GraphState` felépítésekor előre kiszámítódnak minden `ModeKey` (SocialUnweighted, SocialWeighted, BattleUnweighted, BattleWeighted) kombinációra Dijkstra-futtatással. Ez egyszeri előfeldolgozási lépés, amelynek eredménye az összes keresési kérésre újrafelhasználható.

12.5.2 A landmark-kiválasztás stratégiája

A `choose_landmarks()` függvény a kiindulási landmark-halmazt a következő prioritással állítja össze:

1. hybrid cluster bridge-csomópontok: ezek bridgeként és klasztertagként is részt vesznek, ezért nagy valószínűséggel sok legrövidebb úton vannak rajta;
2. klaszterközéppontok;
3. végül egyéb, nagy fokszámú csomópontok, ha az előzők száma nem elég.

Ez a stratégia garantálja, hogy a landmarkok a gráf szemantikailag fontos helyein legyenek, nem véletlenszerűen. Minél jobb helyen van egy landmark, annál jobb (magasabb) alsó becslést ad.

12.5.3 Klaszterugrás-alapú becslés és tie-break

A klaszterugrás-alapú alsó becslés (`cluster_hops`) a forrás és cél klaszter közötti minimális klaszterugrásokon alapul. Ez egy külön Dijkstra-futtatás a klaszter-szintű gráfon (ahol a csomópontok klaszterek, az élek klaszterközi kapcsolatok), és az eredmény az inter-klaszter távolság. Bármely $u \to t$ útvonalnak legalább annyit kell haladni klasztereken keresztül, mint az inter-klaszter legrövidebb út, így ez admisszibilis alsó becslés.

A tie-break (`tie_break_distance`) az egyenlő becsült összköltségű csomópontok között a klaszterközpont-távolság alapján bont döntetlen, elkerülve a determinisztikusan rossz sorrendeket.

Ez azért lényeges, mert a képernyőn látható közelség és a gráfbeli költség nem ugyanaz a fogalom. Ha a rendszer a layoutot valódi heurisztikaként használná, az könnyen inadmisszibilissé tenné a keresést. A jelen megoldás ezt tudatosan kerüli el.

12.6 Backend API és orchestráció

[PLACEHOLDER]

Rust analitikai parancsfolyam — Node backend Rust CLI-hívásai stdin/stdout csatornán és JSON válaszok

12.2. ábra: A Node.js backend és a Rust CLI közötti parancsvezérelt kommunikáció sémája

A Node/Express réteg a Rust számára nem konkurens implementáció, hanem vezérlő és integrációs héj. A fontosabb végpontok:

- `GET /api/pathfinder-rust/options`,
- `GET /api/pathfinder-rust/global-view`,
- `GET /api/pathfinder-rust/engine-spec`,
- `POST /api/pathfinder-rust/run`,
- `POST /api/pathfinder-rust/compare`,
- `POST /api/pathfinder-rust/player-focus`,
- `POST /api/pathfinder-rust/signed-balance`,
- `POST /api/pathfinder-rust/assortativity`,
- `POST /api/pathfinder-rust/balance-sweep`,
- `POST /api/pathfinder-rust/assortativity-significance`.

Ez a struktúra azért szerencsés, mert a frontend így stabil JSON-szerződéseken keresztül kommunikálhat a háttérréteggel, anélkül hogy tudnia kellene a belső gráfreprezentáció minden részletét.

A szerződésesség itt gyakorlati hasznot is hoz. A backend kezeli a datasetváltást, szükség esetén üríti a régi runtime-állapotot, átadja az aktuális útvonalakat a Rust motornak, és a frontend felé már csak a fogyasztható választ adja tovább. Ez csökkenti annak esélyét, hogy a frontend és a motor eltérő adatállapotra gondoljon ugyanabban a pillanatban.

12.7 Strukturális egyensúly mint kísérleti analitika

A `signed_balance.rs` modul a többmeccses kapcsolatokat előjeles egyszerű gráffá projektálja. A fő döntések a következők:

- pozitív él: domináns `ally` kapcsolat;
- negatív él: domináns `enemy` kapcsolat;
- támogatási küszöb: alapértelmezésben legalább 2 kapcsolat;
- döntetlen esetén: alapértelmezés szerint kizárás.

A modul teljesen géppel olvasható választ ad vissza, többek között:

- a projektált gráf összefoglalóját,
- a kiegyensúlyozott és kiegyensúlyozatlan triádok számát,
- a triádtípusok eloszlását,
- példatriádokat,
- a legtöbb instabil triádban részt vevő játékosokat,
- opcionálisan klaszterszintű összegzést.

Ez a rész a rendszerben továbbra is megvalósított analitika, de nem a dolgozat egyik fő empirikus pillére. A formális projekció működik és reprodukálható; a korlát az, hogy az `enemyWeight` nem elég megbízható negatív társas kötés ahhoz, hogy a kapott arányokat erős szociális következtetésként kezeljem. Emiatt a modul a módszertani reflexió és a kísérleti diagnosztika része marad.

12.8 Asszortativitás mint megvalósított analitika

Az `assortativity.rs` modul a kiválasztott metrika és gráfmód mellett Pearson-korrelációt számol az élek két végpontja között, mindkét endpoint-orientációt figyelembe véve. A támogatott változók jelenleg:

- `opscore`,
- `feedscore`,
- `social-path`,
- `battle-path`.

A válasz nem csak egyetlen szám, hanem tartalmazza az analizált élmennyiséget, a kihagyások okait, a klaszteren belüli és klaszterek közötti bontást, valamint az erős és gyenge kapcsolatok szerinti bontást is. Ez kifejezetten erősíti a dolgozat védhetőségét, mert nem „díszanalitika”, hanem ellenőrizhető kimenet keletkezik.

Egy ilyen modul szakdolgozati értékét nem pusztán az adja, hogy „kiad egy korrelációt”, hanem az, hogy a kimenetben a mintaképzés is látható. A kihagyott élek, a hiányzó metrikák és a minimum match-count szabályok nélkül az asszortativitás sokkal kevésbé lenne védhető, mert nem lenne világos, milyen adatokon képződött az eredmény.

12.9 A Graph V2 artifact-pipeline és a Rust motor összefüggése

A Graph V2 pipeline egy specializált előre-generáló folyamat, amely a Rust motortól önállóan is futtatható és a böngészőnek fogyasztható artifact-csomagot állít elő. A Graph V2 kimenetei:

- `manifest.json`: a teljes artifaktum metaadatai (matchCount, nodeCount, edgeCount, generationMs);
- `node_meta.json`: csomópontszintű metaadatok;
- `node_metrics.u32`: tömörített numerikus csomóponti mutatók bináris formátumban;
- `edge_pairs.u32`: él-végpontpárok tömbje;
- `edge_props.u32`: élhez tartozó kódolt tulajdonságok (ally/enemy jelzés, súly);
- `cluster_meta.json`: klaszterszintű leíró adatok, orbitScore, orbitRadius, bridge-jelzők;
- `summary.md`: emberi és gépi olvasásra egyaránt alkalmas összegző fájl.

A Graph V2 és a fő Rust runtime gráfállapota (`GraphState`) szándékosan szétválasztott: a Graph V2 előre kiszámított, determinisztikus exportra optimalizált, a `GraphState` futásidejű útvonalkeresésre és kutatási analitikára. Ez teszi lehetővé, hogy ugyanabból a datasetből a böngésző statikus, gyors renderingre alkalmas artifaktumot kapjon, miközben az asszortativitás, a központiság és a kísérleti signed-balance parancsok futásidőben, a teljes gráfra futnak.

A Graph V2 clusteringAlgorithmVersion (`bounded-ally-groups-v2`) egy 20-as maximális csoportméretre korlátozott, ally-bizonyíték alapú klaszterezési algoritmus. Ez különbözik a Python-oldali moduláris klaszterezéstől: nem a teljes gráf modularitásmaximalizálására törekszik, hanem vizualizálhatóan kezelhető méretű, ally-centrikus csoportokat határol el. A két klaszterezési megközelítés együttes jelenléte a rendszerben nem redundáns, hanem különböző célokat szolgál.

12.10 Kísérleti futtatók és robusztusság

A Rust rétegben külön parancs és backend-végpont létezik az asszortativitás randomizációs szignifikanciatesztjéhez, valamint a signed-balance érzékenységvizsgálathoz. Ezek létezése azért fontos, mert a projekt nem áll meg az első leíró számnál, hanem támogatja a robusztussági és nullmodell-szerű összehasonlításokat is. A dolgozatban ezt úgy kezelem, mint kutatási infrastruktúrát: az asszortativitás központi eredmény, a signed-balance pedig azt dokumentálja, hol húzódik az előjeles projekció értelmezési határa.

13. fejezet

Pathfinder Lab és algoritmus-összehasonlítás

13.1 A Pathfinder Lab szerepe a teljes rendszerben

A Pathfinder Lab a projekt egyik legfontosabb "átfordító" modulja: itt válik a gráf a felhasználó számára nem csak megtekinthető hálózattá, hanem bejárható, kérdezhető kapcsolati térré. A lab célja nem pusztán az, hogy két játékos között egy útvonalat megjelenítsen, hanem az, hogy algoritmusonként is láthatóvá tegye, mit jelent ugyanaz a keresési feladat eltérő számítási szemléletekkel megoldani.

Ez szakdolgozati szempontból különösen értékes, mert a vizualizáció és az algoritmika itt közvetlenül találkozik. A hallgató vagy a bíráló nem csak egy képletet vagy egy táblázatot lát, hanem egy olyan felületet, amely mögött világos gráfelméleti és rendszerszintű döntések vannak.

13.2 A támogatott algoritmusok összehasonlítási logikája

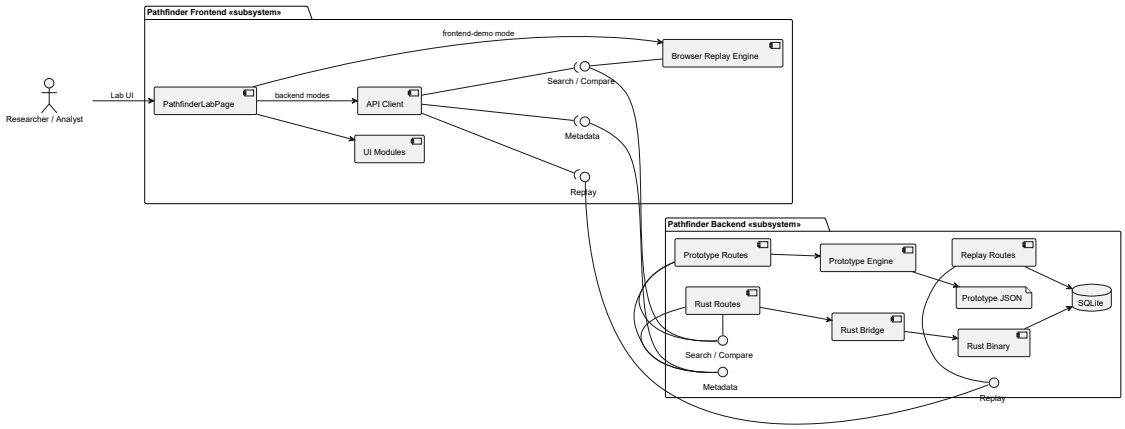
A jelenlegi rendszerben négy algoritmus érhető el összevethető módon: BFS, Dijkstra, kétirányú keresés és A*. Ezek ugyanarra a problémaosztályra adnak választ, de eltérő feltételezésekkel:

13.1. táblázat: A Pathfinder Labban szereplő algoritmusok összevetése

Algoritmus	**Mikor természetes?**	**Megjegyzés a projektben**
BFS	súlyozatlan legrövidebb út	Jó baseline, könnyen magyarázható
Dijkstra	súlyozott utak	Referenciaalgoritmus a költség-alapú módokban
Kétirányú keresés	nagyobb gráf, kevésbé irányított keresés	Gyakorlati gyorsítás bizonyos esetekben
A*	jól kialakított alsó becsléssel	A projektben egzakt marad, nem csak vizuális heurisztika

A dolgozatban az összehasonlítás fő célja nem az, hogy "győztest" hirdessek, hanem az, hogy

megmutassam: a rendszer képes ugyanazon gráfmodellt több algoritmikus nézőpontból kezelni, és a különbségek ellenőrizhetően visszakövethetők.



13.1. ábra: A Pathfinder Lab és a frontend komponensstruktúrája — a keresési kérelmek útja a React nézettől a Rust motorig

13.3 Kézi vezérlés, replay és magyarázhatóság

A Pathfinder Lab egyik erőssége az, hogy a kimenetet nem feketedoboz-szerűen adja vissza. A replay panel és a vizuális léptetés lehetővé teszi, hogy a felhasználó:

- megértse, milyen játékosokon megy keresztül az útvonal;
- lássa a gráfmód és a súlyozás hatását;
- összevesse az alternatív futtatásokat.

Ez a döntés oktatási és védhetőségi szempontból is hasznos. A rendszer nem elrejti az algoritmust, hanem láthatóvá teszi a döntési következményeket.

A `search.rs` alapján a replay mögött strukturált trace-adat áll. A motor minden releváns lépéshez képes rögzíteni az aktív csomópontot, a frontier rövidített állapotát, a meglátogatott csúcsok halmazát és a kiemelt éleket. Ezáltal a BFS, Dijkstra, kétirányú keresés és A* nem csak végső útvonal-hosszal hasonlítható össze, hanem a bejárási stratégia szintjén is. A magyarázhatóság tehát nem utólagos vizuális díszítés, hanem a keresőmotor válaszformátumának része.

[PLACEHOLDER]

Pathfinder Lab tesztfuttatás — összehasonlító állapot két algoritmus útvonal-eredményével

13.2. ábra: Pathfinder Lab: összehasonlító állapot a tesztelési futtatásból

13.4 Az algoritmusok részletes leírása

13.4.1 Szélességi keresés (BFS)

A BFS az egyszerűbb esetekben természetes baseline: súlyozatlan módban garantáltan a minimális lépésszámú útvonalat adja, és a bejárás sorrendje teljesen meghatározott. A `search_bfs()` implementáció egy FIFO-sort (`VecDeque`) tart fenn. Az algoritmus leáll, amint a célcsomópont először felbukkan a szomszédsági listán, ezáltal elkerülve felesleges kiterjesztést.

A trace-rögzítés szempontjából a BFS egyszerű, de informatív: minden "expand" és "discover" lépés rögzíthető, és az utólagos replay során jól látható, hogyan terjed ki a keresési határ egyenletesen minden irányban.

13.4.2 Dijkstra-algoritmus

A Dijkstra-algoritmus [13] binomiális kupac helyett bináris kupacot (`BinaryHeap`) használ, mert a Rust standard könyvtárban ez a beépített prioritásos sor. A `QueueState` struktúra lexikografikusan rendez: először az alacsonyabb `cost`, másodlagosan az alacsonyabb lexikográfiai csomópont-id szerint. Ez garantálja, hogy azonos súlyú utak esetén determinisztikusan mindig ugyanazt az útvonalat kapjuk.

A `traversal_cost()` a kérésben megadott módnak megfelelően számít:

$$c(u,v) = \begin{cases} 1000 & \text{ha súlyozatlan mód} \\ \lceil 1000 / \max(1, w(u,v)) \rceil & \text{ha súlyozott mód} \end{cases} \tag{13.1}$$

A kapcsolaterősség ($w(u,v)$) social-path módban az `allyWeight`, battle-path módban a `total_matches` érték. Az egységteszt (`dijkstra_uses_weighted_costs_to_prefer_stronger_route`) ellenőrzi, hogy a súlyozott mód valóban a közvetett, de erős útvonalat preferálja a gyenge közvetlen éllel szemben.

13.4.3 Kétirányú keresés

A kétirányú BFS a forrásból és a célból párhuzamosan terjeszt ki egy-egy keresési frontot, és megáll, amint a két front találkozik. Az implementáció (`search_bidirectional()`) felváltva bővíti a forrás-oldali és a cél-oldali sort (`expand_frontier()`). A találkozási csomópont (`meeting_node`) meghatározásakor az út az összefűzött forrás- és cél-részútvonalakból áll össze.

A kétirányú keresés elméleti előnye: sűrű vagy nagy gráfokban a meglátogatott csomópontok száma közelítőleg $O(\sqrt{n})$ lehet az egyirányú $O(n)$-hez képest, ha a gráf véletlenszerű. A project gráfján, ahol a komponensek erősen klaszteresek, a tényleges gyorsulás méréssel ellenőrizhető.

13.4.4 A* keresés és heurisztika

Az A* [14] algoritmus a Dijkstra-algoritmust terjeszti ki célirányos heurisztikával. Az `AStarState` struktúra a `path_cost` (eddigi tényleges útvonalköltség) és a `heuristic_cost` (célhoz vett alsó becslés) összegével rendez. A tie-breaking (`tie_break_distance`) egy másodlagos szempontot nyújt az azonos becsült összköltségű csomópontok közt.

A heurisztika (`heuristic_lower_bound()`) a `GraphState`-ben előre kiszámított `landmark_distances` táblából és `cluster_hops` adatból kombinál alsó becsléses értéket. Ez garantálja az elfogadhatóságot (admissibility): a heurisztika soha nem becsüli túl a valódi hátramaradó útvonalköltsége, ezért az A* helyes legrövidebb-utat ad.

Az egységtesztek minden releváns esetben igazolják, hogy az A* eredménye egyezik a Dijkstra-eredménnyel (`assert_astar_matches_dijkstra`). Ez azt jelenti, hogy az A* nem vizuális gyorsítás, hanem valóban egzakt: csak a keresési sorrend más, az eredmény garantáltan azonos. Tesztelt esetek: súlyozatlan social-path, súlyozott social-path, battle-path, vegyes topológia, döntetlen-nehéz gráf, összekötetlen gráf, forrás==cél eset.

13.5 A gráfmód és a súlyozás együttes hatása

A négy algoritmus két gráfmóddal és két súlyozási beállítással kombinálható. Ez $4 \times 2 \times 2 = 16$ lehetséges futtatási konfigurációt jelent, és mindegyik indokolt lehet más kutatási kérdésnél. A Pathfinder Lab ezt a dimenziót az interfészen keresztül elérhetővé teszi: a felhasználó láthatja, hogyan változik az útvonal és a bejárási stratégia, ha gráfmódot, súlyozást vagy algoritmust vált.

13.2. táblázat: A négy algoritmus összehasonlítása a legfontosabb dimenziók mentén

Algoritmus	**Optimális?**	**Memória**	**Fő erősség a projektben**
BFS	Igen (súlyozatlan)	$O(n)$	Egyszerű baseline, jól magyarázható replay
Dijkstra	Igen (súlyozott)	$O(n+m)$	Referencia-algoritmus, lexikografikus döntetlen-kezelés
Kétirányú	Igen (súlyozatlan)	$O(n)$	Potenciális gyorsulás klaszteres gráfon
A*	Igen (heurisztika elfogadható)	$O(n)$	Célirányos bejárás, egzakt eredmény

13.6 Miért fontos ez túl a UI-n?

A Pathfinder Lab azért kap külön fejezetet, mert valójában nem frontend-extra. Ez az a réteg, ahol a Rust algoritmusok, a backend-szerződések, a perzisztens játékosprofilok és a böngészőben értelmezhető interakció egyetlen működő demonstrációvá állnak össze. Ha a dolgozat célja az, hogy az egész rendszer védhető legyen, akkor ezt a fejezetet nem lehet a vizuális melléklet szintjére leszorítani.

14. fejezet

A 3D bird's-eye sphere és globális hálózati vizualizáció

14.1 A frontend szerepe

A frontend jelenleg már nem egyetlen gráfnézet, hanem több, világosan elkülönített felületből álló mini-rendszer. Ez fontos minőségi ugrás a korábbi állapothoz képest, mert a projekt ekkor vált valóban bemutatható és narratívába rendezhető alkalmazássá.

A fő felületek:

- **Match Analysis**: adatgyűjtési és mérkőzésellenőrző nézet,
- **Pathfinder Lab**: útvonalkeresési és algoritmus-összehasonlító nézet,
- **Full 3D Graph Sphere**: teljes hálózati madártávlat,
- **Assortativity**: teljesítmény-hasonlóságot vizsgáló analitikai oldal,
- **Betweenness Centrality**: Brandes-központisági és bridge/broker elemző oldal.

14.2 Miért nem egyetlen képernyő?

A felület több oldalra bontása nem esztétikai döntés volt, hanem használhatósági és védhetőségi kérdés. Egy szakdolgozati demo során sokkal könnyebb végigvezetni a nézőt azon az úton, hogy:

1. adatot gyűjtünk vagy ellenőrzünk,
2. gráfot és klasztereket generálunk,
3. útvonalakat keresünk,
4. globális struktúrát nézünk,
5. kutatási jellegű elemzést futtatunk.

Ha minden ugyanarra a képernyőre kerülne, a projekt inkább eszköztárnak, mintsem koherens rendszernek hatna.

14.3 A 3D globális nézet

A *bird's-eye sphere* oldal a teljes hálózat vizuális áttekintésére szolgál. A legfontosabb tervezési döntés itt az volt, hogy a böngésző ne valós időben számolja a teljes layoutot, hanem a Rust réteg előre exportáljon minden fontos artifaktumot, és a frontend csak rendereljen.

Ennek több előnye van:

- a layout determinisztikusabb,
- a demók megismételhetők,
- a böngésző terhelése kisebb,
- a vizuális réteg jobban elválik a számítási rétegtől.

14.4 A bird's-eye manifest technikai tartalma

Fontos különbséget tenni a teljes bird's-eye gömb-export és a Graph V2 artifact között. A bird's-eye gömb az összes látható játékost és élkapcsolatot rendereli bele a 3D térbe, míg a Graph V2 csak a minimum szövetségesi támaszú, klasztertagságú csomópontokat tárolja elemzésre alkalmas formában. A két struktúra ezért különböző csúcsszámot mutat ugyanarra a datasetre.

A `flexset` dataset esetében a Graph V2 artifact összefoglalója az alábbi adatokat rögzíti:

- **5741** látható csúcs (klasztertagok és hozzájuk kapcsolódó csúcsok),
- **18 548** látható él,
- **13 572** ally-jellegű él (73.17%),
- **4976** enemy-jellegű él (26.83%),
- **1531** klasztercsoport,
- átlagos klaszterméret **3.750**, legnagyobb **20**,
- generálási idő: **87 472 ms**.

A `soloq` dataset Graph V2 artifaktuma:

- **1538** látható csúcs,
- **4111** látható él,
- **2682** ally-jellegű él (65.24%),
- **1429** enemy-jellegű él (34.76%),
- **870** klasztercsoport,
- átlagos klaszterméret **1.768**, legnagyobb **20**,
- generálási idő: **55 674 ms**.

A két dataset ally/enemy arányának különbsége (flexset: 73/27, soloq: 65/35) szintén strukturálisan értelmes. A Flex Queue-ban a csapatonként öt fő játékstílus természetes módon dúsítja az ally-kapcsolatokat, míg a Solo/Duo Queue véletlenszerűbb csapatbeosztása az ellenfél-találkozások arányát növeli. Ez az arány önmagában is módszertani érv amellett, hogy a `flexset` erősebb asszociatív játékosgráfot hoz létre, de nem bizonyítja, hogy az enemy-élek negatív társas kötésekként értelmezhetők.

14.5 A `flexset` mint asszociatív core-periphery játékosgráf

A Graph V2 vizualizációt ebben a dolgozatban nem közvetlen barátsági vagy ellenségességi hálóként, hanem asszociatív játékosgráfként értelmezem. A csúcsok játékosokat, az élek pedig ismétlődő meccsbeli együtt- vagy szembekerülési bizonyítékot jelölnek. A legerősebb minta nem az előjeles egyensúly, hanem a core-periphery szerkezet: a központi régióban sok kereszt-klaszter szövetségesi kapcsolatot hordozó csoport jelenik meg, míg a külső gyűrűben sok kis, gyengén kapcsolt vagy teljesen híd nélküli lokális ally-csoport található.

A `flexset` artifactban **1531** bounded ally-csoport szerepel, ezek közül **815** bridge-jelölt klaszter, míg **716** külső pályára helyezett csoport nem rendelkezik cross-cluster ally híddal. A legfontosabb vizuális jel nem az, hogy egy klaszter balra vagy jobbra esik-e a képen, hanem a radiális pozíció: a kisebb orbit-radius erősebb hídjellegű, visszatérő ally-kapcsolatokra utal, a külső gyűrű pedig gyengén kapcsolt lokális szigeteket jelöl.

Ez az értelmezés módszertanilag erősebb, mint a signed-balance olvasat, mert nem igényli, hogy az ellenfélként való ismételt találkozást negatív társas kötésnek tekintsük. Elég hozzá az óvatosabb állítás: a Flex Queue match history ismétlődő asszociációs mintázatokat termel, amelyekben lokális ally-csoportok és hídgazdag központi régiók különülnek el.

A teljes rendszer bird's-eye gömb-exportja a backend által előre generált bináris és JSON artifaktumokat fogyaszt, és nem valós idejű számítást végez a böngészőben. Ez a tervezési döntés biztosítja, hogy a nagy gráfok megjelenítése is determinisztikus és reprodukálható maradjon.

A `server.js` és a bird's-eye exportútvonalak alapján a 3D nézet mögött több, külön felelősségű artifaktum áll. A frontend nem egyetlen nagy JSON-fájlt tölt be, hanem manifesztet és bináris tömböket fogyaszt.

14.1. táblázat: A bird's-eye export fő artifaktumai

Fájl	**Szerep**
`manifest.json`	Metaadatok, darabszámok, skálázási és generálási információk
`node_meta.json`	Csomópontszintű leíró metaadatok, frontend-hozzárendelések
`node_positions.f32`	Lebegőpontos koordináták tömbje a gyors rendereléshez
`node_metrics.u32`	Tömörített numerikus csomóponti mutatók
`edge_pairs.u32`	Él-végpontpárok tömbje
`edge_props.u32`	Élhez tartozó kódolt tulajdonságok

Ez a bontás rendszerszinten jól indokolható. A nagy tömegű numerikus adat bináris marad, ami gyorsabb böngészőoldali feldolgozást tesz lehetővé, miközben az emberileg olvasható metaadat külön JSON-rétegben marad. A frontend tehát megjelenít, nem pedig újraszámol.

[PLACEHOLDER]

Graph V2 / 3D Bird's-Eye Sphere nézet — a teljes hálózat klaszterek szerinti színezéssel és pontszerű csomópontokkal

14.1. ábra: A teljes hálózat 3D madártávlati nézete az előre generált Rust-artifaktumokra építve

[PLACEHOLDER]

Klaszter-kiemelés a 3D nézeten — sűrűbb klaszterfoltok és a térbeli elkülönítés

14.2. ábra: Sűrűbb klaszterfoltok és a térbeli elkülönítés a globális nézetben

14.6 Pathfinder Lab

A Pathfinder Lab az a felület, ahol a gráf nem elsősorban látványelemként, hanem algoritmikus térként jelenik meg. A felület támogatja:

- forrás- és céljátékos kiválasztását,
- több algoritmus összehasonlítását,
- visszajátszási és léptetési funkciókat,
- súlyozott és nem súlyozott módot,
- többféle útvonalmód közötti váltást.

[PLACEHOLDER]

Pathfinder Lab — BFS/Dijkstra/A* összehasonlítás és replay interfész a léptetési panellel

14.3. ábra: Pathfinder Lab: algoritmus-összehasonlítás és visszajátszási felület

14.7 Assortativity és központisági oldalak

A kutatás-orientált frontend oldalak közös tulajdonsága, hogy nem elégednek meg egyetlen backend-válasz megjelenítésével. Az asszortativitási és központisági nézetek igyekeznek:

- magyarázni a kontrollok jelentését,
- megmutatni a pipeline-összefüggéseket,
- elkerülni a túl gyors, túl erős társadalmi következtetéseket.

Ez különösen fontos, mert az ilyen típusú analitika könnyen félreértelmezhető. A felület ezért szándékosan tartalmaz interpretációs szövegeket és magyarázó elemeket is. A signed-balance oldal a jelenlegi dolgozati bemutatási útvonalból kikerült: a hozzá tartozó backend/Rust kód megmaradhat kísérleti opcióként, de a fő navigáció és a bemutatási útvonal már az asszortativitásra, a core-periphery gráfolvasatra és a Brandes-központiságra épül.

14.8 Betweenness Centrality oldal

A `BetweennessCentralityPage` a projekt egyik legkomplexebb analitikai nézetje. A felület nem pusztán indítja a Brandes-algoritmust, hanem eredményinterpretációs panelekkel, timing-összehasonlítással és döntési dokumentációval egészíti ki az analitikai kimenetet.

A oldal négy fő részből áll:

- **Konfigurációs fejléc**: gráfmód (`battle-path` / `social-path`), él-súlyozás, minimum edge support és top-N korlát megadása; négy előre definiált preset (Strong 20, Medium 10, Wide 5, Full 1) gyors paraméterváltáshoz;
- **Projekcióolvasat**: a felület automatikusan értelmezi a megtartott él arányát — ha az arány 1% alá esik, “erős kötésű broker elemzés”; 10% felett “teljes gráf közelítő elemzés”;
- **Párhuzamossági összehasonlító**: ha a serial baseline futás engedélyezett, a felület megmutatja a serial és parallel futásidőt, a sebességarány-értékelést és a serial–parallel maximális abszolút eltérést (nulla kell legyen, ha a párhuzamosítás determinisztikus);
- **Broker-rangsor**: az első 6 csúcs kártyás nézete (rank, label, klasztercsoport, degree, strength, raw és normalizált BC), majd teljes táblázat.

A “Dokumentált döntések” panel (algorithm, parallelization, edge support rule, edge cost rule, normalization rule, graph scope) közvetlenül a Rust motor válaszából érkezik: a frontend nem rekonstruálja, hanem visszatükrözi az analitika által rögzített módszertani döntéseket. Ez azt jelenti, hogy a felületen megjelenő szöveges leírások mindig konzisztensek a tényleges futtatással.

Ez a nézet kulcsszerepet játszik a szakdolgozati bemutathatóság szempontjából: nem feketedoboz-eredményt ad vissza, hanem a futás körülményeivel, korlátaival és időbeli teljesítményével együtt prezentálja az eredményt.

14.9 Player Detail oldal

A `PlayerDetailPage` egy játékos-centrikus nézet, amely a kollektív analitikáktól eltérően egyetlen játékos profiljára fókuszál. Az oldal URL-paraméterrel (`?puuid=...`) adresszálható, így a Pathfinder Lab más nézeteiből közvetlenül lehet linkelni egy-egy csomópontra.

A nézet két fő részből áll: a játékoskereső mezőből (`PlayerLookupField`) és a `PlayerPerformanceCard` komponensből. Ez utóbbi összesíti az adott játékos pontszámait (opscore, feedscore), meccsadatait, szerepkör-eloszlását és klasztertagságát egy lapozás nélküli, pásztázható kártyanézetben.

Az oldal létjogosultsága a rendszerben az, hogy az aggregált elemzések (betweenness centralitás, asszortativitás, útvonalkeresési eredmények) sokszor néhány konkrét csomópontra irányítják a figyelmet. A Player Detail oldal lehetővé teszi, hogy ezek a csomópontok azonnal kontextusban is megvizsgálhatók legyenek, anélkül hogy az elemzőnek a nyers adatbázishoz kellene visszanyúlni.

14.10 A Graph V2 artifaktum-pipeline részletei

A Graph V2 pipeline nem egyetlen monolitikus JSON-fájlt hoz létre, hanem egy gondosan megtervezett, kétféle formátumú (bináris és JSON) artifaktum-csomagot, amelyből a frontend oldalak és az analitikai motorok is hatékonyan tudnak táplálkozni.

14.10.1 Az artifact-csomag összetétele

A Graph V2 pipeline a `backend/data/cache/{datasetId}/graph-v2/` könyvtárba generálja a következő fájlokat:

- **manifest.json**: metaadat és méretinformáció — meccszám, csúcsszám, élszám, generálási időbélyeg, verzió;
- **node_meta.json**: csomópont-azonosítók és metaadatok — puuid, label, klaszterazonosító, pozíció (x, y), hídjelölő státusz;
- **node_metrics.u32**: bináris, packed uint32 formátum — minden csomóponthoz sorrendben tárolja az opscore és feedscore értékeket egész skálán;
- **edge_pairs.u32**: bináris élpárok — minden él két csomópont-indexeként tárolva;
- **edge_props.u32**: bináris éltulajdonságok — allyWeight, enemyWeight, totalMatches minden élhez;
- **cluster_meta.json**: klaszterszintű metaadatok — clusterID, memberCount, crossAllySupport, connectedAllyClusterCount, internalAllyEdges, orbitScore, orbitRadius, representative.

[PLACEHOLDER]

Graph V2 artifact-csomag elrendezése: manifest.json, node_meta.json, edge_pairs.u32, edge_props.u32, node_metrics.u32, cluster_meta.json

14.4. ábra: A Graph V2 pipeline bináris és JSON artifact-fájljainak szerkezete

A bináris formátumok (`.u32`) célja a betöltési teljesítmény: a frontend nem kénytelen JSON-parszolást végezni a csúcs- és élmutatókon; a bináris buffer közvetlenül beolvasható a WebGL renderelési pipeline-ba. Ez különösen a 5741 csúcsot és 18548 élt tartalmazó flexset esetén kritikus.

14.10.2 A summary.md mint gépi és emberi kontextus

Minden Graph V2 generálás után a pipeline létrehozza a `summary.md` fájlt, amely:

- tartalmazza az összes kulcsmutatót (meccszám, csúcsszám, élszám, arányok, átlagos élsúly);
- dokumentálja a konstrukciós szemantikát (support-küszöb, tie-policy, klaszterméret-korlát);
- ML-szempontú feature-térképet biztosít a csomópont-, él- és klaszterszintű elemzési jelöltekhez;
- listázza a top bridge-orbit klasztereket;
- tartalmaz interpretációs korlátokat és javasolt elemzési irányokat.

Ennek a fájlnak kettős szerepe van: egyrészt fejlesztői ellenőrzőpontként működik (újragenerálásnál gyorsan ellenőrizhető, hogy a számok konzisztensek-e), másrészt gépi kontextusként is felhasználható — pontosan az a célba vett felhasználási mód, amelynek keretében a jelen dolgozat analitikai fejezetei is készültek.

14.10.3 Verziókezelés és cache-érvénytelenítés

A `graphBuilderVersion` mező (`graph-builder-v2.2`) explicit jelölője az artifaktum-csomag kompatibilitásának. Ha az algoritmus vagy a séma változik, a verzió frissítése egyértelműen jelzi, hogy a korábban generált artifact-csomagok nem felelnek meg az új elvárásoknak, és újra kell generálni azokat.

A `generatedAtUnixMs` Unix-időbélyeg lehetővé teszi a dataset-változás utáni automatikus érvénytelenítés detektálását: ha a meccs-JSON fájlok újabbak, mint a generálási időbélyeg, a rendszer tudja, hogy az artifact elavult.

14.11 A backend API-szerződés

A Rust motor és a frontend közötti kommunikáció a Node.js Express szerver (`backend/server.js`) által közvetített HTTP-végpontokon keresztül zajlik. A szerver nem saját maga végzi a számításokat, hanem a Rust folyamat stdin/stdout csatornáján továbbítja a kéréseket.

14.11.1 Az aktuális főbb végpontok

14.2. táblázat: A backend API főbb végpontjai és szerepük

Végpont	Funkció
`POST /api/pathfinder`	Útvonalkeresési kérés (BFS/Dijkstra/A*/kétirányú)
`POST /api/signed-balance`	Kísérleti strukturális egyensúly-elemzés, nem fő felhasználói útvonal
`POST /api/assortativity`	Asszortativitás számítás
`POST /api/betweenness-centrality`	Brandes-betweenness futtatás
`POST /api/balance-sweep`	Kísérleti előjelprojekciós érzékenységi sweep
`POST /api/assortativity-significance`	Permutációs szignifikanciateszt
`GET /api/graph-snapshot`	Futásidejű gráfállapot-összefoglalás
`GET /api/engine-spec`	A Rust motor specifikációjának lekérése
`GET /api/datasets`	Elérhető datasetek listája
`POST /api/switch-dataset`	Dataset-váltás (Rust réteg újraindítással)

14.11.2 Dataset-propagáció

A dataset-váltás (`/api/switch-dataset`) nem csupán egy konfigurációs bit megváltoztatása. A Rust motor a dataset-specifikáció alapján teljes újraépítést végez: betölti az adott dataset meccs-JSON-jait, klasztereit és játékosprofilját, majd újraszámolja a landmark-távolságokat és a min-cost táblát. Ez az operáció jellemzően néhány másodpercig tart (a flexset esetén a futásidejű gráfépítés a teljes 4980 meccssel és 5741 csúccsal dolgozik).

Ez a közvetlen következménye annak az architektúrális döntésnek, hogy a Rust motor *stateful*: nem minden kérésre, hanem beinduláskor (vagy dataset-váltáskor) tölti be a gráfot. Ez teljesítményi szempontból hasznos — nem kell minden egyes kérés előtt újraépíteni a gráfot —, de azt is jelenti, hogy a gráfállapot egy adott dataset-pillanathoz van kötve.

14.11.3 A JSON-alapú kérés-válasz protokoll

A Rust motor stdin-ről olvas egy JSON-objektumot (a kérés), majd stdout-ra ír egy JSON-objektumot (a válasz). A Node.js szerveroldal ezt a kommunikációt aszinkron Promisként kezeli. Ez a prototokol-tervzés egyszerű és debuggolható: a Rust motor tesztelható önállóan is, CLI eszközként, anélkül hogy a HTTP-szerver futna. A Node.js kizárólag a hálózati integrációért és a dataset-kezelésért felel.

15. fejezet

A strukturális egyensúly mint módszertani határeset

15.1 Miért került le a fő eredmények közül?

A projektben elkészült signed-balance implementáció nem hibás algoritmus. A `signed_balance.rs` modul determinisztikusan előállít egy előjeles egyszerű gráfot, teljesen összekapcsolt triádokat enumerál, majd a Heider–Cartwright–Harary-féle szabály szerint osztályozza őket. A visszaminősítés oka nem technikai, hanem szemantikai: a League of Legends matchmaking adataiban az ellenfélként való ismételt találkozás nem elég erős bizonyíték negatív társas kötésre.

Formálisan a projekció működik:

- az `allyWeight` dominanciája pozitív élként kezelhető, mert a Flex Queue-ban a visszatérő csapattársi viszony részben szándékolt együttjátszást is tükrözhet;
- az `enemyWeight` dominanciája technikailag negatív élként jelölhető, de ez nem azonos ellenségességgel, rivalizálással vagy elutasítással;
- a döntetlen-kezelés, a minimum support és a triádenumerálás auditálható és újrafuttatható;
- a kapott arányok ezért inkább a választott projekció viselkedését mutatják, nem pedig bizonyított társas egyensúlyt.

15.2 Az enemy-él szemantikai problémája

A strukturális egyensúlyelmélet eredeti közege pozitív és negatív társas viszonyokra épül: barátságra, ellenszenvre, bizalomra, elutasításra vagy hasonló relációkra. A jelen projektben azonban egy enemy-él csak azt jelenti, hogy két játékos többször került egymással szembe, mint egy csapatba.

Ez sokféle okból történhet:

- hasonló MMR-sávban játszanak;
- hasonló napszakban queue-znak;
- a Master+ Flex Queue aktív játékospoolja kicsi;
- a matchmaking ismételten ugyanazokat a játékosokat osztja ellenkező oldalra;

- a stabil premade-csoportok miatt a külső játékosok gyakrabban ellenfélként jelennek meg;
- az adatgyűjtési stratégia felerősítheti az ismétlődő találkozásokat.

Ezek közül egyik sem követeli meg, hogy a két játékos között negatív társas kapcsolat legyen. Ezért az `enemyWeight` negatív signed-network élként való kezelése túl erős állítás lenne a dolgozat fő empirikus narratívájához.

15.3 Mit őriz meg a dolgozat?

A signed-balance modul a repositoryban kísérleti opcióként megmaradhat, mert hasznos módszertani tanulságot ad: megmutatja, hogy egy formálisan elegáns gráfelméleti módszer hogyan válhat interpretációs szempontból gyengévé, ha az él szemantikája nem felel meg az elmélet előfeltevéseinek. A magas balance-ratio ezért legfeljebb óvatossági példaként használható: a szám lehet stabil a választott projekció alatt, de önmagában nem bizonyít társas harmóniát vagy strukturális stabilitást.

A dolgozati termékből és a fő frontend bemutatási útvonalból ezért kikerül a Signed Balance oldal és a kombinált Signed Balance + Assortativity futtató. A backend/Rust implementáció megtartható, de a dolgozat fő pillérei a következők:

- a `flexset` asszociatív core-periphery játékosgráfként való értelmezése;
- a `flexset` és `soloq` datasetek kontrollált összehasonlítása;
- az `opscore` és `feedscore` asszortativitási eredményei;
- a Graph V2 konstrukciós és vizualizációs pipeline;
- a Rust útvonalkeresés és Brandes-betweenness központiság.

15.4 Módszertani tanulság

A visszaminősítés erősíti, nem gyengíti a dolgozatot. A dolgozat így nem azt állítja, hogy minden elegáns gráfelméleti modell automatikusan átültethető játékos-telemetriára, hanem azt, hogy az adatkeletkezési mechanizmust minden esetben össze kell vetni a választott modell előfeltevéseivel. A signed-balance esetben a formális számítás helyes, de a negatív él jelentése nem elég erős. Emiatt az elemzés határesetként marad: implementált, dokumentált, de nem fő bizonyíték.

16. fejezet

Asszortativitási elemzés

16.1 Az asszortativitás szerepe a dolgozatban

Az asszortativitás azért lett külön kiemelt analitikai irány, mert egyszerre könnyen magyarázható és módszertanilag tiszta. A kérdés itt nem az, hogy ki "jobb" játékos, hanem az, hogy az összekapcsolt játékosok hajlamosak-e hasonló teljesítményprofilhoz tartozni.

A projektben ez különösen jó illeszkedés, mert a gráf és a játékosprofil már korábban is létezett. Nem kellett új, bizonytalan címkét bevezetni; elég volt a meglévő `opscore` és `feedscore` metrikákat a gráfél-végpontokra vetíteni.

16.2 A számítási definíció

A jelenlegi implementáció numerikus asszortativitást számol Pearson-korreláció formájában. Ha az él egyik végpontján mért értékeket x_i, a másik végpontén mért értékeket pedig y_i jelöli, akkor az együttható a szokásos formában írható fel:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}} \tag{16.1}$$

A projekt ezt két fontos kiterjesztéssel használja:

- külön kezeli a `social-path` és `battle-path` gráfmódot;
- a globális együttható mellett külön bontásokat is képez klaszteren belüli, klaszterek közötti, illetve erős és gyenge kapcsolatok szerint.

A `backend/pathfinder-rust/src/engine/assortativity.rs` egyik lényeges technikai döntése, hogy az irányítatlan éleket mindkét endpoint-orientációval figyelembe veszi a Pearson-akkumulációban. Ez azért védhető, mert az élnek nincs természetes iránya, ugyanakkor a két végpont tulajdonságpárja szimmetrikusan járul hozzá a korrelációhoz. A modul emellett külön számlálja az alacsony támogatás, a hiányzó score és a túl rövid játékostörténet miatt kihagyott éleket is. Így az asszortativitási eredmény nem egy feketedoboz-korreláció, hanem egy jól dokumentált mintaképzési folyamat eredménye.

16.2.1 Él-végponti minta kiválasztása

A Pearson-korreláció az élek között számítódik. Az asszortativitási implementáció az alábbiak szerint készíti elő az (x_i, y_i) párokat:

1. Az összes él végigmegy az aktuális gráfmódnak megfelelő szűrőn (social-path vagy battle-path).
2. Minden élhez lekérdezésre kerül az egyik végpont score-ja (`opscore` vagy `feedscore`).
3. Lekérdezésre kerül a másik végpont ugyanolyan score-ja.
4. Ha bármelyik score hiányzik, vagy ha valamelyik játékos túl rövid játékostörténettel rendelkezik (`match_count < minMatches`), az él kiesik.
5. A maradék élek score-párjai a (x_i, y_i) sor-párokkal képezik az akkumulált vektorokat.

Ez a szűrés olyan mintahalmazt eredményez, amely az erős kapcsolatok körül tömörödik: ha egy él alacsony támogatás miatt nem tekinthető az elemzésre alkalmasnak, az kiesik az asszortativitás-számításból is. Ez módszertanilag erős, mert az asszortativitás nem egy "mindent figyelembe vevő" globális mérőszám, hanem a releváns kapcsolathálózat szerkezeti tulajdonsága.

16.2.2 Gráfmód-specifikus él-súlyozás

Az `edge_support()` függvény a gráfmód alapján különbözően súlyozza az éleket:

$$w(u, v) = \begin{cases} \texttt{ally_weight} & \text{ha social-path} \\ \texttt{total_matches} & \text{ha battle-path} \end{cases} \tag{16.2}$$

A social-path módban csak a szövetségesi kapcsolatok sugallják az él meglétét. A battle-path módban az összes ismétlődés (szövetséges és ellenséges) beszámít. Ez a különbség azt fejezi ki, hogy a két mód eltérő társas szerkezeti hipotéziseket tesztel:

- **Social-path**: "A szövetségesi hálózatban a hasonló teljesítmények csoportosulnak?" - **Battle-path**: "Az összes ismétlődő kapcsolaton hasonló teljesítmények korrelálnak?"

Az asszortativitás e két módban eltérő értékeket mutat, mert a korrelálni kívánt edge-halmazok geometriai szerkezetileg különböznek.

16.2.3 Klaszteren belüli és klaszterek közötti bontások

Az implementáció nem csak globális korrelációt számol. Az eredmény az alábbi bontásokat is tartalmazza:

- **Within-cluster**: csak az ugyanabban a klaszterben lévő csomópont-párokat tartalmazó élek;
- **Between-cluster**: az eltérő klaszterekbe tartozó végpontú élek;
- **Strong-tie**: magas support értékű élek (>= küszöb);
- **Weak-tie**: alacsony support értékű élek (< küszöb).

Ezek a bontások felvilágosítanak arról, hogy az asszortativitás a klaszteren belül vagy klaszterek között koncentrálódik-e. Ha a within-cluster érték magasabb, az azt jelzi, hogy az előreklaszterezett csomópontok valóban hasonló score-ral rendelkeznek. Ha a between-cluster érték is magas, az azt mutatja, hogy az asszortativitás klaszter-független tulajdonság.

16.2.4 Missing value policy

A modul három lehetséges stratégiát támogat a hiányzó score-okkal szemben:

16.1. táblázat: Hiányzó score-kezelési politikák

Politika	Hatás
`skip`	Az él kiesik az asszortativitás-számításból
`zero`	Az él meglétét 0.0-val helyettesíti
`mean`	Az él a dataset átlag-score-ját kapja

Az alapértelmezett `skip` politika az óvatos választás: nem extrapolálunk hiányzó adatokra. Ezáltal az asszortativitás-érték az aktualista "mit tudunk ténylegesen" alapon számítódik, nem a feltételezéseken.

16.3 Asszortativitási eredmények — flexset

A `flexset` dataseten elvégzett asszortativitási elemzés összesített eredményei:

16.2. táblázat: Asszortativitás — `flexset`

Gráfmód	Metrika	Korreláció	Mintaszám
`social-path`	`opscore`	0.2346	61 938
`social-path`	`feedscore`	0.1816	61 938
`battle-path`	`opscore`	0.1461	179 492
`battle-path`	`feedscore`	−0.0108	179 492

16.4 Asszortativitási eredmények — soloq

A `soloq` dataseten ugyanolyan paraméterekkel:

16.3. táblázat: Asszortativitás — `soloq`

Gráfmód	Metrika	Korreláció	Mintaszám
`social-path`	`opscore`	0.1750	37 976
`social-path`	`feedscore`	0.1012	37 976
`battle-path`	`opscore`	0.1625	84 393
`battle-path`	`feedscore`	−0.0366	84 393

16.5 Mit mondanak ezek a számok?

[PLACEHOLDER]

Asszortativitási együtthatók összehasonlítása — opscore/feedscore, social-path/battle-path, flexset/soloq sávdiagram

16.1. ábra: Az asszortativitási eredmények vizuális összehasonlítója a két dataseten és két gráfmódban

Mindkét datasetben egyértelműen pozitív asszortativitás jelenik meg az `opscore` mentén a `social-path` módban. Ez azt jelzi, hogy a szövetségesi kapcsolatokra épülő gráfban az összekapcsolt játékosok teljesítményprofilja nem véletlenszerűen keveredik.

A `feedscore` viselkedése jóval érzékenyebb a gráfmódra. A `social-path` módban mindkét dataseten mérsékelt pozitív jel látható, de a `battle-path` módban szinte nullára esik (flexset: -0.0108, soloq: -0.0366), a soloq esetén negatívba fordul. Ez azt mutatja, hogy az ellenfélkapcsolatok bevonásával a halálokon alapuló hibamutató elveszíti a társas-kapcsolati hasonlósági szerkezetet.

A két dataset közötti különbség: a flexset `social-path opscore` asszortativitása (0.2346) magasabb, mint a soloq-é (0.1750). Ez összhangban van azzal az értelmezési kerettel, hogy a Flex Queue erősebb visszatérő csapattársi szignált hoz létre, és ezért a gráf jobban tükrözi a hasonló teljesítményű játékosok összecsoportosulását.

16.5.1 Az `opscore` robusztusabb mint a `feedscore`

Az `opscore` asszortativitása minden vizsgált módban pozitív marad. Ez az összetett, nyolc artifaktból súlyozott kompozit mutató erőssége: szemben a `feedscore` egyszerűbb halálközpontú képletével, az `opscore` több dimenzióból integrálja a teljesítményt, és ezért gráfprojekció-robusztusabb jelet ad. Nem azt állítjuk, hogy az `opscore` "objektív igazság" — a súlyok kézi tervezésűek és nem statisztikailag kalibráltak — de empirikusan megmutatja, hogy a gráf és a score-réteg között érdemi kapcsolat van, és ez a kapcsolat nem tűnik el gráfmódváltáskor.

16.5.2 Gráfmód-érzékenység mint módszertani eredmény

Ha mindkét metrika minden gráfmódban azonos értéket adna, akkor a gráfdefiníció nem hordozna önálló tartalmat. A jelen esetben az `opscore` csökkenése (flexset: 0.2346 → 0.1461, soloq: 0.1750 → 0.1625) a `battle-path` módban és a `feedscore` nullára esése azt mutatja, hogy

a kapcsolatok szemantikája valóban meghatározza a teljesítményhasonlóság mérhetőségét. Ez megerősíti, hogy a projekt két gráfmódjának megkülönböztetése empirikusan indokolt módszertani döntés.

16.6 A randomizációs infrastruktúra helye

A kódbázisban az asszortativitáshoz randomizációs szignifikanciateszt is tartozik. Ez fontos fejlesztési irány, mert a leíró korrelációt nullmodellel is érdemes összevetni. A jelen dolgozatban azonban a fő eredményeket a közvetlenül ellenőrzött leíró és bontott futtatásokra alapozom. Így a szöveg őszinte marad: a szignifikanciakeret jelen van a rendszerben, de a hangsúly még nem a teljes permutációs kísérletsorozaton van.

17. fejezet

Párhuzamos Brandes-féle betweenness centralitás

17.1 Miért szükséges a betweenness centralitás ebben a kontextusban?

A bridge-orbit vizualizáció már a Graph V2 pipeline-ból is meghatároz egyes csomópontokat és klasztereket „hidaknak” az `orbitScore` és `orbitRadius` mezők alapján. Ezek a mezők azonban vizuális heurisztikák: a kereszt-klaszter szövetségesi támasz, az összekötött klasztercsoportszám és a tagszám kombinációján alapulnak, de nem ellenőrzik, hogy a jelölt csomópontok valóban a legtöbb legrövidebb útvonal metszéspontján helyezkednek-e el. Az igazi Brandes-féle közöttiség centralitás (betweenness centrality) éppen ezt az ellenőrzést teszi lehetővé: megméri, hogy egy csomópont hány legrövidebb út mentén helyezkedik el a gráfban, és ezért nem becslésen, hanem teljes gráfbejáráson alapul.

A projektben a betweenness centralitás a `backend/pathfinder-rust/src/engine/centrality.rs` modulban van implementálva. Az implementáció több fontos döntést tartalmaz, amelyek mindegyike önálló módszertani állítás.

17.2 Az algoritmus: Brandes 2001

A Brandes-algoritmus [16] a hagyományos $O(n^3)$ komplexitású betweenness-számítást $O(nm)$-re csökkenti (irányítatlan gráfra, ahol n a csomópontok és m az élek száma). Az ötlet az, hogy a csomópontonkénti legrövidebb-út fákat nem kell külön-külön tárolni: a forráscsomópontból induló BFS/Dijkstra-futtatás eredményéből egy visszaterjesztési lépésben kiszámítható az összes közötti wt-részleges járulék.

A Brandes-algoritmus lépései forráscsomópontonként:

1. Dijkstra/BFS futtatás a forrásból: kiszámolja a $\sigma[s][v]$ értéket (legrövidebb utak száma s-től v-ig) és az $dist[s][v]$ távolságokat.
2. A feltehetős csomópontok fordított sorrendben kerülnek feldolgozásra (a stack-ból): minden w-re, amelynek előde v a legrövidebb úton, a $\delta[s][v]$ függőségi mutató a következő-

képpen frissül:

$$\delta[s][v] \mathrel{+}= \frac{\sigma[s][v]}{\sigma[s][w]} \cdot (1 + \delta[s][w]) \tag{17.1}$$

3. A forrást kivéve minden csomóponthoz hozzáadódik a $\delta[s][v]$ érték a globális betweenness-számlálóhoz.

Irányítatlan gráfra az összes forrás feldolgozása után az eredmény felezése szükséges, mert minden (s, t) és (t, s) pár kétszer volt megszámlálva. A normalizálás az $(n-1)(n-2)/2$ maximális útvonalszámra vetít. Az implementáció pontosan ezt a logikát követi a `source_dependencies()` és a `finalize_undirected_scores()` függvényekben.

17.3 A súlyozott mód és a költségmodell

A centralitás-modulban a súlyozott módban az él költsége az `edge_cost()` függvény alapján:

$$c(u, v) = \left\lceil \frac{10^6}{\max(1, w(u, v))} \right\rceil \tag{17.2}$$

ahol $w(u, v)$ az adott gráfmódnak megfelelő kapcsolaterősség (`allyWeight` social-path módban, `total_matches` battle-path módban). Az `1_000_000`-os skálázási konstans (`COST_SCALE`) integeraritmetikai pontosságot biztosít: az egészre kerekített reciprok kis élsúlyoknál nagy, erős kapcsolatoknál kis költséget ad, és az egész aritmetika determinisztikus rendezési viselkedést biztosít a prioritásos sorban.

Súlyozatlan módban az összes él ára `COST_SCALE` = 10^6, ami egyenértékű az eredeti, súlyozatlan Brandes-bejárással.

Ez a kettős mód (súlyozott/súlyozatlan) közvetlenül összemérhető a Pathfinder-modulban alkalmazott `traversal_cost()` képlettel, ahol a skálázási konstans 1000. A centralitás-modul pontosabb integer-aritmetikát használ a nagyobb összegek miatt.

17.4 Párhuzamos végrehajtás Rayonnal

A Rust `rayon` könyvtára automatikus munkalopásos (work-stealing) párhuzamosságot tesz lehetővé. A centralitás-modul ezt a következő stratégiával alkalmazza:

1. A forráscsomópontok halmazát k egyenlő méretű csomóponthalmazra osztja, ahol $k = \min(n, 4 \cdot \text{szálak száma})$.
2. Minden halmazt egy párhuzamos rayon-feladatban dolgoz fel: a lokális δ vektor független a többi résztől.
3. Az összegzés után a részeredmény-vektorok deterministikusan összeadódnak.
4. A párhuzamos eredményt a soros eredménnyel összevethető maximális abszolút eltérés (`max_abs_delta`) jellemzi.

A determinizmus biztosítása kulcsfontosságú: a forráscsomópontok rendezése és a chunk-határok rögzítése garantálja, hogy azonos input esetén mindig ugyanaz a kimenet. A tesztcsomag (`parallel_matches`) ezt explicit módon ellenőrzi.

[PLACEHOLDER]

Brandes centralitás párhuzamos folyamatábrája — forráscsomópont-szintű párhuzamosság Rayonnal és 1/strength élköltséggel

17.1. ábra: A Brandes-algoritmus párhuzamos végrehajtásának sémája Rayon work-stealing párhuzamossággal

17.1. táblázat: A betweenness centralitás-modul főbb konfigurációs dimenzióinak összefoglalója

Dimenzió	**Lehetséges értékek**	**Hatás**
Gráfmód	social-path, battle-path	Melyik élhalmazon fut
Súlyozás	igen / nem	`edge_cost()` visszaadott értéke
Párhuzamos	igen / nem	rayon vs. soros Brandes
Soros baseline	igen / nem	Delta-ellenőrzés a kimenetben
Min. élsúly	≥ 1	Gyenge élek kizárása

17.5 A projection felépítése

A `build_projection()` függvény a `GraphState` pair_relations térképéből iterál. Az élek sorrend szerint rendezve kerülnek feldolgozásra, így a projekció determinisztikus. Kizárt élek:

- nulla `support` értékűek az adott gráfmódban;
- minimum élsúly alatt lévők (`skipped_low_support_edges`);
- rossz formátumú kulcsok (`skipped_invalid_edges`).

A csomópontok α-betű sorrendben indexelődnek (`BTreeSet`), és a szomszédsági lista minden csúcsnál szintén rendezett. Ez garantálja a determinizmust akkor is, ha a `HashMap`-ben tárolt pair_relations bejárási sorrendje változó.

17.6 Normalizálás és eredményinterpretálás

A normalizált betweenness érték egy $[0, 1]$ intervallumba esik, ahol az 1.0 értékű csomópont minden lehetséges $s \to t$ legrövidebb úton rajta van. A maximálisan normalizált érték ritka: csak csillag-topológia centerének normalizált betweenness értéke pontosan 1.0.

Az egységtesztek ezt a három alapesetet explicit ellenőrzik:

1. **Lánc-topológia** (path graph): a belső csomópontok magasabb betweenness-t kapnak, mint a végpontok.
2. **Csillag-topológia** (star graph): a center normalizált értéke pontosan 1.0.
3. **Dumbbell-topológia**: a hidat képező csomópont rangsorolódik legelőre.
4. **Súlyozási teszt**: erős parellelút esetén a súlyozatlan mód a közvetlen élt preferálja, a súlyozott mód az erősebb közvetett utat.
5. **Soros-párhuzamos egyezés**: determinisztikus azonosság bizonyítva (`max_abs_delta == 0`).

17.7 Centralitás a bridge-orbit hipotézis tesztelésére

A bridge-orbit layout (§14.10) az `orbitScore` és `orbitRadius` mezőkkel azonosítja a kiemelten összekötő klasztereket. Ezeket a heurisztikus pontszámokat egyelőre nem vetítettük össze Brandes-betweenness értékekkel, de a modul elkészültével ez a validálás elvégezhető.

A várható hipotézis: a legmagasabb `crossAllySupport` és `connectedAllyClusterCount` értékű klaszterek átlagos Brandes-betweenness értéke szignifikánsan magasabb az alacsony orbit-score-úakénál. Ha ez teljesül, az megerősíti, hogy az orbit-heurisztika jó előszűrője a valódi topológiai hidat képező csomópontoknak. Ha nem teljesül, az a heurisztika korlátaira világít rá és pontosítja a vizualizáció értelmezési keretét.

Ez a jövőbeli vizsgálat pontosan az a fajta módszertanilag indokolt bővítés, amelyről a `research-workflow` szemlélet szól: a meglévő adatból egy explicit, megcáfolható hipotézis tesztelhető, és az eredmény akár megerősíti, akár megcáfolja a korábbi vizuális heurisztikát.

17.8 Kísérletfuttató infrastruktúra és nullmodellek

A Rust motor nem csak egyedi analitikai kéréseket tud kiszolgálni, hanem strukturált kísérleteket is tud futtatni a `backend/pathfinder-rust/src/engine/experiments.rs` modulon keresztül.

17.8.1 Signed-balance sweep mint módszertani diagnosztika

A `SignedBalanceSweepRequest` egy kötegelt futási kérés, amely egyszerre több minimum-élsúly és döntetlen-kezelési politika kombinációt futtat le, és az összes eredményt egységes `SignedBalanceSw`

válaszban adja vissza. A sweep sémaverziója (`signed-balance-sweep-v1`) az eredmények jövőbeli összehasonlíthatóságát biztosítja.

Minden egyes sweep-futás az alapfuttatással azonos Rust kódon megy keresztül, így az összes kombinált eredmény egységes implementációs alapra támaszkodik. A sweep főbb paraméterei:

- `min_support_values`: vizsgált minimális élsúlyok listája;
- `tie_policies`: vizsgált döntetlen-kezelési politikák listája (`exclude`, `ally`, `enemy`);
- `graph_modes`: `social-path` és/vagy `battle-path`.

Ez lehetővé teszi, hogy egy kérésben teljes érzékenységi mátrix készüljön, például: minden min_support × tie_policy kombinációra megkapjuk az analizált triádok számát és az egyensúlyi arányt. A dolgozatban ezt már nem fő eredményként használom, hanem annak ellenőrzésére, hogy mennyire erősen függ a formális signed-balance kimenet a projekciós döntésektől. A sweep így a módszertani határ dokumentálására szolgál.

17.8.2 Asszortativitási szignifikanciateszt

Az `AssortativitySignificanceRequest` permutációs nullmodellt futtat: véletlenszerűen kever össze játékos–score párokat az éleken, és az így kapott nulldisztribúcióval veti össze a megfigyelt korrelációt. A determinisztikus RNG-mag (`SystemTime::UNIX_EPOCH` alapú, de rögzíthető) garantálja a reprodukálhatóságot.

A nullmodell-futtatás főbb jellemzői:

- permutáció-szám konfigurálható (`num_permutations`);
- a figyelt korreláció (r_{obs}) összehasonlítható a null-eloszlás átlagával és szórásával;
- a p-érték az $r_{\text{null}} \geq r_{\text{obs}}$ arányból becsülhető;
- a sémaverziójú kimenet (`assortativity-significance-v1`) az eredményeket önmagukon belül értelmezhetővé teszi.

A jelen dolgozatban a szignifikanciafuttatások megléte a rendszerben dokumentált, de az összes kombinációra kiterjedő, végleges nullmodell-kísérletsorozat a jövőbeli fejlesztések közé tartozik. A hangsúly egyelőre az asszortativitás leíró korrelációin és a dataset-összehasonlításon van, amelyek önmagukban is érvényes empirikus eredmények.

17.8.3 A kísérletfuttató infrastruktúra módszertani értéke

A sweep és a szignifikanciateszt léte nem csak technikai kényelmi funkció. Módszertanilag azt jelenti, hogy a dolgozatban közölt eredmények reprodukálhatók és bővíthetők: ugyanazon infrastruktúrán egy jövőbeli vizsgálat egyre finomabb paraméterekkel futhat le, anélkül hogy az alapimplementációt módosítani kellene. Ez összhangban van a reprodukálható kutatás alapelvével [20]: a szoftver, nem csak a szöveg hordozza a módszertani döntéseket.

18. fejezet

Kísérleti futtatások, validáció és eredmények

18.1 Verifikációs lépések

A dolgozat elkészítésekor a rendszer aktuális állapotát nemcsak a dokumentáció, hanem közvetlen futtatás alapján is ellenőriztem. A főbb helyi ellenőrzések:

- a Rust futtatókörnyezet sikeresen lefuttatta az `assortativity` parancsot;
- a Rust futtatókörnyezetben a `signed-balance` parancs kísérleti modulként továbbra is futtatható volt;
- a fő adatbázis és dataset-nyilvántartás elérhető és olvasható volt;
- a dolgozati frontend útvonalak közül az asszortativitási és központisági oldalak maradtak a fő analitikai folyamat részei.

Ez azért lényeges, mert a dolgozat ezen a ponton már nem csak tervekről, hanem ténylegesen végrehajtható modulokról számol be.

A verifikációs lépések tudatosan nem teljes funkcionalitástesztek voltak, hanem tézisszintű ellenőrzések. A cél itt az volt, hogy minden olyan rétegről legyen közvetlen bizonyíték, amelyre a dolgozat állításokat épít: a dataset-regiszter olvashatósága, a Rust analitikai parancsok futtathatósága, valamint a frontendoldalak fizikai jelenléte. Ez összhangban van a reprodukálható kutatási gyakorlat logikájával is: a szöveg és az alátámasztó futások között legyen világos kapcsolat [20].

18.2 Az aktív állapot összefoglalása

A 2026. április 28-án ellenőrzött aktív állapotban a rendszer az `soloq` datasetre épül, a `flexset` dataset a fő kutatási korpusz. Az adatkészletek és a gráfállapot összefoglalása:

18.1. táblázat: A jelenlegi rendszerállapot fő mennyiségi mutatói — `flexset` és `soloq`

Mutató	flexset	soloq
Meccs-JSON állományok	4981	2001
Graph V2 meccsszám	4980	2000
Nyers játékosok (`players.db`)	15 147	2822
Finomított játékosok (`playersrefined.db`)	15 421	6768
Klaszterek (`playersrefined.db`)	599	140
Klasztertagok	5741	1538
Látható Graph V2 csúcsok	5741	1538
Látható Graph V2 élek	18 548	4111
Átlagos élsúly (Graph V2)	3.405	2.364
Graph V2 klaszterek	1531	870
Átlagos klasztermére t	3.750	1.768

Ezeket a számokat lépcsőzetesen kell olvasni. A finomított játékosszám a teljes score-réteg kiterjedése. A Graph V2 látható csúcsok száma már csak az érvényes klasztertagsággal rendelkező játékosokat tartalmazza. Minden réteg a saját, a megelőzőből leszűkített univerzumán számol, ezért a gráf- és score-alapú eredményeknél mindig ki kell mondani, hogy pontosan melyik populáción készültek.

A két dataset összehasonlítása megerősíti, hogy a `flexset` és a `soloq` lényegesen eltérő karakterű korpuszok, és ez az eltérés minden analitikai szinten mérhető.

18.3 A korpusz összetétele mint értelmezési háttér

Az eredmények értékeléséhez fontos hozzárakni, hogy a vizsgált korpusz nem homogén. A queue-eloszlás alapján a ranked módok dominálnak, de a rendszerben ARAM és normál meccsek is jelen vannak. Ez két következménnyel jár:

- a gráf egy része erősen kompetitív, ismétlődő társas térből származik;
- egy másik része lazább, zajosabb kapcsolatokkal terhelt.

Éppen ezért a küszöbölés, az erős és gyenge kapcsolatok bontása, valamint a `social-path` és `battle-path` megkülönböztetése nem puszta technikai részlet, hanem az adatkorpuszból fakadó szükségszerűség.

Ez a felismerés a teljes eredményfejezetet meghatározza. Ha a korpusz heterogén, akkor a gráfmodellt sem lehet úgy kezelni, mintha egyetlen társas mechanizmust mérne. A `social-path` és `battle-path` különválasztása nem csupán algoritmus-beállítás, hanem módszertani védekezés a túlzó interpretáció ellen.

18.4 A két klasztercsalád gyakorlati különbsége

A perzisztált klasztereknél az is ellenőrizhető volt, hogy a Python és a Rust család eltérő karakterű. A `python_population` klaszterek kicsit kisebb maximális méretűek és inkább populációs

közösségképet adnak, míg a `rust_pathfinding` klaszterek futásidejű szempontokhoz igazodnak, és jóval nagyobb maximumklasztert is tartalmaznak.

Ez összhangban van a rendszertervvel. A két klasztercsalád nem egymás hibás duplikátuma, hanem eltérő feladatokra optimalizált nézet.

18.5 Analitikai eredmények

Az asszortativitási eredmények részletes táblái és módszertani kifejtése a dedikált fejezetben kapott helyet. A strukturális egyensúly külön fejezetben csak módszertani határesetként szerepel: a formális implementáció megvan, de az enemy-élek szemantikája miatt nem használom fő empirikus eredményként. A két dataset összesített, szintetikus összehasonlítása a rákövetkező összehasonlító fejezetben található.

18.6 A frontend és a rendszerkoherencia értékelése

A projekt egyik fontos eredménye nem kizárólag a számszerű analitikában áll, hanem abban is, hogy a rendszer ma már koherens egészként működik. A jelenlegi állapotban:

- az adatgyűjtés és dataset-kezelés külön rétegben történik;
- a pontszámítás explicit, ellenőrizhető képletekre épül;
- a gráfépítés és klaszterezés nem ad hoc frontend-logika;
- a futásidejű Rust réteg a gráfanalitikát külön kezeli;
- a frontend több specializált felületen, eltérő célok mentén mutatja meg az eredményeket.

Ez azért fontos, mert a dolgozat nem egyszerűen egy algoritmus vagy egy látványos oldal bemutatása, hanem annak dokumentálása, hogy ezek a részek együtt már egy kutatásképes rendszerként viselkednek.

A koherencia egyik kézzelfogható jele, hogy a frontend route-struktúrája és a backend végpontstruktúrája egymásra illeszkedik. A `frontend/src/App.tsx` külön oldalakat tart fenn a collectorhoz, a graph sphere-höz, az assortativity-hez, a betweenness-központisághoz és a Pathfinder Labhoz, vagyis a felület nem egyetlen „mindent bele” dashboard. A Signed Balance route a dolgozati termékből kikerült, mert a fő narratíva már nem erre az értelmezésre épül.

18.7 A korábbi ország-elemzési ág helye a jelenlegi rendszerben

A korábbi `assign_countries.py` és `conclude.py` ág továbbra is a projekt része. Ez megőrzi azt a lehetőséget, hogy a klaszterekből származó országcímkézést összekapcsoljuk aggregált teljesítményelemzéssel. Ugyanakkor a jelenlegi dolgozat fő tudományos súlypontja már nem ez, hanem a gráf mint kutatási objektum.

Védhetőnek azt tartom, ha ezt az ország-elemzési vonalat *kiegészítő, exploratív* modulnak nevezzük:

- implementált,
- hasznos lehet,
- de bizonytalansága nagyobb,
- ezért a dolgozat központi állításait nem erre építem.

[PLACEHOLDER]

Validációs összefoglaló — a verifikált modulok, futtatott parancsok és mért eredmények egy ábrán

18.1. ábra: A kísérleti futtatások és validáció összefoglaló sémája

19. fejezet

Rendszerszintű értékelés és megvitatás

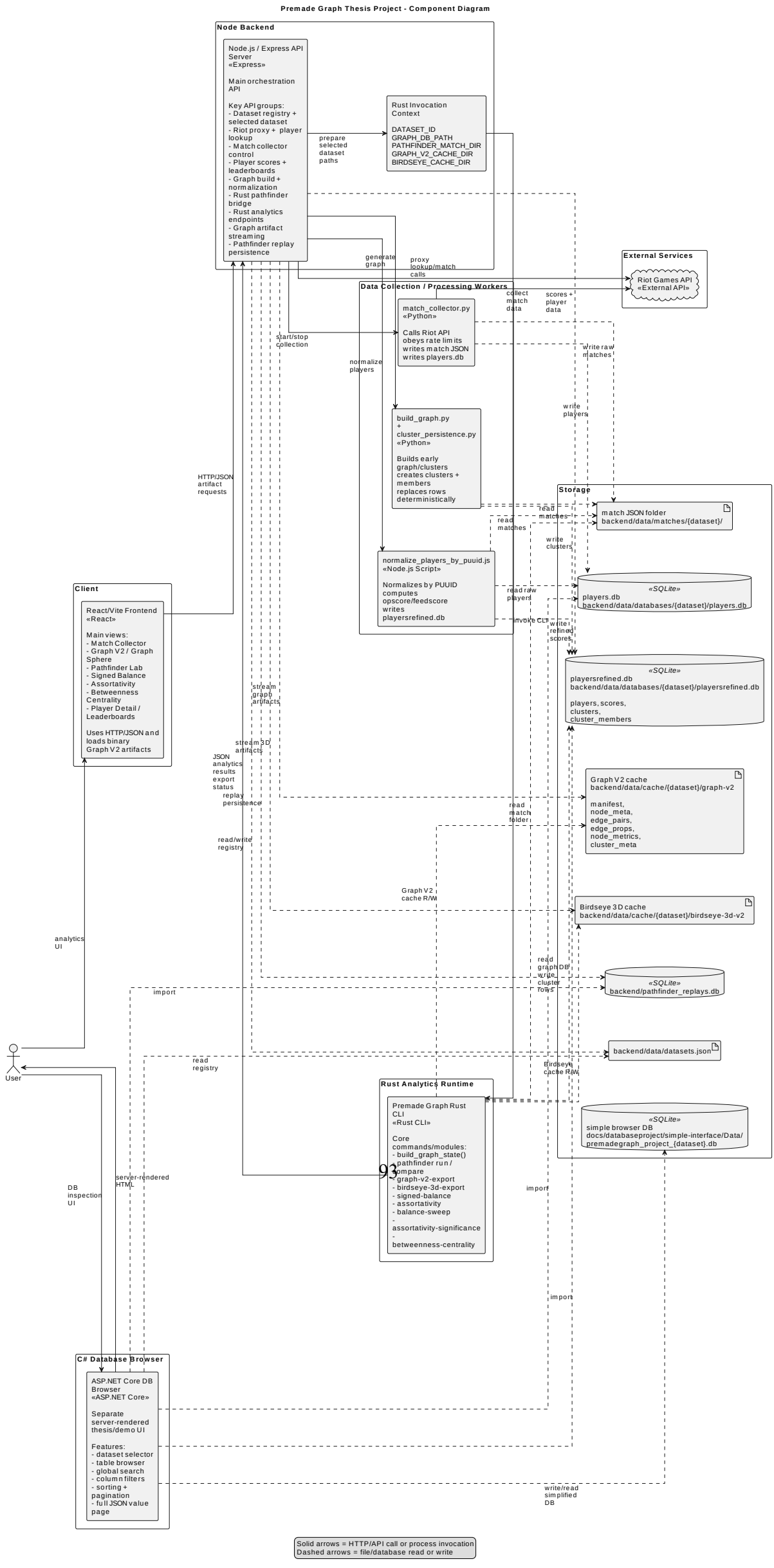
19.1 Mi bizonyult a legerősebb rendszertervezési döntésnek?

Az egész projektet visszanézve a legerősebb döntésnek azt tartom, hogy a különböző feladatokat nem próbáltam egyetlen univerzális technológiába kényszeríteni. A rendszer nem eszközpreferencia, hanem minőségi attribútumok mentén fejlődött. A fontos attribútumok a következők voltak:

- reprodukálhatóság,
- magyarázhatóság,
- perzisztencia,
- futási teljesítmény,
- demo- és védhetőségi koherencia.

Ezek közül ritkán lehet mindent ugyanazzal az eszközzel a legjobban kiszolgálni. A Python gyors iterációt és kényelmes adatfeldolgozást ad, a Rust teljesítményt és pontos gráfkontrollt, a Node/Express stabil integrációs felületet, a React pedig erős bemutathatóságot. A többnyelvűség tehát itt nem öncélú komplexitás, hanem funkcionális specializáció.

19.2 A rendszer erősségei



19.1. ábra: A teljes rendszer komponensdiagramja — az adatút a Riot API-tól a React frontendig

A jelenlegi állapot alapján a projekt legfontosabb erősségei:

- a teljes adatút végigkövethető a Riot API-tól a frontendig;
- a játékosprofilok explicit, ellenőrizhető képleteken és mezőkön alapulnak;
- a gráfépítés és klaszterezés nem csak vizuális export, hanem perzisztens objektum;
- a Rust réteg valódi analitikai és futtatási értéket ad, nem csak "gyorsabb átírás";
- az asszortativitás, a core-periphery gráfolvasat és a Brandes-központiság valódi kutatási kérdésekké emelik a rendszert.

19.3 Fenyegetések a validitásra

Egy védhető szakdolgozatban legalább ennyire fontos kimondani a korlátokat is. A jelen projekt legfontosabb validitási kockázatai:

- a dataset nem reprezentálja a teljes játékospopulációt;
- a gyűjtés seed- és queue-függő;
- a `feedscore` és `opscore` interpretálható, de nem "igazságmérő";
- a signed-balance eredménye nem kap fő empirikus szerepet, mert az enemy-él negatív társas kötésként való értelmezése nem elég erős;
- a country-ág sokkal bizonytalanabb, mint a közvetlen gráf- és meccsanalitika.

Ezek a korlátok azonban nem rombolják le a dolgozatot. Inkább kijelölik azt a határt, amelyen belül az állítások védhetők. Ezért is volt fontos, hogy a megvalósult és a jövőbeli elemeket szigorúan külön kezeljem.

Érdemes a validitási kockázatokat rétegenként is szétbontani:

- **Mintavételi validitás**: a seedválasztás, a premade-probe és a queue-orientáció együtt nem véletlen mintát adnak a teljes játékospopulációból.
- **Mérési validitás**: a `feedscore` és `opscore` értelmezhető, de konstrukciós döntésekre épülő metrikák; különösen az `opscore` tartalmaz kézi szerepkör-súlyozást.
- **Projekciós validitás**: a gráfmód, az élküszöb és az ally/enemy kapcsolatértelmezés érdemben alakítja a végső analitikai mintát; a signed-balance emiatt csak módszertani határesetként használható.
- **Klasztervaliditás**: a közösségdetektálás eredménye függ a választott gráftól, a küszöböléstől és a modularitás felbontási korlátjától [11].
- **Időbeli validitás**: a korpusz fokozatosan épülő archívum, ezért a teljes háló nem egyetlen szinkron időpillanat felvétele.

Ezek a fenyegetések azonban nem cáfolják a projektet. Inkább megmutatják, hogy meddig lehet erős állításokat tenni, és hol kell tudatosan visszafogni az interpretációt. A dolgozat védhetőségét végső soron éppen ez a rétegzett őszinteség erősíti.

19.4 Miért értékes a projekt szakdolgozatként?

Szakdolgozati szempontból a projekt értéke nem csak abban áll, hogy "sok mindent tud". A valódi érték az, hogy több informatikai terület kapcsolódik össze benne:

- adatgyűjtés és API-használat,
- adatbázistervezés,
- hálózatkutatás,
- algoritmusok és teljesítmény,
- frontend és vizuális kommunikáció.

Ez a kombináció azért erős, mert minden réteg hozzájárul a végső állításhoz. Ha bármelyik hiányozna, a dolgozat jóval gyengébb lenne: pusztán frontendként látványos lenne, pusztán algoritmusként kontextustalan, pusztán adatelemzésként pedig bemutatási réteg nélkül maradna.

Ehhez hozzáadódik a reprodukálhatósági érték is. A rendszer fő eredményei nem kézzel összeollózott prezentációs pillanatképekből állnak, hanem olyan pipeline-okból, amelyek a nyers meccs-JSON-tól a normalizált játékosprofilon és a perzisztált klasztereken át a Rust-analitikáig követhetők. Ez különösen erős szakdolgozati tulajdonság: a bemutatott állításokhoz tartozik egy végrehajtható technikai út is.

19.5 A megvalósult és a jövőbeli elemek szétválasztása

A dolgozat egyik tudatos módszertani döntése az volt, hogy a jelenleg implementált és a még csak tervezett elemeket nem keverem össze. A jelenlegi, ténylegesen megvalósult mag a következő:

- Riot API-ra épülő adatgyűjtés;
- `puuid`-alapú normalizálás;
- `feedscore`/`opscore` számítás;
- Python gráfépítés és közösségdetektálás;
- SQLite klaszterperzisztencia;
- Rust útvonalkeresés és 3D globális export;
- asszortativitás és betweenness-központiság.

A jelenleg megvalósult mag tartalmazza a databaseproject és C# böngésző alprojektet is, amely formálisan nem a főrendszer része, de valódi, futtatható eszközként él a repositoryban.

A még nem lezárt, de az architektúra alapján célzottan megvalósítható irányok:

- a Brandes-féle betweenness centralitás teljesebb benchmarkolása és Graph V2 artifact-integrációja;
- adatvezérelt `opscore`-kalibráció meccsnyerési aránnyal;
- erősebb permutációs nullmodellek az asszortativitáshoz.

Visszavont irányok (a korábbi tervdokumentumokban szerepeltek, jelenleg nem aktívak): Genetic NeuroSim v2, temporal consistency/stabilitás és community cohesion vs. performance vizsgálat eltávolítva a jelenlegi scope-ból. A Brandes-központiság ezzel szemben már implementált modul, amelynek további feladata nem az alapalgoritmus megírása, hanem a benchmarkolás és a Graph V2 artifact-rétegbe illesztés.

Ez a szétválasztás nem gyengíti, hanem erősíti a dolgozatot. A bírálat szempontjából sokkal védhetőbb egy olyan szöveg, amely pontosan kijelöli a rendszer jelenlegi határát, mint egy olyan, amely minden jó ötletet kész eredménynek próbál feltüntetni.

20. fejezet

A két adatkészlet empirikus összehasonlítása: `flexset` versus `soloq`

20.1 Miért szükséges a két adatkészlet összehasonlítása?

A projekt egyik legfontosabb kutatási-módszertani döntése az volt, hogy nem egyetlen adatkészlettel dolgozik, hanem egymással összehasonlítható, eltérő queue-szemantikájú korpuszokkal. Ez nem pusztán mennyiségi bővítés, hanem kvalitatív különbségtétel: a `flexset` (EUNE Master+ Flex Queue) és a `soloq` (EUNE Master+ Solo/Duo Queue) nemcsak méretükben, hanem a bennük keletkező kapcsolatok jellegében is eltérnek.

A kutatási kérdés így fogalmazható meg: ha ugyanazon analitikai pipeline-t lefuttatjuk mindkét dataseten, mikor kapunk eltérő eredményt, és ez az eltérés értelmezhető-e a két queue eltérő mechanikájából?

A **research-workflow** szemlélet szerint ez az összehasonlítás nem utólagos illusztráció, hanem a módszertan integráns része. Az egyetlen datasetre épülő eredmény interpretálása mindig kétséges: az eredmény lehet a módszer hatása, lehet a dataset sajátossága, vagy lehet valódi gráfstruktúra. Két, eltérő generáló mechanizmussal bíró dataseten kapott konvergens vagy divergens eredmény sokkal erősebb támpontot ad.

20.2 A két dataset konstruktív különbségei

20.2.1 Queue-szemantika és kapcsolatgeneráló mechanizmus

A Flex Queue (440-es queue ID) csapatszedést tesz lehetővé: játékosok 2–5 főig premade csapatot alkothatnak, és egységként lépnek be a meccsbe. Ez azt jelenti, hogy a szövetségesi pozícióban repeatedly megjelenő játékospárok egy része *szándékolt ismétlődés*: a premade csapat azonos tagjai meccsről meccsre együtt indulnak. Az ally/enemy arány (73.17% ally a flexseten) részben ennek a mechanizmusnak a következménye.

A Solo/Duo Queue (420-as queue ID) ezzel szemben legfeljebb két fős premade-et engedélyez. A matchmaking itt jóval véletlenszerűbb: egy adott játékos öttagú csapatának többi tagja döntően nem szándékolt, hanem algoritmikus összerendelés. Az ally/enemy arány (65.24% ally a

soloq-on) alacsonyabb, ami összhangban van azzal, hogy a véletlenszerűbb meccsek kevesebb domináns szövetségesi kapcsolatot hoznak létre.

20.2.2 Adatskála és meccssűrűség

20.1. táblázat: A két dataset alapvető méretei

Mutató	flexset	soloq
Meccs-JSON állományok	4981	2001
Nyers játékosok	15 147	2822
Finomított játékosok	15 421	6768
Klaszterek (DB)	599	140
Klasztertagok (DB)	5741	1538
Graph V2 látható csúcsok	5741	1538
Graph V2 látható élek	18 548	4111
Átlagos élsúly	3.405	2.364
Graph V2 ally/enemy arány	73.17% / 26.83%	65.24% / 34.76%
Graph V2 klaszterek	1531	870
Átlagos klaszterméret	3.750	1.768
Legnagyobb klaszterméret	20	20

Az átlagos élsúly különbsége (3.405 vs. 2.364) különösen figyelemreméltó. Ez azt jelenti, hogy a flexset-en a látható kapcsolatok átlagosan sűrűbb, ismétlődőbb előfordulásból épülnek fel. Ahol a soloq-on egy él két meccs ismétlődéséből áll össze, ott a flexset-en tipikusan több mint három meccs van mögötte. Ez közvetlen mérőszáma annak, hogy a Flex Queue-ban az azonos játékospárok egymással sűrűbben, tartósabban találkoznak.

20.3 A két dataset vizuális klaszterstruktúrájának különbsége

Az átlagos klaszterméret különbsége (3.750 vs. 1.768) szintén tartalmas. A flexset-en a csoportok átlagosan majdnem négytagúak, míg a soloq-on szinte mindenütt kis, kétfős vagy egyedülálló klaszterek dominálnak. Az 1.768-as átlag azt jelenti, hogy a soloq-ban a Graph V2 által látható csomópontok közel fele egyedül álló csúcs (azaz nincs mellette kellő súlyú szövetségesi partner).

Ez a különbség nem adatgyűjtési hiba, hanem a queue-mechanika közvetlen következménye. A premade Flex Queue-ban természetesen keletkeznek 3–5 fős, ismétlődő klasztermagok, amelyek a Graph V2-ben összefüggő csoportként jelennek meg. A véletlenszerűbb soloq-ban ez a csoportosulás nem jön létre organikusan.

A bridgecandidateClusters és outerOrbitClusters értékei is különböznek: a flexseten 815 bridgejelölt-klaszter van 1531 összesből (53.2%), a soloq-on 605 jelölt 870-ből (69.5%). Ez meglepő arány — a soloq klaszterei relatíve több hidat képeznek. Ennek magyarázata valószínűleg az, hogy a soloq-on a kis klaszterek jobban szétszórtabb, kevésbé összefüggő grafikonban helyezkednek el, így relatív módon több közöttük a vizuálisan kitüntetett pozícióban lévő.

20.4 A metrikafedettség összehasonlítása

Mindkét dataseten a Graph V2 artifact teljesen fedett opscore és feedscore értékekkel:

- **flexset**: opscore-fedettség 5741/5741, feedscore-fedettség 5741/5741
- **soloq**: opscore-fedettség 1538/1538, feedscore-fedettség 1538/1538

A teljes fedettség azt jelenti, hogy a Graph V2-ben megjelenő minden látható csomóponthoz rendelkezésre áll érvényes opscore és feedscore. Ez az asszortativitási és a gráf-alapú elemzések szempontjából kritikus: nincs olyan csomópont, amely hiányzó score miatt kiesne az elemzésből.

Az opscore átlagok is közel esnek egymáshoz: flexset 6.1015, soloq 6.1701. Ez arra utal, hogy a percentilis-alapú normalizáció adatsetfüggetlenül hasonló eloszlást állít elő, ami megerősíti, hogy a kétdataset-összehasonlítás az összehasonlíthatóság feltételei között zajlik.

20.5 A teljes összehasonlítás tanulságai

20.2. táblázat: A két dataset összehasonlítás összefoglalója

Dimenzió	**flexset**	**soloq**
Queue-mechanika	Flex, premade-fókusz	Solo/Duo, egyénközpontú
Átlagos élsúly	3.405 (sűrűbb)	2.364 (ritkább)
Ally arány	73.17%	65.24%
Átlagos klaszterméret	3.750	1.768
Triádok/csomópont	4.95	1.03
Core-periphery olvasat	Hídgazdag asszociatív mag és sok perifériku ally-sziget	Kisebb, véletlenszerűbb lokális csoportok
Social-path opscore assz.	0.2346	0.1750
Battle-path opscore assz.	0.1461	0.1625
Feedscore battle-path assz.	-0.0108	-0.0366
Kutatási szerep	Asszociatív gráf és teljesítménykapcsolat	Teljesítmény-alapvonal, kontroll

Az összehasonlítás általános tanulsága: a `flexset` az asszociatív játékosgráf és a visszatérő szövetségesi kapcsolatok természetesebb terepe, a `soloq` pedig a teljesítmény-alapvonal és a kontroll-dataset szerepét tölti be. Ahol a két dataset hasonló eredményt ad (pl. mindkettőn pozitív social-path opscore asszortativitás), ott az eredmény általánosabb jellegű. Ahol eltérnek (pl. klaszterméret, élsűrűség, bridge-orbit szerkezet), ott az eltérés a queue-mechanikából fakad, és önmagában is tudományosan értelmezhető.

Figyelemreméltó részlet a táblázatban: a `battle-path opscore` értéke a soloq-on *magasabb* (0.1625), mint a flexset-en (0.1461). Ennek magyarázata a soloq szűk MMR-sávján alapuló matchmaking: az ismétlődően egymással találkozó játékosok opscore-ban közel esnek egymáshoz, de ez a struktúra gyengébb és esetlegesebb, mint a flexset szándékolt premade-kapcsolatai. A `social-path` módban a flexset erősebbebb (0.2346 vs. 0.1750), ami a premade-szövetségesi mechanizmust tükrözi.

[PLACEHOLDER]

Flexset vs. SoloQ összehasonlító sávdiagram — átlagos élsúly, klaszterméret és social-path opscore asszortativitás egymás mellett

20.1. ábra: A két adatkészlet főbb mérőszámainak összehasonlítása

21. fejezet

Összegzés

21.1 A projekt összefoglalása

A dolgozatban bemutatott rendszer mára egy egyszerű premade gráfötletből olyan összetett, de világosan tagolt projektté fejlődött, amely képes:

- valódi League of Legends meccsadatokat begyűjteni és adatkészletek szerint kezelni;
- a játékosokat `puuid` alapon normalizálni és szerepkörérzékeny mutatókkal leírni;
- az ismétlődő együtt- és egymás ellen játszásokból súlyozott asszociatív gráfokat építeni;
- közösségszerkezetet tárolni és újrafelhasználni;
- a Rust futtatókörnyezetben útvonalakat, globális nézeteket és analitikákat számolni;
- ezeket a kimeneteket több különálló, de koherens frontend oldalon bemutatni.

A jelenlegi rendszerállapot alapján a dolgozat legfontosabb üzenete számomra az, hogy ez a projekt már nem csak ötletszinten érdekes. A `flexset` core-periphery szerkezete, a Flex/SoloQ összehasonlítás, az asszortativitás és a Rust-alapú központisági réteg azt mutatják, hogy a hálózatban ténylegesen megfogható, mérhető szerkezetek jelennek meg. Közben a teljes architektúra is azt támasztja alá, hogy a rendszer nem véletlenszerűen összehordott komponensekből áll, hanem egyre inkább tudatosan szétválasztott felelősségi körök szerint fejlődött.

Az is fontos tanulság, hogy a projekt legerősebb része nem egyetlen „nagy trükk”. Nem az a lényege, hogy van benne 3D nézet, vagy hogy Rustban fut néhány algoritmus, vagy hogy vannak klaszterek. A valódi érték abban van, hogy az adatgyűjtés, a pontszámítás, a gráfépítés, a perzisztencia, az útvonalkeresés és a kutatási analitika végre ugyanannak a rendszernek a része.

21.2 A dolgozat védhető fő következtetései

Az aktuális implementáció és az elvégzett empirikus elemzések alapján a következő állítások tekinthetők védhetőnek:

- A League of Legends ismétlődő mérkőzésadataiból értelmes, több rétegben elemezhető játékosháló építhető; ez a flexset dataseten 4980 meccs, 5741 látható csomópont és 18548 él formájában valósult meg.

- A szerepkörérzékeny, percentilis-alapú `opscore`/`feedscore` pipeline kellően értelmezhető ahhoz, hogy gráfanalitikai bemenetként használjuk; a teljes metrikafedettség (5741/5741 a flexseten, 1538/1538 a soloq-on) ezt megerősíti.
- A Python és Rust közötti munkamegosztás a jelenlegi projektméretben indokolt és szakmailag jól védhető; a kiegészítő C# / ASP.NET Core böngésző az adatbázis-engineering alprojekt részeként önállóan is védhető eszközként jelenik meg.
- A `flexset` dataseten az `opscore` mindkét gráfmódban pozitív asszortativitást mutat (`social-path`: 0.2346, `battle-path`: 0.1461); ez a `soloq` dataseten is teljesül, de gyengébb jel mellett (`social-path`: 0.1750, `battle-path`: 0.1625).
- A `flexset` Graph V2 artifact core-periphery asszociatív szerkezetet mutat: hídgazdag központi ally-csoportok és sok kicsi, gyengén kapcsolt perifériku csoport jelenik meg.
- A kétdataset-összehasonlítás legszilárdabb eredménye: a Flex Queue lényegesen sűrűbb és ismétlődőbb asszociatív gráfot generál, mint a Solo/Duo Queue; ez az átlagos élsúlyban, klasztermeretben és az asszortativitási különbségekben is megjelenik.
- A `feedscore` `battle-path` módban közel nulla vagy negatív asszortativitást mutat mindkét dataseten, ami megerősíti, hogy a gráfprojekció megválasztása módszertanilag lényeges, nem puszta technikai beállítás.

22. fejezet

Az adatbázis-engineering kiegészítő alprojekt: databaseproject és C# böngésző

22.1 A kiegészítő alprojekt szerepe és motivációja

A Premade Graph fő rendszere SQLite-alapú perzisztenciával dolgozik, és a gráfanalitikai pipelinehoz és a frontendhez optimalizált sémát használ. A `docs/databaseproject` könyvtárban egy ezzel párhuzamos, de önálló alprojekt él: egy formálisan megtervezett, PostgreSQL-re épülő relációs adatbázismodell és egy ASP.NET Core / C# alapú adatbázis-böngésző alkalmazás. Ez az alprojekt kettős értékkel bír: egyrészt erősíti az adatbázis-tervezés akadémiai oldalát, másrészt valódi, futtatható eszközt biztosít az elemzési eredmények böngészéséhez.

Az `architecture-decision` szemlélet alapján fontos tisztázni, miért kettő rendszer létezik egymás mellett. Az SQLite-alapú fő rendszer a gráfépítés, az analitika és a demo-pipeline kiszolgálásához optimalizált: egyszerű telepítés, nincs szerverfüggőség, gyors az iteráció. A PostgreSQL-alapú adatbázismodell ezzel szemben a relációs adatbázistervezés akadémiai igényeivel igazodik: tárolt eljárások, triggerek, rekurzív lekérdezések, JSON-mezők, hivatkozási integritás. A kettő nem versenyez egymással, hanem különböző célt szolgál ugyanabban a domainben.

22.2 A formális relációs séma

A `docs/databaseproject/schema.sql` egy teljes PostgreSQL sémát definiál. A séma tíz főtáblája a következő:

22.1. táblázat: A databaseproject PostgreSQL sémájának táblái

Tábla	Szerepkör a sémában
`players`	Játékosazonosság, összesített pontszámok, szerepkör
`matches`	Meccsszintű adatok, queue, patch, időtartam, győztes
`teams`	Egy meccs két csapata, oldal, objektíva adatok
`match_participants`	Résztvevői adatok: champion, lane, kills, deaths, assists
`player_relationships`	Játékos–játékos ally/enemy kapcsolat súllyal
`clusters`	Klaszter metaadatok, típus, algoritmus
`cluster_members`	Tagsági rekordok, szerepkör-összegzés
`analysis_runs`	Elemzési futások metaadatai
`performance_snapshots`	Játékos-teljesítmény pillanatfelvételek futásonként
`path_queries`	Útkeresési kérések és eredmények

A séma normalizált: a meccsadatok nem ismétlődnek játékosonként, a résztvevői rekord tartalmazza a meccsre vonatkozó joinolt mezőket, a klasztertagság idegen kulcson keresztül kapcsolódik a játékoshoz és a klaszterhez. A `CHECK` constraintek biztosítják, hogy az `opscore` és `feedscore` értékek a $[0, 10]$ tartományban maradnak.

22.2.1 Nézetek és összetett lekérdezések

A `docs/databaseproject/views_queries.sql` fájl elemző nézeteket és lekérdezéseket tartalmaz. Ezek közé tartoznak:

- **Szerepkörös teljesítményvizsgálat**: aggregált opscore/feedscore elemzés szerepkörönként;
- **Klaszterösszegzők**: belső sűrűség, hídjátékos-jelzők, átlagos teljesítmény klaszterenként;
- **Kapcsolat-erősségi rang**: a legerősebb ismétlődő ally/enemy pár keresése;
- **Teljesítménystabilitás időben**: teljesítmény-pillanatfelvételek összehasonlítása elemzési futásonként;
- **JSON-alapú metaadat-lekérdezések**: a klaszter `summary_json` és `role_json` mezőinek lekérdezése PostgreSQL JSON operátorokkal.

22.2.2 Tárolt eljárások és triggerek

A `docs/databaseproject/procedures_triggers.sql` tárolt eljárásokat és triggereket definiál. Ezek közé tartoznak:

- **Pontszámfrissítési trigger**: meccs-résztvevő beírásakor automatikusan frissíti az aggregált player-szintű opscore-t;
- **Klaszterépítési eljárás**: adott elemzési futáshoz klaszterező logikát futtat és klasztertagságot rögzít;
- **Útkeresés-naplózó eljárás**: útkeresési kérés és eredmény konzisztens rögzítése;
- **Teljesítménypillanatkép-karbantartó**: elemzési futásonként snapshot-ot ír, megőrizve a longitudinális adatot.

Az akadémiai szempont itt az, hogy a tárolt eljárások és triggerek az adatbázis-logikát az alkal-

mazáslogikától elválasztják: az adatmodellek önállóan karbantarthatók, és az elemzési pipeline nem függhet attól, hogy az alkalmazásréteg helyesen hívja-e meg az aggregálási logikát.

22.3 Az ASP.NET Core / C# adatbázis-böngésző

A `docs/databaseproject/simple-interface/` könyvtárban egy ASP.NET Core webalkalmazás él, amelyet a Premade Graph SQLite adatbázisainak böngészésére terveztek. A böngésző `Program.cs` fájlja mutatja, hogyan épül fel ez az eszköz.

22.3.1 Adatbázis-generálás és importálás

Az alkalmazás indulásakor automatikusan futtatja a `ProjectImporter.Import()` folyamatot, amely a Premade Graph repositoryban lévő SQLite adatbázisból importál a böngésző saját, projektre generált SQLite állományába:

- `players`: opscore, feedscore, szerepkör, meccsszám;
- `clusters`: klasztertípus, méret, metaadatok;
- `cluster_members`: tagsági rekordok;
- `replay`: útkeresési mentések;
- `replay_run`: útkeresési futások.

Az import eredménye visszajelzést ad a konzolra és webes hibaoldalon jelzi, ha az importált adatbázis nem elérhető. Ez azt jelenti, hogy a böngésző önállóan indítható, és ha az adatforrás hiányzik, explicit hibaüzenetet ad, nem csendesen üres felületet mutat.

22.3.2 Dataset-szelektálás

Az alkalmazás tudatosan illeszkedik a Premade Graph több-adatkészletes architektúrájához. A `DatasetCatalog.Load(root)` a repository `datasets.json` regiszterét olvassa be, és a böngésző minden oldalán URL-paraméterként átadható a kívánt dataset azonosítója (`?dataset=flexset`, `?dataset=soloq`).

Ez azt jelenti, hogy egyetlen böngészőpéldány az összes regisztrált adatkészlethez adatbázist tud generálni és megjeleníteni, anélkül hogy a mögöttes SQLite állományokat összekeverne.

22.3.3 Táblabögészés, szűrés, lapozás és JSON-nézetek

A böngésző a következő funkciókat valósítja meg:

- **Táblalistázás**: az összes importált tábla nevének és sorainak megjelenítése;
- **Szűrés**: szövegmező alapú szűrés játékosnévre, szereplőre, klasztertípusra;
- **Lapozás**: nagy táblák esetén oldalanként meghatározott számú sor;
- **Rendezés**: oszlop szerinti növekvő/csökkenő sorrend;
- **Teljes értéknézet**: a `/database/value` endpoint egy adott cella teljes értékét mutatja meg, különösen hasznos a `summary_json` és `role_json` típusú, hosszú JSON-oszlopoknál.

22.3.4 Technikai architektúra

Az alkalmazás egész programja egyetlen `Program.cs` fájlban helyezkedik el, minimális top-level statement stílussal. Az ASP.NET Core pipeline konfigurálása, az útvonaldefiníciók és az adatbázis-lekérdezések egységes szerkezetben élnek. A renderelés szerver oldali HTML-generálással történik, nincs JavaScript frontend: a böngésző célja az áttekinthetőség és a bemutathatóság, nem egy SPA-szerű élmény.

A `Microsoft.Data.Sqlite` NuGet csomag az adatbázis-elérés alapja, amely a SQLite adatbázisokhoz natív, közvetlen hozzáférést biztosít .NET-ből. A kapcsolatfelépítés és a lekérdezések a `ProjectDatabase` osztályon keresztül vannak absztrahálva, ami megkönnyíti a dataset-váltás és az import-logika szétválasztását.

22.4 A kiegészítő alprojekt helye a dolgozatban

22.4.1 Miért valódi alprojekt, nem csak dokumentáció?

A databaseproject nem puszta papír-terv: a `schema.sql` futtatható, a `views_queries.sql` lekérdezései tesztelhetők PostgreSQL-ben, a C# böngésző lefordul és futtatható a repository-ból. Ez teszi valódi, védhető mellékprojektté.

Az akadémiai értéke kettős:

- **Adatbázis-tervezési szempontból**: bemutatja, hogyan formalizálható ugyanaz a domain-modell egy erősebb, tárolt logikát támogató adatbázisrendszerben; ez az SQLite-on dolgozó fő rendszer korlátait is megvilágítja.
- **Szoftverarchitektúra szempontból**: demonstrálja, hogy a Premade Graph domain-modellje természetesen leírható relációs sémával, tárolt eljárásokkal és triggerekkel, miközben az alapvető fogalmak (players, clusters, cluster_members, opscore, feedscore) mindkét adatbázis-környezetben koherensek.

22.4.2 A C# böngésző mint prezentációs eszköz

A C# böngésző azért is értékes a dolgozat szempontjából, mert lehetővé teszi, hogy a SQLite-ban tárolt elemzési eredmények közvetlenül, táblázatosan és JSON-nézettel ellenőrizhetők legyenek. Ez a reprodukálható kutatás egyik alapkövetelménye: az eredmények nemcsak a frontend vizualizációján láthatók, hanem adatbázis-szinten is auditálhatók [20].

22.4.3 A databaseproject korlátai

A databaseproject PostgreSQL-sémája és a fő rendszer SQLite-adatbázisa között nincs automatikus, élő szinkronizáció. A PostgreSQL-modell akadémiai-demonstrációs célt szolgál, nem production-szinkronizációt. Ez explicit módon van dokumentálva a `docs/databaseproject/documentati` fájlban: "A beadandó adatbázis-változat PostgreSQL-re készült, mert a feladat tárolt eljárásokat, triggereket, rekurzív lekérdezéseket, JSON mezőket és összetettebb SQL elemzéseket is kér. A tényleges alkalmazás jelenleg SQLite-ot használ helyi perzisztenciára, de a dokumentált modell ugyanazokat a fogalmakat viszi át egy erősebb adatbázisrendszerbe."

Ez a szétválasztás előny, nem hiba: megmutatja, hogy a szoftvermérnöki döntés (SQLite a fő rendszerben) indokolt a project méretéhez és céljához, miközben az akadémiai adatbázistervezési kompetencia bemutatható a PostgreSQL-modellen keresztül.

23. fejezet

Jövőbeli fejlesztések

23.1 Jövőbeli fejlesztési irányok

A projekt jelenlegi állapotában már elég erős ahhoz, hogy a további fejlesztéseknél ne újabb random feature-öket, hanem célzott, tudományosan indokolható bővítéseket keressünk. Az alábbiakban csak azok az irányok szerepelnek, amelyek a kódbázisban jelenleg *nem* implementáltak, de az architekturális alapok megvannak hozzájuk.

23.1.1 Brandes-központiság benchmarkolása és artifact-integrációja

A Rust rétegben a betweenness centrality soros és `rayon`-alapú párhuzamos változata már bekerült [16]. A következő feladat nem az alapalgoritmus megírása, hanem a teljesebb valós adatkészletes benchmarkolás és a Graph V2 artifact-réteggel való szorosabb összekötés. A betweenness centralitás különösen értékes a bridge-orbit vizualizáció validálásához: az orbitscore és orbitRadius mezők vizuális heurisztikák, amelyek az igazi Brandes-középséggel összevethetők.

A további Brandes-munka ebben a kontextusban azért erős irány, mert:

- a Rust `rayon` könyvtára adatpárhuzamosságot tesz lehetővé minimális kódváltoztatással;
- a flexset gráf (5741 csúcs, 18548 él) már elég nagy ahhoz, hogy a párhuzamos verzió mérhetően gyorsabb legyen;
- az eredmény közvetlenül integrálható a Graph V2 artifact-ba mint validált centralitás-mező;
- a bridge-orbit heurisztika így nemcsak vizuális ötletként, hanem mérhető központisági összehasonlításként is értékelhető.

23.1.2 Adatvezérelt opscore-kalibráció

A jelenlegi rendszer átlátható, de kézi súlyokon alapuló szerepkör-szorzókat használ. Egy adatvezérelt kalibráció, amely a meccsnyerési arányhoz igazítja a szorzókat, erősítheti a módszertant anélkül, hogy feladná az interpretálhatóságot. Ez az irány különösen indokolt, mert a flexset és soloq dataset együtt már elég nagyméretű korpuszt jelent ehhez.

23.1.3 Erősebb nullmodellek az asszortativitáshoz

A rendszerben már van randomizációs infrastruktúra az asszortativitáshoz, de a szignifikanciakeret nem teljes. A jövőbeli fejlesztés célja permutációs null-modell futtatása: random opscore/feedscore érték-hozzárendelésekkel kapott korreláció eloszlásához viszonyítva lesz-e szignifikáns a tényleges eredmény? Ez különösen értékes a soloq dataset alacsonyabb asszortativitási értékeinek értelmezéséhez.

23.1.4 Visszavont vagy elhalasztott irányok

A dolgozat korábbi tervdokumentumaiban szerepelt néhány irány, amelyeket a jelenlegi verzióban visszavontam vagy elhalasztottam:

- **Contraction Hierarchies**: a jelenlegi gráfméret mellett nem indokolt az implementálása; az A* egzakt verziója elegendő teljesítményt ad.
- **Temporal consistency / játékos stabilitás**: eltávolítva a jelenlegi verzióból; a meccsek időrendje nem elég erős módszertani alap hosszanti stabilitási állításokhoz.
- **Community cohesion vs. performance kapcsolat**: eltávolítva a jelenlegi verzióból; az asszortativitás tisztábban, védhetőbb matematikai keretben válaszolja meg a releváns teljesítmény-hasonlósági kérdést.
- **Genetic NeuroSim v2**: kutatási kísérlet maradt; az aktuális system scope-on kívül esik.

Ezeket a visszavont irányokat azért sorolom fel, mert a tervdokumentumokban nyomuk maradt. A dolgozat konzisztenciája szempontjából fontos kimondani: ezek jelenlegi állapotban nem implementáltak, és a jelen szöveg nem tekinti őket kész feature-nek.

23.2 Zárszó

Ha a projektet egy mondatban kellene összefoglalnom, azt mondanám: a *Premade Graph* nem egyszerűen játékosstatisztikák megjelenítése, hanem egy olyan próbálkozás, amely a kompetitív játékot kapcsolati és analitikai térként kezeli. Ez a szemlélet tette lehetővé, hogy a meccsekből gráf, a gráfból asszociatív core-periphery szerkezet, a score-rétegből pedig ténylegesen vitatható és megvédhető asszortativitási és központisági kutatási kérdés legyen.

Irodalomjegyzék

[1] Riot Developer Portal Documentation.

[2] Riot Games API Documentation for League of Legends.

[3] SQLite Documentation.

[4] PyVis Documentation.

[5] NetworkX Documentation: `greedy_modularity_communities`.

[6] Heider, F. (1946). *Attitudes and Cognitive Organization*. The Journal of Psychology, 21(1), 107–112. doi:10.1080/00223980.1946.9917275.

[7] Cartwright, D., & Harary, F. (1956). *Structural Balance: A Generalization of Heider's Theory*. Psychological Review, 63(5), 277–293. doi:10.1037/h0046049.

[8] Newman, M. E. J. (2003). *Mixing Patterns in Networks*. Physical Review E, 67(2), 026126. doi:10.1103/PhysRevE.67.026126.

[9] Girvan, M., & Newman, M. E. J. (2002). *Community Structure in Social and Biological Networks*. Proceedings of the National Academy of Sciences, 99(12), 7821–7826. doi:10.1073/pnas.122653799.

[10] Fortunato, S. (2010). *Community Detection in Graphs*. Physics Reports, 486(3–5), 75–174. doi:10.1016/j.physrep.2009.11.002.

[11] Fortunato, S., & Barthelemy, M. (2007). *Resolution Limit in Community Detection*. Proceedings of the National Academy of Sciences, 104(1), 36–41. doi:10.1073/pnas.0605965104.

[12] Clauset, A., Newman, M. E. J., & Moore, C. (2004). *Finding Community Structure in Very Large Networks*. Physical Review E, 70(6), 066111. doi:10.1103/PhysRevE.70.066111.

[13] Dijkstra, E. W. (1959). *A Note on Two Problems in Connexion with Graphs*. Numerische Mathematik, 1, 269–271. doi:10.1007/BF01386390.

[14] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107. doi:10.1109/TSSC.1968.300136.

[15] Goldberg, A. V., & Harrelson, C. (2005). *Computing the Shortest Path: A* Search Meets Graph Theory*. Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms, 156–165. Microsoft Research edition.

[16] Brandes, U. (2001). *A Faster Algorithm for Betweenness Centrality*. The Journal of Mathematical Sociology, 25(2), 163–177. doi:10.1080/0022250X.2001.9990249.

[17] Leskovec, J., Huttenlocher, D., & Kleinberg, J. (2010). *Signed Networks in Social Media*. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 1361–1370. doi:10.1145/1753326.1753532.

[18] Lerner, J. (2016). *Structural Balance in Signed Networks: Separating the Probability to Interact from the Tendencies to Like and Dislike*. Advances in Complex Systems, 19(3–4), 1650004. doi:10.1142/S0219525916500048.

[19] Facchetti, G., Iacono, G., & Altafini, C. (2011). *Computing Global Structural Balance in Large-Scale Signed Social Networks*. Proceedings of the National Academy of Sciences, 108(52), 20953–20958. doi:10.1073/pnas.1109521108.

[20] Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). *Ten Simple Rules for Reproducible Computational Research*. PLoS Computational Biology, 9(10), e1003285. doi:10.1371/journal.pcbi.1003285.

[21] Premade Graph repository.

[22] Mora-Cantallops, M., & Sicilia, M.-Á. (2019). *MOBA Games: A Literature Review*. Entertainment Computing, 33, 100337. doi:10.1016/j.entcom.2019.100337.

[23] Bahrololloomi, F., Sauer, S., Klonowski, M., Horst, R., & Dörner, R. (2021). *A Machine Learning Based Analysis of e-Sports Player Performances in League of Legends for Winning Prediction Based on Player Roles*. Proceedings of the 16th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications.

[24] Novak, A. R., Bennett, K. J. M., Pluss, M. A., & Fransen, J. (2022). *Performance Analysis in Esports: Modelling Performance at the 2018 League of Legends World Championship*. International Journal of Sports Science & Coaching, 17(2), 315–325. doi:10.1177/17479541211005510.

[25] Ghawi, R., Müller, J., & Pfeffer, J. (2019). *Team Communication Networks and Performance in Online Games*. Proceedings of the ACM CHI Conference on Human Factors in Computing Systems.

[26] Sánchez-González, P., & González-Bailón, S. (2023). *Network Structure and Team Performance in Professional League of Legends*. Journal of Computational Social Science, 6, 225–248.

[27] Microsoft. *ASP.NET Core Documentation.* docs.microsoft.com/aspnet/core.

[28] The PostgreSQL Global Development Group. *PostgreSQL Documentation.* www.postgresql.org/docs.

[29] Klabnik, S., & Nichols, C. (2019). *The Rust Programming Language*. No Starch Press.